JAVA

CHEAT SHEETS

A Complete Reference for Modern Java

Syntax & Types · Control Flow · Methods & Lambdas · Collections · Streams · OOP & Records · Files & NIO · Maven & Gradle · java.time · Testing · Concurrency · JDBC & JPA · Swing · JavaFX

136 REFERENCE SHEETS

CONTENTS

LANGUAGE CORE

01–20 Basics · 20
21–33 Control Flow · 13
34–43 Methods & Functions · 10
44–57 Data Structures · 14

OBJECTS, I/O & LIBRARY

58–75 Object-Oriented Programming · 18
76–83 Files & I/O · 8
84–87 Modules & Packages · 4
88–101 Standard Library · 14

ECOSYSTEM & GUI

102–111 External Libraries · 10
112–116 Concurrency · 5
117–122 Database · 6
123–125 GUI — Swing · 3
126–134 GUI — JavaFX · 9
135–136 Quick Reference

JAVA

Table of Contents

136 sheets · syntax · examples · notes

BASICS 01 – 20

01 Introduction to Java
02 Program Structure
03 Output Statements
04 Comments & Javadoc
05 Variables & Constants
06 Primitive Data Types
07 Type Conversion & Ca…
08 Autoboxing & Wrapper…
09 Arithmetic Operators
10 Comparison Operators
11 Logical Operators
12 Bitwise Operators
13 Assignment Operators
14 String Basics
15 String Methods
16 String Formatting
17 Scanner Input
18 BufferedReader Input
19 Advanced Strings
20 Arrays — 1D

CONTROL FLOW 21 – 33

21 If / Else
22 Ternary Operator
23 Switch — Classic
24 Switch — Expressions …
25 For Loop
26 While Loop
27 Do-While Loop
28 Enhanced For Loop
29 Break & Continue
30 Try / Catch
31 Throws & Checked Exc…
32 Custom Exceptions
33 Finally & Try-with-Res…

METHODS & FUNCTIONS 34 – 43

34 Defining & Calling Met…
35 Parameters & Argume…
36 Method Overloading
37 Return Values
38 Variable Scope
39 Static vs Instance Met…
40 Lambda Expressions
41 Method References
42 Recursion
43 Varargs

DATA STRUCTURES 44 – 57

44 Arrays — 2D & Multi
45 ArrayList
46 LinkedList
47 HashMap
48 HashSet
49 TreeMap & TreeSet
50 Stack & Queue
51 Deque
52 Generics
53 Streams
54 map & filter
55 reduce & Collectors
56 Sorting & Searching
57 Iterators & Iterables

OBJECT-ORIENTED PROGRAMMING 58 – 75

58 Classes & Objects
59 Constructors
60 The this Keyword
61 Instance vs Static Me…
62 Inheritance
63 The super Keyword
64 Method Overriding
65 Polymorphism
66 Encapsulation
67 Getters & Setters
68 Abstract Classes
69 Interfaces
70 implements vs extends
71 Enums
72 Records (Java 16+)
73 Sealed Classes (Java 1…
74 Annotations & Reflecti…
75 Inner & Anonymous Cl…

FILES & I/O 76 – 83

76 FileReader & FileWriter
77 Buffered I/O
78 Files & NIO
79 File Paths & Directories
80 JSON in Java
81 CSV in Java
82 Serialization
83 Byte Streams & I/O Str…

MODULES & PACKAGES 84 – 87

84 Packages & Imports
85 Creating Packages & M…
86 Maven
87 Gradle

STANDARD LIBRARY 88 – 101

88 Math Class
89 Random Numbers
90 StringBuilder
91 LocalDate
92 LocalTime
93 LocalDateTime & Zones
94 DateTimeFormatter
95 Collections Utility Class
96 Arrays Utility Class
97 Optional
98 System Class
99 Regex — Pattern & Mat…
100 Scanner — Advanced
101 Comparable & Comp…

EXTERNAL LIBRARIES 102 – 111

102 JUnit 5 — Basics
103 JUnit 5 — Advanced
104 Mockito
105 Logging — SLF4J & Lo…
106 HttpClient (Built-in)
107 OkHttp
108 Gson
109 Jackson
110 Lombok
111 OpenCSV

CONCURRENCY 112 – 116

112 Threads
113 ExecutorService
114 CompletableFuture
115 Synchronization & Lo…
116 Virtual Threads (Java …

DATABASE 117 – 122

117 JDBC Basics
118 JDBC CRUD Operatio…
119 PreparedStatement
120 Connection Pooling (…
121 Hibernate & JPA Basi…
122 Hibernate CRUD & JP…

GUI — SWING 123 – 125

123 Swing Basics
124 Swing Layout Manage…
125 Swing Event Handling

GUI — JAVAFX 126 – 134

126 JavaFX Basics
127 JavaFX Layouts
128 JavaFX Controls
129 JavaFX Events & Thre…
130 Properties & Binding
131 FXML
132 FXML Controllers
133 JavaFX CSS Styling
134 JavaFX Charts & Media

QUICK REFERENCE 135 – 136

135 Quick Reference — S…
136 Quick Reference — C…

Introduction to Java

WHAT IS JAVA

Java is a statically typed, class-based, object-oriented language created by James Gosling at Sun Microsystems and released in 1995. Source code compiles to **bytecode** (`.class` files) that runs on the **Java Virtual Machine**, so the same compiled program runs on Windows, macOS, and Linux. Oracle has owned Java since 2010 and ships a new version every six months, with a **Long-Term Support** release every two years.

KEY FACTS

- Motto: "Write once, run anywhere" (WORA)
- Current LTS releases: Java 17, 21, and 25
- Primary uses: backend services, Android, enterprise apps, big data (Kafka, Spark), desktop tools
- Memory is managed by a **garbage collector**; no manual `free`

HELLO, WORLD

```
// File name must match the public class: Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

// Terminal
// javac Hello.java      → creates Hello.class (bytecode)
// java Hello            → Hello, World!
// java Hello.java       → compile + run in one step (Java 11+)
```

HOW IT RUNS

- → `javac` — compiler; turns `.java` source into `.class` bytecode
- → `java` — launcher; starts the JVM and runs `main`
- → **Bytecode** — platform-neutral instructions the JVM executes
- → **JIT** — the JVM compiles hot bytecode to native machine code at runtime
- → `java File.java` — single-file source launch, no separate compile

JDK VS JRE VS JVM

TERM	CONTAINS	YOU NEED IT TO…
`JVM`	Bytecode interpreter + JIT + GC	Execute `.class` files
`JRE`	JVM + core class libraries	Run Java apps (no longer shipped separately)
`JDK`	JRE + `javac`, `jar`, `jshell`, `javadoc`	Write and compile Java

FILE EXTENSIONS

EXTENSION	MEANING
`.java`	Source code file
`.class`	Compiled bytecode
`.jar`	Zipped archive of classes + resources
`.jmod`	Module packaging format (JDK)

⚠ COMMON MISTAKES

- Naming the file differently from its `public class` — `javac` refuses to compile it
- Running `java Hello.class` — the launcher wants the class name, not the file name
- Installing only a JRE build and then wondering why `javac` is missing

✓ TIPS & BEST PRACTICES

- Stick to an **LTS** release (17, 21, 25) for real projects
- Use `jshell` to try one-liners without writing a class
- Run `java -version` and `javac -version` to confirm both match

QUICK SUMMARY

- `javac` — compile source to bytecode
- `java` — run bytecode on the JVM
- `JDK` — everything a developer needs
- `LTS` — the versions to target in production

Program Structure

ANATOMY OF A JAVA PROGRAM

Every Java program lives inside a **class**. Execution starts at the `main` method, which must be `public static void` and take a `String[]`. Statements end with semicolons, blocks use **curly braces**, and indentation is for readers only. Optional `package` and `import` lines always come first, in that order.

RULES

- One `public` class per file, and the file name must match it
- Java is **case-sensitive**: `Main` ≠ `main`
- Java 21+ preview / Java 25: `void main() { }` works in a single-file program
- Class names use `PascalCase`; methods and variables use `camelCase`

ANNOTATED BOILERPLATE

```
package com.example.app;            // 1. package (optional, first line)

import java.util.List;              // 2. imports
import java.util.Scanner;

public class Main {                 // 3. class declaration
    static final int MAX = 10;      // 4. fields (class members)

    public static void main(String[] args) {   // 5. entry point
        System.out.println("args: " + args.length);
        greet("Ada");               // 6. call another method
    }

    static void greet(String name) {           // 7. other methods
        System.out.println("Hi, " + name);
    }
}
```

LINE BY LINE

- → `package` — namespace; must match the folder path
- → `import` — lets you use `List` instead of `java.util.List`
- → `public` — visible to the JVM launcher
- → `static` — runs without creating a `Main` object
- → `void` — `main` returns nothing
- → `String[] args` — command-line arguments

MAIN SIGNATURES

SIGNATURE	VALID?
`public static void main(String[] args)`	Yes, classic
`public static void main(String... args)`	Yes, varargs form
`static public void main(String[] a)`	Yes, order of modifiers is free
`void main()`	Java 25 instance main
`public void main(String[] args)`	No main found before Java 21

COMPILE & RUN STEPS

COMMAND	RESULT
`javac Main.java`	Creates Main.class
`javac -d out src/com/example/app/Main.java`	Output into out/ with package dirs
`java -cp out com.example.app.Main`	Run a packaged class
`java Main a b c`	args = ["a","b","c"]

⚠ COMMON MISTAKES

- Forgetting a semicolon or closing brace — the error often points at the **next** line
- Putting `import` above `package` — `package` must be the first statement
- Writing `Main.main` logic in a non-static method and calling it from `main` without an object

✓ TIPS & BEST PRACTICES

- Keep `main` short: parse args, then call well-named methods
- Mirror packages in folders: `com/example/app/Main.java`
- Let your IDE format braces; mixed styles hurt readability

QUICK SUMMARY

- `package` → `import` → `class` — fixed order
- `main` — `public static void main(String[] args)`
- `{ }` — blocks; `;` — ends statements
- File name = public class name

Output Statements

WRITING TO THE CONSOLE

`System.out` is a `PrintStream` connected to standard output. `println` prints and adds a newline, `print` does not, and `printf` formats values with **format specifiers**. `System.err` writes to standard error, which many IDEs show in red and which can be redirected separately.

NOTES

- `println()` with no argument prints a blank line
- Any type can be printed; objects call `toString()`
- `%n` in `printf` is the platform-correct newline
- `printf` and `String.format` share the same specifier syntax

OUTPUT VARIANTS

```
System.out.println("Hello");           // Hello⏎
System.out.print("A"); System.out.print("B");  // AB
System.out.println(42);                // 42
System.out.println(3.0 / 2);           // 1.5
System.out.println('A' + 1);           // 66   (char math → int)
System.out.println("A" + 1);           // A1   (string concat)
System.out.println(1 + 2 + "x");       // 3x   (left to right)
System.out.println("x" + 1 + 2);       // x12
System.out.printf("%s is %d%n", "Ada", 36);   // Ada is 36
System.out.printf("%.2f%n", Math.PI);          // 3.14
System.out.println("Tab\there");       // Tab     here
System.out.println("Quote: \"hi\"");    // Quote: "hi"
System.err.println("Error!");          // goes to stderr
```

NOTES ON THE EXAMPLE

- → `+` with a `String` on either side means **concatenation**
- → `char + int` is arithmetic: `'A'` is 65
- → `%s` any value, `%d` integer, `%f` decimal
- → `.2f` — two decimal places
- → `System.err` — unbuffered error stream

ESCAPE SEQUENCES

ESCAPE	OUTPUT
`\n`	Newline
`\t`	Tab
`\"`	Double quote
`\'`	Single quote
`\\`	Backslash
`\u00e9`	Unicode char (é)

COMMON PRINTF SPECIFIERS

SPEC	EXAMPLE	OUTPUT
`%d`	printf("%d", 7)	7
`%5d`	printf("%5d", 7)	`····7`
`%-5s|`	printf("%-5s|", "ab")	`ab···|`
`%,d`	printf("%,d", 1234567)	1,234,567
`%.3f`	printf("%.3f", 2.0)	2.000
`%b`	printf("%b", true)	true

⚠ COMMON MISTAKES

- `"Sum: " + 1 + 2` prints `Sum: 12` — wrap math in parentheses: `"Sum: " + (1 + 2)`
- Using `\n` in `printf` on Windows tools that expect `\r\n` — prefer `%n`
- Passing an `int` to `%f` throws `IllegalFormatConversionException`

✓ TIPS & BEST PRACTICES

- Use `printf` for tables of numbers; width specifiers align columns
- Send diagnostics to `System.err` so they don't mix with program output
- For real apps, use a logger (Sheet 105) instead of `println`

QUICK SUMMARY

- `println` — print + newline
- `print` — no newline
- `printf` — formatted output with `%` specifiers
- `System.err` — error stream

Comments & Javadoc

THREE KINDS OF COMMENTS

Java has **single-line** comments (`//`), **block** comments (`/* */`), and **Javadoc** comments (`/** */`). The compiler ignores all three. Javadoc comments placed directly above a class, field, or method are read by the `javadoc` tool to generate HTML API docs and by IDEs to show hover documentation.

NOTES

- Block comments **cannot be nested**: `/* /* */ */` breaks
- Javadoc's first sentence becomes the summary line
- Javadoc accepts HTML tags (`<p>` , `<code>`) and `{@code ...}`

COMMENT STYLES

```java
// Single-line comment
int count = 0;   // trailing comment

/*
 * Block comment spanning
 * several lines.
 */

/**
 * Returns the area of a circle.
 *
 * @param radius the radius in centimeters; must be ≥ 0
 * @return the area in square centimeters
 * @throws IllegalArgumentException if radius is negative
 * @see Math#PI
 */
public static double area(double radius) {
    if (radius < 0) throw new IllegalArgumentException("radius");
    return Math.PI * radius * radius;
}
```

WHERE JAVADOC SHOWS UP

- → IDE hover on `area(` shows the summary and params
- → `javadoc -d docs Shapes.java` builds an HTML site
- → `@param` — one line per parameter
- → `@return` — omit for `void` methods
- → `@throws` — documents exceptions callers should expect

JAVADOC TAGS

TAG	PURPOSE
`@param name`	Describe a parameter
`@return`	Describe the return value
`@throws Type`	Exception and when it occurs
`@see Ref`	Cross-reference link
`@since 1.2`	Version introduced
`@deprecated`	Mark as obsolete (pair with `@Deprecated`)
`{@code x}`	Inline code, no HTML parsing

SHORTCUTS

ACTION	INTELLIJ	VS CODE
`Toggle line comment`	Ctrl+/	Ctrl+/
`Toggle block comment`	Ctrl+Shift+/	Shift+Alt+A
`Generate Javadoc stub`	Type /** + Enter	Type /** + Enter

⚠ COMMON MISTAKES

- Commenting **what** the code does instead of **why** — the code already says what
- Leaving commented-out code in commits; version control keeps history
- Writing `/* ... */` inside an existing block comment to disable code — it ends the outer comment early

✓ TIPS & BEST PRACTICES

- Write Javadoc for every `public` API; skip it for obvious private helpers
- Start the summary with a verb: "Returns…", "Creates…"
- Use `{@code null}` instead of plain text so it renders as code

QUICK SUMMARY

- `//` — rest of the line
- `/* */` — block, not nestable
- `/** */` — Javadoc for docs and IDE hovers
- `@param` `@return` `@throws` — core tags

Variables & Constants

DECLARING VARIABLES

A variable is a named storage slot with a fixed **type**. You declare it with the type and name, optionally assigning an initial value. Since Java 10, `var` lets the compiler **infer** the type of a local variable from its initializer, but the type is still fixed. `final` makes a variable assignable only once.

NOTES

- Local variables have no default value; reading one before assignment is a compile error
- Fields get defaults: `0`, `false`, `null`
- `var` only works for **local** variables with an initializer
- Constants: `static final` + `UPPER_SNAKE_CASE`

DECLARATION FORMS

```
int age;                    // declare
age = 30;                   // assign
int year = 2026;            // declare + initialize
int a = 1, b = 2, c;        // several at once (c unassigned)
double price = 9.99;
String name = "Ada";

var count = 10;             // inferred: int
var list = new ArrayList<String>();   // ArrayList<String>
// var x;                   // ✖ no initializer
// var y = null;            // ✖ type cannot be inferred

final int LIMIT = 5;
// LIMIT = 6;               // ✖ cannot assign a final variable

static final double TAX_RATE = 0.08;  // class-level constant
```

KEY POINTS

- → Type comes first: `int age`
- → `var` is not dynamic typing; `count` stays `int`
- → `final` locals can be assigned later, but only once
- → `static final` — one shared, unchangeable value

NAMING CONVENTIONS

KIND	STYLE	EXAMPLE
`Local / field`	camelCase	totalPrice
`Constant`	UPPER_SNAKE	MAX_SIZE
`Class`	PascalCase	OrderService
`Package`	lowercase	com.acme.billing
`Boolean`	is/has prefix	isActive

LEGAL VS ILLEGAL NAMES

NAME	LEGAL?
`_count`	Yes, but avoid
`$value`	Yes, reserved for generated code
`2fast`	No, starts with digit
`my-var`	No, hyphen
`class`	No, keyword
`_`	No, reserved since Java 9

⚠ COMMON MISTAKES

- Using `var` where the type is unclear: `var data = load();` hides what `data` is
- Shadowing a field with a same-named local, then wondering why the field never changes
- Assuming `final` makes an object immutable — `final List` can still be modified

✓ TIPS & BEST PRACTICES

- Declare variables as close as possible to first use
- Prefer `final` for values that never change; it documents intent
- Use `var` when the right side already names the type: `var map = new HashMap<String,Integer>();`

QUICK SUMMARY

- `type name = value;` — declaration
- `var` — local type inference
- `final` — assign once
- `static final` — class constant

Primitive Data Types

byte · short · int · long · float · double · char · boolean · literals

THE EIGHT PRIMITIVES

Java has **eight primitive types** that hold raw values directly, not object references. Integer types differ only in size and range; `float` and `double` are IEEE 754 floating point; `char` is a 16-bit UTF-16 code unit; `boolean` is `true` or `false` and cannot be treated as a number. Everything else is a reference type.

NOTES

- Integer literals are `int` by default; add `L` for `long`
- Decimal literals are `double` by default; add `f` for `float`
- Underscores improve readability: `1_000_000`
- Integer overflow **wraps silently**: no exception

PRIMITIVE TYPES

TYPE	SIZE	RANGE / VALUES	DEFAULT	LITERAL
byte	8 bit	-128 to 127	0	(byte) 10
short	16 bit	-32,768 to 32,767	0	(short) 10
int	32 bit	±2.1 billion ($2^{31}-1$)	0	42
long	64 bit	±9.2 × 10^{18}	0L	42L
float	32 bit	~7 decimal digits	0.0f	3.14f
double	64 bit	~15–16 decimal digits	0.0	3.14, 1e-3
char	16 bit	'\u0000' to '\uffff'	'\u0000'	'A', '\n'
boolean	JVM-defined	true / false	false	true

LITERALS & EDGE CASES

```
int hex = 0xFF;             // 255
int bin = 0b1010;           // 10
int oct = 017;              // 15  (leading 0 = octal!)
long big = 9_000_000_000L; // needs L
float f = 1.5f;             // needs f
double d = 1.5;
char c = 'A';               // 65 as a number
char u = '\u03A9';           // Ω
int max = Integer.MAX_VALUE;   // 2147483647
max + 1;                    // -2147483648 (overflow)
0.1 + 0.2;                  // 0.30000000000000004
1.0 / 0;                    // Infinity
0.0 / 0;                    // NaN
```

EDGE CASES

- → `017` is octal; never pad decimals with zeros
- → `Math.addExact(a, b)` throws on overflow instead of wrapping
- → `double` cannot represent 0.1 exactly; use `BigDecimal` for money
- → `Double.isNaN(x)` — `NaN == NaN` is `false`

⚠ COMMON MISTAKES

- `long ms = 24 * 60 * 60 * 1000 * 365;` overflows as `int` before assignment; write `365L`
- Comparing doubles with `==` after arithmetic; compare with a tolerance
- Writing `float f = 1.5;` — a `double` literal won't narrow implicitly

✓ TIPS & BEST PRACTICES

- Default to `int` and `double`; use others when size or range demands it
- Use `long` for timestamps and IDs
- Use `BigDecimal` for currency, `BigInteger` for huge integers

QUICK SUMMARY

- `int` `long` — whole numbers
- `double` `float` — decimals
- `char` — one UTF-16 unit
- `boolean` — true / false only

Type Conversion & Casting

CONVERTING BETWEEN TYPES

Widening conversions (small → large, like `int` → `long`) happen automatically because no data is lost. **Narrowing** conversions (large → small, like `double` → `int`) require an explicit **cast** and may truncate or wrap. Converting between strings and numbers uses methods such as `Integer.parseInt` and `String.valueOf`, not casts.

NOTES

- Casting `double` → `int` **truncates** toward zero; it does not round
- Arithmetic promotes `byte`, `short`, `char` to `int`
- `parseInt` throws `NumberFormatException` on bad input
- You cannot cast between `boolean` and any number

CONVERSION EXAMPLES

```
int i = 100;
long l = i;                    // widening: 100
double d = i;                  // widening: 100.0

double price = 9.99;
int whole = (int) price;       // 9     (truncated)
int neg = (int) -9.99;         // -9
long r = Math.round(price);    // 10    (rounds)
byte b = (byte) 200;           // -56   (wraps)
char c = (char) 66;            // 'B'
int code = 'B';                // 66

int n = Integer.parseInt("42");          // 42
double x = Double.parseDouble("3.5");    // 3.5
String s1 = String.valueOf(42);          // "42"
String s2 = Integer.toString(255, 16);   // "ff"
int fromHex = Integer.parseInt("ff", 16); // 255
```

NOTES ON THE EXAMPLE

- → `(type)` — cast operator; you accept possible data loss
- → `Math.round(double)` returns `long`
- → `(byte) 200` keeps the low 8 bits → -56
- → Radix argument converts to/from other bases

WIDENING ORDER (AUTOMATIC)

FROM	WIDENS TO
`byte`	short, int, long, float, double
`short / char`	int, long, float, double
`int`	long, float, double
`long`	float, double
`float`	double

STRING ↔ NUMBER

GOAL	METHOD
`String → int`	Integer.parseInt(s)
`String → double`	Double.parseDouble(s)
`String → boolean`	Boolean.parseBoolean(s)
`number → String`	String.valueOf(n)
`number → String`	"" + n (shortcut)
`char → digit`	Character.getNumericValue('7')

⚠ COMMON MISTAKES

- `byte b = 10; b = b + 1;` fails: `b + 1` is `int`. Use `b += 1` or a cast
- `(int) "42"` — strings can't be cast to numbers; use `parseInt`
- Expecting `(int) 2.9` to give 3 — it gives 2

✓ TIPS & BEST PRACTICES

- Use `Math.round`, `Math.floor`, or `Math.ceil` when you mean rounding
- Wrap `parseInt` in `try/catch NumberFormatException` for user input
- `int → long → float` can lose precision on huge values; check if it matters

QUICK SUMMARY

- Widening — automatic, safe
- `(type)` — explicit narrowing cast
- `parseXxx` — String → number
- `String.valueOf` — anything → String

Sheet 08

Autoboxing & Wrapper Types

Integer · Double · Boolean · Character · valueOf · unboxing · Integer cache · null

WRAPPER CLASSES

Each primitive has a **wrapper class** (`int` → `Integer`, `double` → `Double`) that holds the value as an object. Collections and generics only accept objects, so Java converts automatically: **autoboxing** wraps a primitive, **unboxing** unwraps it. The convenience hides costs and two traps: `null` unboxing and == on wrappers.

NOTES

- `List<int>` is illegal; use `List<Integer>`
- Unboxing `null` throws `NullPointerException`
- `Integer.valueOf` caches values from **-128 to 127**
- Wrapper objects are **immutable**

BOXING IN ACTION

```
Integer boxed = 42;              // autobox: Integer.valueOf(42)
int plain = boxed;               // unbox:   boxed.intValue()

List<Integer> nums = new ArrayList<>();
nums.add(5);                     // autoboxed
int first = nums.get(0);         // unboxed

Integer a = 127, b = 127;
a == b;                          // true  (cached)
Integer c = 128, d = 128;
c == d;                          // false (different objects!)
c.equals(d);                     // true  ✔

Integer missing = null;
int boom = missing;              // NullPointerException

Integer.MAX_VALUE;               // 2147483647
Integer.compare(3, 7);           // -1
Character.isDigit('7');          // true
```

NOTES ON THE EXAMPLE

- → `==` compares **references** for wrappers
- → The cache makes small-number bugs invisible in testing
- → `map.get(key)` returns `null` when absent → NPE if unboxed
- → Static helpers live on the wrapper classes

PRIMITIVE ↔ WRAPPER

PRIMITIVE	WRAPPER	UNBOX METHOD
`int`	Integer	intValue()
`long`	Long	longValue()
`double`	Double	doubleValue()
`boolean`	Boolean	booleanValue()
`char`	Character	charValue()
`byte / short / float`	Byte / Short / Float	byteValue() …

USEFUL STATIC HELPERS

METHOD	RESULT
`Integer.parseInt("12")`	12 (int)
`Integer.valueOf("12")`	12 (Integer)
`Integer.toBinaryString(5)`	"101"
`Integer.sum(2, 3)`	5
`Double.isNaN(x)`	true / false
`Character.toUpperCase('a')`	'A'

⚠ COMMON MISTAKES

- Comparing `Integer` objects with `==` — use `.equals()` or unbox first
- `int count = map.get("x");` when the key is missing throws NPE; use `getOrDefault`
- Summing boxed values in a hot loop creates millions of objects; use `int`

✓ TIPS & BEST PRACTICES

- Use primitives for local math, wrappers only where objects are required
- Prefer `Integer.valueOf` over `new Integer(...)` (constructor is deprecated)
- Use `IntStream` / `mapToInt` to avoid boxing in streams

QUICK SUMMARY

- Autoboxing — primitive → wrapper
- Unboxing — wrapper → primitive (NPE on `null`)
- `.equals()` — compare wrapper values
- Cache: -128 … 127

Arithmetic Operators

DOING MATH

Java's arithmetic operators work on all numeric primitives. The key rule: if **both operands are integers, the result is an integer**, so `/` discards the remainder. `%` returns the remainder and keeps the sign of the left operand. `++` and `--` change a variable by one, with different results depending on prefix or postfix position.

NOTES

- `int / int` → `int`; make one operand `double` to keep decimals
- Integer division by zero throws `ArithmeticException`
- Floating division by zero gives `Infinity` or `NaN`
- `* / %` bind tighter than `+ -`

ARITHMETIC OPERATORS

OPERATOR	NAME	EXAMPLE	RESULT
`+`	Addition	7 + 2	9
`-`	Subtraction	7 - 2	5
`*`	Multiplication	7 * 2	14
`/`	Division (int)	7 / 2	3
`/`	Division (double)	7.0 / 2	3.5
`%`	Remainder	7 % 2	1
`%`	Remainder (negative)	-7 % 2	-1
`-`	Unary minus	-x	negated

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
1	++ -- (postfix)
2	++ -- + - ! ~ (unary)
3	* / %
4	+ -
5	<< >> >>>
6	= += -= … (assignment)

INCREMENT, DECREMENT & PRECEDENCE

```java
int x = 5;
int a = x++;      // a = 5, x = 6   (use, then increment)
int b = ++x;      // b = 7, x = 7   (increment, then use)
int c = x--;      // c = 7, x = 6
int d = --x;      // d = 5, x = 5

int r1 = 2 + 3 * 4;         // 14
int r2 = (2 + 3) * 4;       // 20
int r3 = 10 - 4 - 3;        // 3   (left to right)
double avg = (3 + 4) / 2;   // 3.0  ✖ int division first
double ok  = (3 + 4) / 2.0; // 3.5  ✔
int last = 1234 % 10;       // 4   (last digit)
boolean even = 8 % 2 == 0; // true
Math.floorMod(-7, 3);       // 2   (always non-negative)
```

NOTES

- → Prefix vs postfix only matters when the value is used in the same expression
- → `(3 + 4) / 2` is computed as `int` before widening to `double`
- → `Math.floorMod` gives a true modulo for negatives

⚠ COMMON MISTAKES

- `double avg = sum / count;` with two ints truncates — cast: `(double) sum / count`
- `x = x++;` leaves `x` unchanged — postfix returns the old value
- Using `%` to test odd numbers with `n % 2 == 1` fails for negatives; use `n % 2 != 0`

✓ TIPS & BEST PRACTICES

- Use parentheses whenever precedence isn't obvious to a reader
- Keep `++` / `--` on their own line or in a `for` header
- `Math.addExact` / `multiplyExact` throw on overflow

QUICK SUMMARY

- `/` on ints — truncates
- `%` — remainder, sign of left operand
- `x++` — old value; `++x` — new value
- `* / %` before `+ -`

Comparison Operators

COMPARING VALUES

Relational operators return a `boolean`. For **primitives**, == compares values. For **objects**, == compares **references**: it is true only when both variables point to the same object. To compare object contents, use `.equals()`; to order objects, use `compareTo` or a `Comparator`.

NOTES

- `<` `>` `<=` `>=` only work on numeric primitives and `char`
- `String` must be compared with `.equals()`
- `Objects.equals(a, b)` is null-safe
- `Double.NaN` is not equal to anything, including itself

RELATIONAL OPERATORS

OPERATOR	MEANING	EXAMPLE	RESULT
==	Equal to	5 == 5	true
!=	Not equal	5 != 3	true
<	Less than	3 < 5	true
>	Greater than	3 > 5	false
<=	Less or equal	5 <= 5	true
>=	Greater or equal	4 >= 5	false

WHICH COMPARISON TO USE

COMPARING	USE
`int, double, char`	== != < >
`String contents`	s.equals(t)
`Possibly null objects`	Objects.equals(a, b)
`Ordering strings`	s.compareTo(t)
`Enums`	== (safe, one instance each)
`Doubles approximately`	Math.abs(a - b) < eps

REFERENCE VS VALUE

```
String a = "hi";
String b = "hi";
String c = new String("hi");
a == b;                    // true   (same pooled literal)
a == c;                    // false  (different object)
a.equals(c);               // true   ✔ compare contents
"HI".equalsIgnoreCase(a); // true

Objects.equals(null, null);  // true  (no NPE)
"apple".compareTo("banana"); // negative (a < b)
"b".compareTo("a");          // positive
Integer.compare(10, 3);      // positive

'a' < 'b';                 // true   (compares char codes)
0.1 + 0.2 == 0.3;          // false
Math.abs((0.1 + 0.2) - 0.3) < 1e-9;  // true ✔
```

NOTES

- → String literals are **interned**, which is why `a == b` can mislead you
- → `compareTo` returns negative / 0 / positive
- → Compare floating point with a tolerance (epsilon)

⚠ COMMON MISTAKES

- `if (name == "admin")` — works by accident with literals, fails with input; use `.equals`
- `input.equals("yes")` throws NPE if `input` is null; write `"yes".equals(input)`
- Chaining `1 < x < 10` doesn't compile; write `1 < x && x < 10`

✓ TIPS & BEST PRACTICES

- Put the literal first in `equals` to avoid NPEs
- Override `equals` **and** `hashCode` together in your own classes
- Use `Comparator.comparing(...)` for custom ordering (Sheet 101)

QUICK SUMMARY

- `==` — value for primitives, identity for objects
- `.equals()` — object contents
- `compareTo` — ordering (−, 0, +)
- Floats — compare with tolerance

Logical Operators

COMBINING CONDITIONS

Logical operators combine `boolean` values. && (AND) and || (OR) **short-circuit**: they stop evaluating as soon as the result is known, which lets you guard a risky check behind a safe one. The non-short-circuit forms & and | always evaluate both sides, and ^ is exclusive OR.

NOTES

- Operands must be `boolean`; `if (1)` does not compile in Java
- `!` binds tighter than `&&`, which binds tighter than `||`
- Short-circuiting is the standard null-guard pattern
- `^` on booleans is true when exactly one side is true

LOGICAL OPERATORS

OPERATOR	NAME	EXAMPLE	RESULT
&&	AND (short-circuit)	true && false	false
||	OR (short-circuit)	true || false	true
!	NOT	!true	false
&	AND (both sides evaluated)	true & false	false
|	OR (both sides evaluated)	false | true	true
^	XOR	true ^ true	false

TRUTH TABLE

A	B	A && B	A || B	A ^ B
true	true	true	true	false
true	false	false	true	true
false	true	false	true	true
false	false	false	false	false

SHORT-CIRCUIT IN PRACTICE

```
String s = null;
if (s != null && s.length() > 3) { }   // safe: right side skipped
if (s.length() > 3 && s != null) { }   // ✖ NullPointerException

int i = 0;
boolean r1 = false && (++i > 0);        // i stays 0
boolean r2 = false &  (++i > 0);        // i becomes 1

boolean inRange = age >= 18 && age <= 65;
boolean weekend = day == SAT || day == SUN;
boolean ok = !(a || b);                 // same as !a && !b
boolean mix = a || b && c;              // a || (b && c)
```

NOTES

- → Put the cheap or null check **first** in `&&`
- → `&` and `|` are mainly for bitwise math (Sheet 12)
- → De Morgan: `!(a && b)` ≡ `!a || !b`
- → Add parentheses when mixing `&&` and `||`

⚠ COMMON MISTAKES

- Writing `&` when you meant `&&` — both sides run, and null guards stop working
- `if (x > 0 || x < 10)` is always true; you probably meant `&&`
- `if (flag == true)` is redundant; write `if (flag)`

✓ TIPS & BEST PRACTICES

- Extract long conditions into named booleans: `boolean canVote = ...`
- Order `&&` checks from cheapest to most expensive
- Use `Objects.nonNull` / `Objects.isNull` in stream filters

QUICK SUMMARY

- `&&` — both true, short-circuits
- `||` — either true, short-circuits
- `!` — invert
- `^` — exactly one true

Bitwise Operators

WORKING WITH BITS

Bitwise operators act on the individual bits of integer types (`int`, `long`, and smaller types promoted to `int`). They are used for **flags**, **masks**, hashing, low-level protocols, and fast multiplication or division by powers of two. Java integers are **two's complement**, which matters for ~ and right shifts.

NOTES

- `>>` keeps the sign bit (arithmetic shift)
- `>>>` fills with zeros (logical shift); no `<<<` exists
- `int` shifts use only the low 5 bits of the distance: `1 << 32 == 1`
- `~x` equals `-x - 1`

BITWISE OPERATORS

OPERATOR	NAME	EXAMPLE (A=12, B=10)	RESULT
`&`	AND	a & b	8
`|`	OR	a | b	14
`^`	XOR	a ^ b	6
`~`	NOT	~a	-13
`<<`	Left shift	a << 2	48
`>>`	Signed right shift	-16 >> 2	-4
`>>>`	Unsigned right shift	-16 >>> 28	15

WORKED EXAMPLE (8 BITS SHOWN)

EXPRESSION	BITS	DECIMAL
`a`	0000 1100	12
`b`	0000 1010	10
`a & b`	0000 1000	8
`a | b`	0000 1110	14
`a ^ b`	0000 0110	6
`a << 1`	0001 1000	24

FLAGS & MASKS

```
final int READ = 1, WRITE = 2, EXEC = 4;     // 001, 010, 100
int perms = READ | WRITE;                    // 3   (set flags)
boolean canWrite = (perms & WRITE) != 0;     // true (test flag)
perms &= ~WRITE;                             // 1   (clear flag)
perms ^= EXEC;                               // 5   (toggle flag)

int x = 7;
boolean odd = (x & 1) == 1;                  // true
int times8 = x << 3;                         // 56
int half = x >> 1;                           // 3
Integer.toBinaryString(10);                  // "1010"
Integer.bitCount(255);                       // 8
int lowByte = 0xABCD & 0xFF;                 // 205 (0xCD)
```

PATTERNS

- → `|` sets, `& ~` clears, `^` toggles, `&` tests
- → `(x & 1)` — fastest odd/even test
- → `& 0xFF` — keep the low 8 bits
- → For readable flags, prefer `EnumSet` (Sheet 71)

⚠ COMMON MISTAKES

- `if (perms & WRITE)` doesn't compile — compare to `0`
- Forgetting parentheses: `perms & WRITE != 0` parses as `perms & (WRITE != 0)`
- Using `>>` on negative values when you wanted unsigned results; use `>>>`

✓ TIPS & BEST PRACTICES

- Write masks in hex or binary literals: `0b0100`, `0xFF`
- `Integer.toBinaryString` is your debugging friend
- `EnumSet` gives bit-flag speed with readable names

QUICK SUMMARY

- `&` `|` `^` `~` — bit logic
- `<<` — × 2^n
- `>>` — ÷ 2^n, keeps sign
- `>>>` — shift in zeros

Assignment Operators

= · += · -= · *= · /= · %= · compound assignment · implicit cast

ASSIGNING AND UPDATING

= stores the right-hand value in the variable on the left, and the whole expression evaluates to that value. **Compound assignment** operators combine an operation with assignment: `x += 5` means `x = (type of x)(x + 5)`. That hidden cast lets compound operators compile where the long form would not.

NOTES

- Assignment is right-associative: `a = b = 0` sets both
- `+=` on a `String` concatenates
- Compound operators include an **implicit narrowing cast**
- `=` is assignment; `==` is comparison

ASSIGNMENT OPERATORS

OPERATOR	EXAMPLE	EQUIVALENT	X AFTER (X = 10)
=	x = 5	x = 5	5
+=	x += 3	x = x + 3	13
-=	x -= 3	x = x - 3	7
*=	x *= 3	x = x * 3	30
/=	x /= 3	x = x / 3	3
%=	x %= 3	x = x % 3	1

BITWISE COMPOUND

OPERATOR	EXAMPLE	EQUIVALENT
&=	f &= mask	f = f & mask
|=	f |= mask	f = f | mask
^=	f ^= mask	f = f ^ mask
<<=	n <<= 2	n = n << 2
>>=	n >>= 1	n = n >> 1
>>>=	n >>>= 1	n = n >>> 1

COMPOUND ASSIGNMENT DETAILS

```
int a, b, c;
a = b = c = 0;            // chained: all 0

short s = 10;
// s = s + 1;             // ✖ int → short needs a cast
s += 1;                   // ✔ 11 (implicit (short) cast)

byte by = 127;
by += 1;                  // -128 (silent overflow)

int total = 7;
total /= 2;               // 3  (integer division)
double avg = 7;
avg /= 2;                 // 3.5

String msg = "Hi";
msg += ", Ada";           // "Hi, Ada"

int n;
if ((n = readValue()) > 0) { }   // assign inside expression
```

NOTES

- → The implicit cast can hide overflow — `by += 1` wraps
- → `/=` follows the variable's type, not the right side
- → Assigning inside a condition works but hurts readability

⚠ COMMON MISTAKES

- `if (x = 5)` — in Java this fails to compile for `int`, but `if (flag = true)` compiles and is always true
- Using `+=` on `String` inside a big loop creates many objects — use `StringBuilder`
- Expecting `i += 0.5` on an `int` to add half — it truncates back to `int`

✓ TIPS & BEST PRACTICES

- Use compound operators for counters and totals: `sum += price`
- Avoid chained assignment for objects; all variables share one reference
- Write `if (flag)` instead of comparing booleans to avoid `=` / `==` typos

QUICK SUMMARY

- `=` — assign, returns the value
- `op=` — update in place
- Compound ops cast implicitly
- `String +=` — concatenates

String Basics

STRINGS IN JAVA

A `String` is an **immutable** sequence of characters and a full object, not a primitive. Every method that seems to change a string actually returns a **new** one. Literals are stored in a shared **string pool**, so identical literals share one object. Java 15 added **text blocks** (`"""`) for multi-line strings.

NOTES

- `String` is in `java.lang`, so no import is needed
- `length()` is a method; arrays use `.length` field
- `null` (no object) is different from `""` (empty string)
- `isBlank()` (Java 11+) treats whitespace-only as empty

CREATING & COMBINING STRINGS

```
String a = "Java";                  // literal (pooled)
String b = new String("Java");      // forces a new object (rarely needed)
String empty = "";
String none = null;

a.length();                         // 4
"".isEmpty();                       // true
"  ".isBlank();                     // true

String full = a + " " + 21;         // "Java 21"
String c = "Hi".concat("!");        // "Hi!"
String joined = String.join("-", "a", "b", "c");   // "a-b-c"
"ab".repeat(3);                     // "ababab"

a.toUpperCase();                    // "JAVA" (new string)
System.out.println(a);              // "Java" (unchanged)

String json = """
    {"name": "Ada",
     "age": 36}
    """;                            // text block, keeps line breaks
```

NOTES

- → `toUpperCase()` returns a new string; `a` is unchanged
- → `+` with any type converts it via `String.valueOf`
- → Text blocks strip common leading indentation
- → `none.length()` throws `NullPointerException`

LITERAL FORMS

FORM	EXAMPLE	NOTES
Normal	"Hello"	Escapes like \n allowed
Text block	"""…"""	Java 15+, multi-line
Char	'H'	A `char`, not a `String`
Unicode	"\u00e9"	é
Concatenation	"a" + "b"	Compile-time constant if all literals

NULL VS EMPTY VS BLANK

VALUE	ISEMPTY()	ISBLANK()
""	true	true
" "	false	true
"hi"	false	false
null	NPE	NPE

⚠ COMMON MISTAKES

- Calling `s.toUpperCase();` without assigning — strings are immutable, the result is lost
- Using `.length` (no parentheses) on a `String` — that's for arrays
- Confusing `'a'` (char) with `"a"` (String)

✓ TIPS & BEST PRACTICES

- Check `s == null || s.isBlank()` before trusting user input
- Use text blocks for SQL, JSON, and HTML snippets
- Use `String.join` instead of manual loops with separators

QUICK SUMMARY

- `String` — immutable object
- `length()` — character count
- `+` / `concat` / `join` — combine
- `"""` — text block

String Methods

COMMON STRING METHODS

`String` has a large built-in API for searching, extracting, and transforming text. Indexes are **zero-based**, and ranges like `substring(begin, end)` include `begin` but **exclude** `end`. Because strings are immutable, every transforming method returns a new string.

NOTES

- Out-of-range indexes throw `StringIndexOutOfBoundsException`
- `indexOf` returns `-1` when not found
- `split` takes a **regex**: escape `.` and `|`
- `strip()` is Unicode-aware; `trim()` only removes ≤ U+0020

SEARCH METHODS

METHOD	EXAMPLE (S = "HELLO WORLD")	RESULT
`charAt(i)`	s.charAt(4)	'o'
`indexOf(str)`	s.indexOf("o")	4
`lastIndexOf(str)`	s.lastIndexOf("o")	7
`indexOf(str, from)`	s.indexOf("o", 5)	7
`contains(str)`	s.contains("World")	true
`startsWith(p)`	s.startsWith("He")	true
`endsWith(p)`	s.endsWith("ld")	true
`matches(regex)`	s.matches("H.*d")	true

TRANSFORM METHODS

METHOD	EXAMPLE	RESULT
`substring(b)`	"Hello".substring(1)	"ello"
`substring(b, e)`	"Hello".substring(1, 3)	"el"
`replace(a, b)`	"a-b-c".replace("-", "+")	"a+b+c"
`replaceAll(re, b)`	"a1b2".replaceAll("\\d", "")	"ab"
`toLowerCase()`	"HeLLo".toLowerCase()	"hello"
`trim() / strip()`	" hi ".strip()	"hi"
`stripLeading()`	" hi".stripLeading()	"hi"

SPLITTING & CHAINING

```
"a,b,,c".split(",");            // [a, b, , c]
"a,b,,c,,".split(",");          // [a, b, , c]   trailing empties dropped
"a,b,,c,,".split(",", -1);      // [a, b, , c, , ] keep them
"1.2.3".split("\\.");           // [1, 2, 3]   escape the dot
"a | b".split("\\s*\\|\\s*");    // [a, b]
"one  two".split("\\s+");       // [one, two]

String clean = "  Ada LOVELACE ".strip().toLowerCase();  // "ada lovelace"
String initials = clean.charAt(0) + "" + clean.charAt(4);  // "al"
"banana".chars().filter(ch -> ch == 'a').count();         // 3
String.valueOf(new char[]{'h','i'});                       // "hi"
"hello".toCharArray();                                     // [h, e, l, l, o]
```

NOTES

- → `split(",", -1)` keeps trailing empty fields (important for CSV)
- → `"\\."` — escaped for regex, then for Java
- → `chars()` gives an `IntStream` of character codes

⚠ COMMON MISTAKES

- `"1.2.3".split(".")` returns an empty array — `.` means "any character" in regex
- Using `replaceAll` for a plain-text replace with special chars; use `replace`
- Off-by-one in `substring(b, e)` — the end index is exclusive

✓ TIPS & BEST PRACTICES

- Use `replace` for literal text, `replaceAll` only for regex
- Prefer `strip()` over `trim()` in modern code
- Use `s.indexOf(x) >= 0` or `contains` — don't loop manually

QUICK SUMMARY

- `charAt` `indexOf` `contains` — search
- `substring(b, e)` — e exclusive
- `replace` `strip` `toLowerCase` — transform
- `split(regex)` — tokenize

String Formatting

FORMATTING TEXT

`String.format` builds a string from a **format string** containing % specifiers, and `"...".formatted(args)` (Java 15+) does the same as an instance method. Specifiers follow the pattern `%[index$][flags][width][.precision]conversion`. `printf` uses the identical syntax but prints directly.

NOTES

- Specifiers are matched to arguments left to right
- `%n` is the platform line separator
- `%%` prints a literal percent sign
- Pass a `Locale` for grouping and decimal symbols

FORMATTING EXAMPLES

```
String.format("%s has %d items", "Cart", 3);   // "Cart has 3 items"
"%s scored %.1f%%".formatted("Ada", 92.456);    // "Ada scored 92.5%"

String.format("[%8s]", "right");      // "[   right]"
String.format("[%-8s]", "left");      // "[left    ]"
String.format("[%05d]", 42);          // "[00042]"
String.format("%,d", 1234567);        // "1,234,567"
String.format("%+d", 5);              // "+5"
String.format("%.3f", 2.0);           // "2.000"
String.format("%10.2f|", 3.14159);    // "      3.14|"
String.format("%x / %X", 255, 255);   // "ff / FF"
String.format("%e", 12345.678);       // "1.234568e+04"
String.format("%2$s %1$s", "a", "b"); // "b a"
String.format(Locale.GERMANY, "%,.2f", 1234.5); // "1.234,50"
```

ANATOMY

- → `%-8s` — flag `-` (left), width 8, string
- → `%05d` — flag `0` (zero pad), width 5
- → `%.1f` — precision 1
- → `%2$s` — argument index 2

CONVERSIONS

SPEC	TYPE	EXAMPLE → OUTPUT
%s	Any (toString)	"hi" → hi
%d	Integer types	42 → 42
%f	float/double	1.5 → 1.500000
%c	char	'A' → A
%b	boolean	true → true
%x	Hex	255 → ff
%e	Scientific	1500.0 → 1.500000e+03

FLAGS

FLAG	MEANING	EXAMPLE
-	Left-justify	%-6s
0	Zero-pad	%06d
,	Group digits	%,d
+	Always show sign	%+d
(	Negatives in parens	%(d → (5)
(space)	Space before positives	% d

⚠ COMMON MISTAKES

- Mismatched types: `%d` with a `double` throws `IllegalFormatConversionException`
- Forgetting `%%` and getting `UnknownFormatConversionException`
- Using `String.format` in tight loops for simple concatenation; it's slower

✓ TIPS & BEST PRACTICES

- Use `.formatted()` for readability: `"Hi %s".formatted(name)`
- Build aligned text tables with fixed widths like `%-12s%8.2f`
- For currency, use `NumberFormat.getCurrencyInstance(locale)`

QUICK SUMMARY

- `String.format` / `.formatted` — build strings
- `%[flags][width][.prec]conv` — pattern
- `%s %d %f %n` — core specifiers
- `-` `0` `,` — alignment and grouping

Scanner Input

READING KEYBOARD INPUT

`java.util.Scanner` reads text from `System.in` (or a file or string) and breaks it into **tokens** separated by whitespace. Methods like `nextInt()` parse the next token; `nextLine()` reads the rest of the current line. Mixing the two is the most common beginner bug: `nextInt()` leaves the newline behind.

NOTES

- `import java.util.Scanner;` is required
- Create **one** Scanner for `System.in` and reuse it
- `nextInt()` throws `InputMismatchException` on non-numbers
- Closing a Scanner on `System.in` closes `System.in` too

READING INPUT

```java
import java.util.Scanner;

Scanner in = new Scanner(System.in);

System.out.print("Name: ");
String name = in.nextLine();          // whole line: "Ada Lovelace"

System.out.print("Age: ");
int age = in.nextInt();               // 36
in.nextLine();                        // ✔ consume leftover newline

System.out.print("City: ");
String city = in.nextLine();          // works now

System.out.print("Enter a number: ");
while (!in.hasNextInt()) {            // validate before reading
    System.out.print("Not a number, try again: ");
    in.next();                        // discard bad token
}
int n = in.nextInt();
```

NOTES

- → `nextLine()` — reads up to and removes the line break
- → `nextInt()` — reads a token, leaves `\n` in the buffer
- → `hasNextInt()` — peeks without consuming
- → `in.next()` — read one word (token)

SCANNER METHODS

METHOD	RETURNS	READS
`nextLine()`	String	Rest of line
`next()`	String	Next token (word)
`nextInt()`	int	Next token as int
`nextDouble()`	double	Next token as double
`nextBoolean()`	boolean	"true"/"false"
`nextLong()`	long	Next token as long

CHECKING METHODS

METHOD	TRUE WHEN…
`hasNext()`	Another token exists
`hasNextLine()`	Another line exists
`hasNextInt()`	Next token parses as int
`hasNextDouble()`	Next token parses as double

⚠ COMMON MISTAKES

- `nextInt()` then `nextLine()` returns `""` — call an extra `nextLine()` first
- Creating a new `Scanner(System.in)` inside a loop; keep one instance
- Closing the Scanner early, then trying to read again — `System.in` is gone

✓ TIPS & BEST PRACTICES

- Read everything with `nextLine()` and parse with `Integer.parseInt` for predictable behavior
- Use `hasNextXxx()` to validate instead of catching exceptions
- For large input, `BufferedReader` is much faster (Sheet 18)

QUICK SUMMARY

- `new Scanner(System.in)` — keyboard input
- `nextLine()` — whole line
- `nextInt()` `nextDouble()` — tokens
- `hasNextInt()` — validate first

BufferedReader Input

FAST LINE-BASED INPUT

`BufferedReader` wraps a character stream and reads text **a line at a time** through an internal buffer. Paired with `InputStreamReader(System.in)`, it reads console input far faster than `Scanner`, which matters for large inputs and competitive programming. It only returns strings, so you parse values yourself.

NOTES

- `readLine()` returns `null` at end of input
- Methods throw the checked `IOException`
- No regex tokenizing overhead, unlike `Scanner`
- `Console` (`System.console()`) can read passwords without echo

READING CONSOLE INPUT

```
import java.io.*;

public static void main(String[] args) throws IOException {
    BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in));

    String name = br.readLine();                 // "Ada"
    int age = Integer.parseInt(br.readLine().trim());   // 36

    String[] parts = br.readLine().split("\\s+");   // "3 4 5"
    int sum = 0;
    for (String p : parts) sum += Integer.parseInt(p);  // 12

    String line;
    while ((line = br.readLine()) != null) {     // until EOF
        System.out.println(line.toUpperCase());
    }
}
```

NOTES

- `throws IOException` — or wrap in `try/catch`
- `.trim()` before parsing removes stray spaces
- `split("\\s+")` handles multiple spaces
- Ctrl+D (Mac/Linux) or Ctrl+Z (Windows) sends EOF

SCANNER VS BUFFEREDREADER

FEATURE	SCANNER	BUFFEREDREADER
Speed	Slower	Much faster
Parses types	Yes (nextInt…)	No, strings only
Buffer size	1 KB	8 KB default
Exceptions	Unchecked	Checked IOException
Best for	Small interactive input	Large input, files

READING PATTERNS

GOAL	CODE
One int	Integer.parseInt(br.readLine())
Ints on a line	Arrays.stream(line.split(" ")).mapToInt(Integer::parseInt)
All lines	br.lines().toList()
Password	System.console().readPassword()

⚠ COMMON MISTAKES

- Forgetting `throws IOException`, which gives a compile error
- Calling `.trim()` on a `null` line at EOF — check for `null` first
- Mixing `Scanner` and `BufferedReader` on the same `System.in`; buffers steal each other's data

✓ TIPS & BEST PRACTICES

- Use `br.lines()` for a stream of all remaining lines
- Wrap in try-with-resources when reading files (Sheet 77)
- `System.console()` returns `null` inside most IDEs; test in a real terminal

QUICK SUMMARY

- `BufferedReader` — fast line input
- `readLine()` — next line or `null`
- Parse manually with `parseInt`
- Checked `IOException`

Advanced Strings

BEYOND THE BASICS

This sheet covers the less common parts of `String`: **lexicographic comparison**, the **string pool** and `intern()`, Unicode code points, and the thread-safe `StringBuffer`. For everyday string building, use `StringBuilder` (Sheet 90); this sheet explains when its older sibling still applies.

NOTES

- `compareTo` compares UTF-16 values, so `"Z" < "a"`
- `intern()` returns the pooled copy of a string
- Emoji take **two** `char`s (a surrogate pair)
- `StringBuffer` is synchronized; `StringBuilder` is not

COMPARISON, POOLING & UNICODE

```
"apple".compareTo("apricot");          // -2  ('p' - 'r')
"Zebra".compareTo("apple");            // -7  (uppercase sorts first)
"Zebra".compareToIgnoreCase("apple");  // 25
"abc".compareTo("abcd");               // -1  (shorter first)

String a = new String("hi");
String b = a.intern();
b == "hi";                             // true (pooled instance)

String e = "Hi 👋";
e.length();                            // 5  (UTF-16 units)
e.codePointCount(0, e.length());       // 4  (real characters)
e.codePoints().forEach(cp ->
    System.out.print(Character.toChars(cp)));   // Hi 👋

StringBuffer sb = new StringBuffer("ab");
sb.append("cd").reverse();              // "dcba" (thread-safe)
"2026-10-02".matches("\\d{4}-\\d{2}-\\d{2}");    // true
```

NOTES

- → Use `Collator` for language-aware sorting
- → `intern()` is rarely needed; the JVM pools literals automatically
- → Use `codePoints()` when text may contain emoji
- → `matches` must match the **whole** string

STRINGBUILDER VS STRINGBUFFER VS STRING

TYPE	MUTABLE	THREAD-SAFE	USE
`String`	No	Yes (immutable)	Fixed text
`StringBuilder`	Yes	No	Building text (default)
`StringBuffer`	Yes	Yes, synchronized	Shared across threads (rare)

COMPARISON METHODS

METHOD	IGNORES CASE	RETURNS
`equals`	No	boolean
`equalsIgnoreCase`	Yes	boolean
`compareTo`	No	int (-, 0, +)
`compareToIgnoreCase`	Yes	int
`regionMatches`	Optional	boolean
`contentEquals`	No	boolean (vs CharSequence)

⚠ COMMON MISTAKES

- Sorting user-facing names with `compareTo` — uppercase letters sort before lowercase
- Using `length()` to count characters in emoji-rich text
- Reaching for `StringBuffer` by default — it's slower and almost never needed

✓ TIPS & BEST PRACTICES

- `String.CASE_INSENSITIVE_ORDER` is a ready-made `Comparator`
- Use `Collator.getInstance(Locale.US)` for natural-language sorting
- Use `StringBuilder` (Sheet 90) for all single-threaded building

QUICK SUMMARY

- `compareTo` — lexicographic order
- `intern()` — pooled instance
- `codePoints()` — Unicode-safe
- `StringBuffer` — synchronized builder

Arrays — 1D

FIXED-SIZE COLLECTIONS

An **array** stores a fixed number of elements of one type in contiguous memory. Its size is set when created and **cannot change**. Elements are accessed by zero-based index, and `length` (a field, not a method) gives the size. Arrays are objects, so array variables hold references.

NOTES

- New arrays are filled with defaults: `0`, `false`, `null`
- Accessing index `length` or beyond throws `ArrayIndexOutOfBoundsException`
- Assigning one array variable to another shares the **same** array
- For resizable lists, use `ArrayList` (Sheet 45)

CREATING & USING ARRAYS

```
int[] scores = new int[5];              // [0, 0, 0, 0, 0]
String[] names = {"Ada", "Alan", "Grace"};   // literal
double[] temps = new double[]{1.5, 2.5};    // explicit form

scores[0] = 90;                         // set
int first = scores[0];                  // get: 90
names.length;                           // 3
names[names.length - 1];                // "Grace" (last)

for (int i = 0; i < scores.length; i++) {
    scores[i] = i * 10;                 // [0, 10, 20, 30, 40]
}
for (String n : names) System.out.println(n);

int[] copy = scores;                    // same array!
copy[0] = 99;                           // scores[0] is now 99
int[] real = Arrays.copyOf(scores, scores.length);  // true copy
System.out.println(Arrays.toString(scores));   // [99, 10, 20, 30, 40]
```

NOTES

- → `int[] a` is preferred over `int a[]`
- → Array literals `{...}` only work in declarations
- → Printing an array directly shows `[I@1b6d3586`
- → `Arrays.toString` gives readable output

DEFAULT VALUES

ELEMENT TYPE	DEFAULT
`int, long, short, byte`	0
`double, float`	0.0
`char`	'\u0000'
`boolean`	false
`Any object (String…)`	null

QUICK ARRAYS HELPERS

METHOD	RESULT
`Arrays.toString(a)`	"[1, 2, 3]"
`Arrays.sort(a)`	Sorts in place
`Arrays.fill(a, 7)`	Every slot = 7
`Arrays.copyOf(a, n)`	New array of length n
`Arrays.equals(a, b)`	Compare contents
`See Sheet 96`	Full Arrays utility

⚠ COMMON MISTAKES

- Looping with `i <= a.length` — the last valid index is `length - 1`
- Comparing arrays with `==` or `.equals()` — use `Arrays.equals`
- Expecting `a = b` to copy — it copies the reference

✓ TIPS & BEST PRACTICES

- Use the enhanced `for` when you don't need the index
- Prefer `List` when the size may change
- `int[] a = {}` creates an empty array safely, avoiding `null`

QUICK SUMMARY

- `type[] name = new type[n]` — create
- `a[i]` — zero-based access
- `a.length` — size (field)
- `Arrays.toString` — print contents

If / Else

MAKING DECISIONS

An `if` statement runs a block only when its condition is `true`. Add `else if` branches to test more conditions in order, and an optional `else` to catch everything else. Java checks branches **top to bottom** and runs only the first one that matches. Conditions must be `boolean`; there is no truthy/falsy conversion.

NOTES

- Braces are optional for one statement, but always use them
- Only the **first** matching branch runs
- `if (x)` requires `x` to be `boolean`
- Java 16+: `if (obj instanceof String s)` binds `s`

SYNTAX & PATTERNS

```
int score = 83;
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';                        // ← this runs
} else if (score >= 70) {
    grade = 'C';
} else {
    grade = 'F';
}

// Guard clause: exit early instead of nesting
if (user == null) return;
if (!user.isActive()) return;
process(user);

// Pattern matching for instanceof (Java 16+)
if (obj instanceof String s && !s.isBlank()) {
    System.out.println(s.length());
}
```

NOTES

- → Order branches from most to least specific
- → Guard clauses keep the "happy path" unindented
- → `s` is only in scope where the match is certain

BRANCH FORMS

FORM	USE WHEN
`if`	Optional single action
`if / else`	Exactly one of two paths
`if / else if / else`	Several exclusive ranges
`Nested if`	Second check depends on first
`Ternary ?:`	Choosing a value (Sheet 22)
`switch`	Many discrete values (Sheet 23)

CONDITION EXAMPLES

CONDITION	MEANING
`x > 0 && x < 10`	Between 1 and 9
`s != null && !s.isEmpty()`	Has text
`list.isEmpty()`	No items
`a.equals(b)`	Same contents
`o instanceof Dog d`	Is a Dog; bind d

⚠ COMMON MISTAKES

- `if (x > 0);` — the stray semicolon ends the `if`; the block always runs
- Dangling `else` without braces attaches to the **nearest** `if`
- Ordering `>= 70` before `>= 90` makes the higher branches unreachable

✓ TIPS & BEST PRACTICES

- Always use braces, even for one-line bodies
- Replace deep nesting with guard clauses or extracted methods
- Use `switch` when testing one variable against many constants

QUICK SUMMARY

- `if` — run when true
- `else if` — next test, in order
- `else` — fallback
- `instanceof` pattern — test + cast

Ternary Operator

CHOOSING A VALUE INLINE

The **conditional operator** `condition ? a : b` evaluates to `a` when the condition is true and `b` otherwise. Unlike `if`, it is an **expression**, so it produces a value you can assign, return, or pass to a method. Use it for simple value choices, not for running statements.

NOTES

- Both branches must produce compatible types
- Only the chosen branch is evaluated
- Mixed numeric types are promoted: `true ? 1 : 2.0` → `1.0`
- Mixing `int` and `Integer` can unbox `null` → NPE

EXAMPLES

```
int age = 20;
String status = age >= 18 ? "adult" : "minor";     // "adult"

int max = a > b ? a : b;                           // Math.max(a, b)
System.out.println(n == 1 ? "1 item" : n + " items");
return list.isEmpty() ? null : list.get(0);

String name = input != null ? input : "Guest";      // null default
String name2 = Objects.requireNonNullElse(input, "Guest");  // same

// Nested — readable only when formatted like a chain
String size = n < 10  ? "small"
            : n < 100 ? "medium"
            :           "large";

Object o = true ? 1 : "one";   // Integer 1 (common supertype)
Integer x = null;
int bad = flag ? x : 0;        // NPE if flag is true (unboxing)
```

NOTES

- → Great inside `println`, `return`, and assignments
- → Chain-format nested ternaries, one condition per line
- → `requireNonNullElse` (Java 9+) replaces null-default ternaries

TERNARY VS IF

SITUATION	PREFER
`Pick one of two values`	Ternary
`Run statements / side effects`	if / else
`Three or more outcomes`	switch expression
`Null default`	Objects.requireNonNullElse
`Optional value`	Optional.orElse (Sheet 97)

TYPE RESULTS

EXPRESSION	RESULT TYPE
`c ? 1 : 2`	int
`c ? 1 : 2.0`	double
`c ? "a" : "b"`	String
`c ? 'a' : 0`	char
`c ? 1 : null`	Integer

⚠ COMMON MISTAKES

- Using the ternary as a statement: `x > 0 ? doA() : doB();` doesn't compile unless the value is used
- Deeply nested ternaries on one line that no one can read
- Surprise `double` results from `true ? 1 : 2.0`

✓ TIPS & BEST PRACTICES

- Keep each branch short; extract a method if it isn't
- Parenthesize the condition when it contains operators
- Prefer a `switch` expression for 3+ outcomes

QUICK SUMMARY

- `cond ? a : b` — value expression
- Only one branch is evaluated
- Branch types must be compatible
- Use for values, not actions

Switch — Classic

THE CLASSIC SWITCH STATEMENT

A `switch` statement compares one value against a list of `case` labels and jumps to the match. Without a `break`, execution **falls through** into the next case, which is occasionally useful and frequently a bug. `default` handles every unmatched value. This colon-style form works in every Java version.

NOTES

- Supported types: `int`, `char`, `byte`, `short`, wrappers, `String`, `enum`
- Not allowed: `long`, `float`, `double`, `boolean` (before Java 21 patterns)
- Case labels must be **compile-time constants**
- `switch` on a `null` String throws NPE

CLASSIC SYNTAX

```
int day = 3;
String name;
switch (day) {
    case 1:
        name = "Mon";
        break;
    case 2:
        name = "Tue";
        break;
    case 3:
        name = "Wed";       // ← match
        break;              // exits the switch
    default:
        name = "Other";
}

switch (month) {            // intentional fall-through grouping
    case 12: case 1: case 2:
        season = "Winter"; break;
    case 6: case 7: case 8:
        season = "Summer"; break;
    default:
        season = "Spring/Fall";
}
```

NOTES

- → `break` — leave the switch
- → Stacked labels share one body
- → `default` can appear anywhere but belongs last
- → Enum cases use bare names: `case RED:`

FALL-THROUGH EXAMPLE

CODE	X = 1 PRINTS
`case 1: print("A");`	A
`case 2: print("B");`	B (fell through)
`case 3: print("C"); break;`	C
`default: print("D");`	not reached

STRING & ENUM SWITCH

FORM	EXAMPLE
`String`	switch (cmd) { case "start": … }
`Enum`	switch (color) { case RED: … }
`char`	switch (c) { case 'y': case 'Y': … }
`Wrapper`	switch (integerObj) — NPE if null

⚠ COMMON MISTAKES

- Missing `break` causes the next case's code to run too
- Using `switch` on a possibly null String — check for null first
- Duplicate `case` values don't compile; neither do variables as labels

✓ TIPS & BEST PRACTICES

- Comment intentional fall-through: `// fall through`
- Prefer arrow syntax (Sheet 24) in Java 14+ to make fall-through impossible
- Always include `default`, even if it just throws

QUICK SUMMARY

- `switch (x) { case v: … break; }`
- No `break` → fall-through
- `default` — no match
- Constants only as labels

Sheet 24

Switch — Expressions (Java 14+)

-> arrow · yield · switch expression · exhaustiveness · pattern matching · when guard · case null

MODERN SWITCH

Java 14 made `switch` usable as an **expression** with **arrow labels** (`case X ->`). Arrow cases never fall through, can list several values, and the whole switch can return a value. Java 21 added **pattern matching**: switch on any object type, deconstruct records, add `when` guards, and handle `case null`.

NOTES

- A switch **expression** must be exhaustive (cover every case)
- `yield` returns a value from a block body
- Enums and sealed types can be exhaustive without `default`
- Arrow and colon labels can't be mixed in one switch

ARROW SYNTAX & PATTERNS

```
String name = switch (day) {
    case 1, 7 -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default -> {
        log(day);
        yield "Unknown";              // block result via yield
    }
};

// Pattern matching (Java 21)
String describe(Object o) {
    return switch (o) {
        case null -> "null!";
        case Integer i when i > 100 -> "big int " + i;
        case Integer i -> "int " + i;
        case String s -> "text of " + s.length();
        case Point(int x, int y) -> "point " + x + "," + y;
        default -> "something else";
    };
}
```

NOTES

- → Comma-separated labels replace stacked cases
- → `when` adds a condition to a pattern
- → `Point(int x, int y)` is a **record pattern**
- → Order matters: specific patterns before general ones

CLASSIC VS MODERN

FEATURE	CLASSIC (:)	ARROW (->)
Fall-through	Yes	No
Multiple labels	Stacked cases	case 1, 2 ->
Returns value	No	Yes (expression)
Exhaustive check	No	Yes for expressions
Pattern matching	No	Java 21+

VERSION HISTORY

VERSION	FEATURE
Java 7	switch on String
Java 14	Arrow labels, expressions, yield
Java 17	Sealed types for exhaustiveness
Java 21	Patterns, when guards, case null
Java 21	Record deconstruction patterns

⚠ COMMON MISTAKES

- Using `return` inside a switch expression block — use `yield`
- Putting `case Integer i` before `case Integer i when …` — the guarded case becomes unreachable
- Forgetting `;` after the closing brace of a switch expression assignment

✓ TIPS & BEST PRACTICES

- Default to arrow syntax in all new code
- Omit `default` on enums so the compiler flags new constants
- Combine with sealed interfaces (Sheet 73) for type-safe branching

QUICK SUMMARY

- `case A, B ->` — no fall-through
- `switch` as expression → value
- `yield` — return from block
- `case Type t when …` — patterns (21+)

For Loop

COUNTING LOOPS

The classic `for` loop packs three parts into its header: an **initializer** that runs once, a **condition** checked before each pass, and an **update** run after each pass. It's the natural choice when you know how many times to loop or need the index. Any of the three parts can be empty.

NOTES

- The loop variable is scoped to the loop
- `for (;;)` is an infinite loop
- The condition is checked **before** each iteration
- Modifying a list while indexing it shifts positions

FOR LOOP VARIANTS

```
for (int i = 0; i < 5; i++) {          // 0 1 2 3 4
    System.out.print(i + " ");
}
for (int i = 10; i > 0; i -= 2) { }    // 10 8 6 4 2
for (int i = 1; i <= 100; i *= 10) { } // 1 10 100

int[] a = {4, 8, 15};
for (int i = 0; i < a.length; i++) {
    System.out.println(i + ": " + a[i]);   // 0: 4 …
}
for (int i = a.length - 1; i >= 0; i--) { }   // reverse

for (int i = 0, j = 9; i < j; i++, j--) { }   // two pointers

for (int r = 1; r <= 3; r++) {          // nested: times table
    for (int c = 1; c <= 3; c++) {
        System.out.print(r * c + "\t");
    }
    System.out.println();
}
```

NOTES

- → `i < a.length` — correct upper bound
- → Reverse loops start at `length - 1`
- → Comma operator allowed only in init and update
- → Nested loops multiply: 3 × 3 = 9 iterations

HEADER PARTS

PART	RUNS	EXAMPLE
Init	Once, before loop	int i = 0
Condition	Before every pass	i < n
Update	After every pass	i++
Body	When condition true	{ … }

COMMON LOOP SHAPES

GOAL	HEADER
n times	for (int i = 0; i < n; i++)
1 to n inclusive	for (int i = 1; i <= n; i++)
Backwards	for (int i = n - 1; i >= 0; i--)
Every other	for (int i = 0; i < n; i += 2)
Forever	for (;;)

⚠ COMMON MISTAKES

- `i <= a.length` — off-by-one, throws `ArrayIndexOutOfBoundsException`
- A semicolon after the header `for (...);` makes an empty loop
- Removing items from a `List` in an index loop skips elements; loop backwards or use `removeIf`

✓ TIPS & BEST PRACTICES

- Use the enhanced for (Sheet 28) when you don't need `i`
- Use `IntStream.range(0, n)` for stream-style counting
- Cache expensive bounds: `for (int i = 0, n = list.size(); i < n; i++)`

QUICK SUMMARY

- `for (init; cond; update)`
- Best when count or index matters
- Bounds: `0` to `< length`
- Nested loops multiply work

While Loop

CONDITION-DRIVEN LOOPS

A `while` loop repeats its body **as long as** its condition stays true, checking the condition before every pass. If the condition starts false, the body never runs. Use it when you don't know in advance how many iterations you need: reading until end of input, waiting for a state, or processing until a sentinel value appears.

NOTES

- Something inside the loop must eventually make the condition false
- `while (true)` + `break` is a valid, common pattern
- The body may run **zero** times
- Prefer `for` when you have a counter

WHILE PATTERNS

```
int n = 1;
while (n < 100) {
    n *= 2;                          // 2 4 8 16 32 64 128
}

// Sentinel: read until "quit"
Scanner in = new Scanner(System.in);
String cmd = in.nextLine();
while (!cmd.equals("quit")) {
    handle(cmd);
    cmd = in.nextLine();
}

// Digits of a number
int x = 4096, digits = 0;
while (x > 0) { x /= 10; digits++; }   // digits = 4

while (true) {                       // loop-and-a-half
    String line = in.nextLine();
    if (line.isEmpty()) break;
    process(line);
}
```

NOTES

- → The sentinel loop reads once before and once at the end
- → `x /= 10` strips one digit per pass
- → `while (true)` reads clearly when the exit is in the middle

WHILE VS FOR VS DO-WHILE

LOOP	CHECKS CONDITION	RUNS AT LEAST ONCE	BEST FOR
`while`	Before	No	Unknown count
`for`	Before	No	Counted / indexed
`do-while`	After	Yes	Menus, retry prompts

COMMON CONDITIONS

CONDITION	STOPS WHEN
`while (it.hasNext())`	No items left
`while ((line = br.readLine()) != null)`	End of file
`while (!done)`	Flag set
`while (x > 0)`	Number consumed
`while (attempts < 3)`	Retry limit hit

⚠ COMMON MISTAKES

- Forgetting to update the loop variable → infinite loop
- `while (x != 10)` with `x += 3` skips 10 and never stops; use `<`
- A stray semicolon `while (cond);` loops forever doing nothing

✓ TIPS & BEST PRACTICES

- Add a safety counter while debugging possibly-infinite loops
- Assign-and-test in one line: `while ((line = br.readLine()) != null)`
- Keep the exit condition visible at the top when possible

QUICK SUMMARY

- `while (cond) { }` — check first
- Zero or more iterations
- Use for unknown counts
- `while (true)` + `break` — mid-exit

Do-While Loop

RUN FIRST, CHECK AFTER

A `do-while` loop runs its body **once before** checking the condition, then repeats while the condition is true. That guarantee makes it the natural fit for menus, "play again?" prompts, and input validation, where you must ask at least once before you can test the answer.

NOTES

- Note the **semicolon** after `while (cond);`
- Variables declared inside the body aren't visible in the condition
- Body always executes at least once
- `break` and `continue` work as in other loops

MENUS & VALIDATION

```
Scanner in = new Scanner(System.in);
int choice;
do {
    System.out.println("1) New  2) Open  3) Quit");
    choice = in.nextInt();
    switch (choice) {
        case 1 -> newFile();
        case 2 -> openFile();
        case 3 -> System.out.println("Bye");
        default -> System.out.println("Invalid");
    }
} while (choice != 3);

int age;
do {
    System.out.print("Age (1-120): ");
    age = in.nextInt();
} while (age < 1 || age > 120);       // re-ask until valid

int tries = 0;
boolean ok;
do {
    ok = tryConnect();
    tries++;
} while (!ok && tries < 3);           // retry with a limit
```

NOTES

- → `choice` is declared **outside** the loop so the condition can see it
- → The validation loop reads first, then decides
- → Combine success flags and counters for retries

TRACE: AGE INPUT 0, 150, 30

PASS	AGE READ	CONDITION	LOOP AGAIN?
1	0	0 < 1	Yes
2	150	150 > 120	Yes
3	30	in range	No → exit

WHEN TO USE

SITUATION	LOOP
Must show prompt once	do-while
Might skip entirely	while
Known count	for
Iterating a collection	for-each

⚠ COMMON MISTAKES

- Forgetting the semicolon after `while (cond)` — compile error
- Declaring the tested variable inside `do { }` — it's out of scope in `while (...)`
- Using `do-while` where the body must not run on empty input

✓ TIPS & BEST PRACTICES

- Use it for menus and validation; otherwise `while` is usually clearer
- Print why input was rejected so the user isn't stuck guessing
- Cap retries with a counter to avoid infinite loops

QUICK SUMMARY

- `do { } while (cond);`
- Body runs at least once
- Condition checked after each pass
- Ideal for menus and prompts

Enhanced For Loop

LOOPING OVER ELEMENTS

The **enhanced for** loop (for-each) visits every element of an array or any `Iterable` (lists, sets, queues) without an index. It is shorter and avoids off-by-one errors. The trade-off: you can't see the index, can't go backwards, and can't safely add or remove elements from the collection during the loop.

NOTES

- The loop variable is a **copy** of each element (or reference)
- Works on arrays and anything implementing `Iterable`
- Maps aren't `Iterable`; loop over `entrySet()`, `keySet()`, or `values()`
- Structural changes throw `ConcurrentModificationException`

FOR-EACH EXAMPLES

```
int[] nums = {3, 5, 7};
int sum = 0;
for (int n : nums) sum += n;              // 15

List<String> names = List.of("Ada", "Alan");
for (String name : names) {
    System.out.println(name.toUpperCase());
}

Map<String, Integer> stock = Map.of("apples", 4, "pears", 2);
for (Map.Entry<String, Integer> e : stock.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
for (var e : stock.entrySet()) { }        // var shortens it

for (int n : nums) n *= 2;                // ✖ array unchanged

for (String s : list) {
    if (s.isEmpty()) list.remove(s);      // ✖ ConcurrentModificationException
}
list.removeIf(String::isEmpty);           // ✔
```

NOTES

- → Read it as "for each `n` in `nums`"
- → Assigning to the loop variable doesn't change the array
- → Use `removeIf` or an `Iterator` to delete safely
- → `entrySet` gives key and value in one pass

ENHANCED VS INDEXED

NEED	ENHANCED FOR	INDEXED FOR
Read every element	✔	✔
Know the index	✖	✔
Modify array slots	✖	✔
Go backwards / skip	✖	✔
Works on Set / Queue	✔	✖

ITERATING MAPS

LOOP OVER	GIVES YOU
`map.keySet()`	Keys only
`map.values()`	Values only
`map.entrySet()`	Key + value (fastest)
`map.forEach((k, v) -> …)`	Lambda style

⚠ COMMON MISTAKES

- Adding or removing items from a collection inside its own for-each loop
- Expecting `for (int x : arr) x = 0;` to clear the array
- Calling `map.get(key)` inside a `keySet()` loop; use `entrySet()` instead

✓ TIPS & BEST PRACTICES

- Default to for-each; switch to indexed only when you need `i`
- Use `list.forEach(System.out::println)` for one-liners
- Need index and element? `IntStream.range(0, list.size())`

QUICK SUMMARY

- `for (T x : items)` — every element
- Arrays + any `Iterable`
- No index, no structural changes
- Maps: loop `entrySet()`

Break & Continue

CONTROLLING LOOP FLOW

`break` exits the innermost loop (or `switch`) immediately. `continue` skips the rest of the current iteration and moves to the next one. Both act on the **innermost** loop unless you use a **label** to target an outer loop. `return` exits the whole method, ending every loop at once.

NOTES

- Labels are identifiers followed by a colon: `outer:`
- In a `for` loop, `continue` still runs the update step
- In a `while` loop, `continue` jumps straight to the condition
- Overusing labels usually means the method should be split

EXAMPLES

```
for (int i = 0; i < 10; i++) {
    if (i == 5) break;           // stop entirely
    System.out.print(i);          // 01234
}

for (int i = 0; i < 6; i++) {
    if (i % 2 == 0) continue;    // skip evens
    System.out.print(i);          // 135
}

outer:
for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[r].length; c++) {
        if (grid[r][c] == target) {
            found = true;
            break outer;          // leaves BOTH loops
        }
    }
}
```

NOTES

- → `break outer` jumps past the labeled loop
- → `continue outer` would start the next outer iteration instead
- → Searching? `return` from a helper method is often cleaner

STATEMENT EFFECTS

STATEMENT	EXITS	NEXT STEP
`break`	Innermost loop/switch	Code after the loop
`continue`	Current iteration	Next iteration
`break label`	Labeled loop	Code after labeled loop
`continue label`	Inner loops	Next labeled iteration
`return`	Whole method	Back to caller

JAVA VS OTHER LANGUAGES

CONCEPT	JAVA
`pass (Python)`	Empty block {} or ;
`goto`	Reserved word, unusable
`Labeled break`	Supported
`break in switch`	Exits switch only

⚠ COMMON MISTAKES

- Expecting `break` inside a `switch` in a loop to exit the loop — it only exits the switch
- `continue` in a `while` loop placed before the counter update causes an infinite loop
- Using flag variables and nested `if`s where a labeled `break` or `return` is clearer

✓ TIPS & BEST PRACTICES

- Extract nested searches into methods and `return` the result
- Keep `continue` at the top of the loop as a filter
- Name labels by meaning: `rows:`, `search:`

QUICK SUMMARY

- `break` — leave loop
- `continue` — skip to next pass
- `break label` — leave outer loop
- `return` — leave method

Try / Catch

HANDLING EXCEPTIONS

An **exception** is an object thrown when something goes wrong at runtime. Code that might fail goes in a `try` block, and a `catch` block handles a specific exception type. If no `catch` matches, the exception travels up the call stack and, if never caught, ends the program with a **stack trace**.

NOTES

- Catch blocks are checked in order; first match wins
- Catch **specific** types before general ones
- Java 7+: catch several types with `|`
- All exceptions extend `Throwable`

SYNTAX

```
try {
    int n = Integer.parseInt(input);     // may throw
    int r = 100 / n;                     // may throw
    System.out.println(r);
} catch (NumberFormatException e) {
    System.out.println("Not a number: " + e.getMessage());
} catch (ArithmeticException e) {
    System.out.println("Can't divide by zero");
} catch (Exception e) {                  // catch-all, last
    e.printStackTrace();
}

try {
    Files.readString(path);
} catch (IOException | SecurityException e) {   // multi-catch
    log.error("Read failed", e);
}
```

NOTES

- → Execution jumps to the first matching `catch`
- → Lines after the failing line in `try` never run
- → `getMessage()` — short description
- → `printStackTrace()` — full call stack to stderr

COMMON EXCEPTIONS

EXCEPTION	THROWN WHEN
`NullPointerException`	Using a null reference
`ArrayIndexOutOfBoundsException`	Bad array index
`NumberFormatException`	parseInt("abc")
`ArithmeticException`	Integer divide by zero
`ClassCastException`	Invalid cast
`IllegalArgumentException`	Bad method argument
`IllegalStateException`	Wrong object state
`IOException`	File / network failure

HIERARCHY

CLASS	KIND
`Throwable`	Root of all
`Error`	Serious JVM issues; don't catch
`Exception`	Checked (must handle)
`RuntimeException`	Unchecked (Sheet 31)

⚠ COMMON MISTAKES

- Empty `catch` blocks that silently swallow errors
- Catching `Exception` first, which makes later catches unreachable (compile error)
- Using exceptions for normal flow, like ending a loop

✓ TIPS & BEST PRACTICES

- Catch only what you can handle; let the rest propagate
- Always log or rethrow with the original exception as the cause
- Validate input (`hasNextInt`) instead of relying on catches

QUICK SUMMARY

- `try` — risky code
- `catch (Type e)` — handle one type
- `A | B` — multi-catch
- Specific before general

Throws & Checked Exceptions

THROWING AND DECLARING

`throw` raises an exception object; `throws` in a method signature **declares** which checked exceptions the method may pass to its caller. **Checked** exceptions (subclasses of `Exception` but not `RuntimeException`) must be caught or declared, which the compiler enforces. **Unchecked** exceptions (`RuntimeException` and its subclasses) need neither.

NOTES

- Checked = recoverable conditions outside your control (files, network)
- Unchecked = programming errors (null, bad index, bad argument)
- Lambdas can't throw checked exceptions through standard functional interfaces
- Wrap a low-level exception with a cause to keep the stack trace

THROW, THROWS, RETHROW

```
public void setAge(int age) {
    if (age < 0)
        throw new IllegalArgumentException("age must be ≥ 0: " + age);
    this.age = age;
}

public String load(Path p) throws IOException {   // declare
    return Files.readString(p);                    // checked
}

public Config readConfig(Path p) {
    try {
        return parse(load(p));
    } catch (IOException e) {
        throw new UncheckedIOException("Can't read " + p, e);  // wrap
    }
}
```

NOTES

- → `throw new …` — create and raise in one step
- → `throws IOException` — caller must handle it
- → Passing `e` as the cause preserves the original trace
- → `UncheckedIOException` wraps `IOException` for lambdas

CHECKED VS UNCHECKED

ASPECT	CHECKED	UNCHECKED
Superclass	Exception	RuntimeException
Must catch/declare	Yes	No
Typical cause	External failure	Bug in code
Examples	IOException, SQLException	NPE, IllegalArgumentException

CHOOSING AN EXCEPTION

SITUATION	THROW
Bad argument	IllegalArgumentException
Null argument	NullPointerException (Objects.requireNonNull)
Wrong state	IllegalStateException
Not implemented	UnsupportedOperationException
Index out of range	IndexOutOfBoundsException

⚠ COMMON MISTAKES

- `throws Exception` on every method — callers can no longer tell what really fails
- Wrapping without a cause: `new RuntimeException(e.getMessage())` loses the stack trace
- Catching checked exceptions just to print them and continue in a broken state

✓ TIPS & BEST PRACTICES

- Use `Objects.requireNonNull(x, "x")` to validate arguments
- Prefer standard exceptions before writing custom ones
- Include the bad value in the message

QUICK SUMMARY

- `throw` — raise an exception
- `throws` — declare checked exceptions
- Checked: catch or declare
- Wrap with a cause

Custom Exceptions

DEFINING YOUR OWN EXCEPTIONS

Create a custom exception by **extending** `Exception` (checked) or `RuntimeException` (unchecked). Custom types let callers catch a specific domain failure, like `InsufficientFundsException`, and can carry extra fields that describe what went wrong. Provide constructors that accept a message and a cause.

NOTES

- Name ends with `Exception` by convention
- Exceptions are `Serializable`; add `serialVersionUID` to silence warnings
- Make extra fields `final` and expose getters
- Group related exceptions under one base class

DEFINING & USING

```
public class InsufficientFundsException extends Exception {
    private static final long serialVersionUID = 1L;
    private final double shortfall;

    public InsufficientFundsException(String msg, double shortfall) {
        super(msg);
        this.shortfall = shortfall;
    }
    public InsufficientFundsException(String msg, Throwable cause) {
        super(msg, cause);
        this.shortfall = 0;
    }
    public double getShortfall() { return shortfall; }
}

public void withdraw(double amt) throws InsufficientFundsException {
    if (amt > balance)
        throw new InsufficientFundsException("Balance too low", amt - balance);
    balance -= amt;
}

try { acct.withdraw(500); }
catch (InsufficientFundsException e) {
    System.out.printf("Short by $%.2f%n", e.getShortfall());
}
```

NOTES

→ `super(msg)` stores the message for `getMessage()`

→ `super(msg, cause)` chains the original exception

→ The extra field gives callers data, not just text

CHECKED OR UNCHECKED?

CHOOSE	WHEN
`extends Exception`	Caller can reasonably recover
`extends RuntimeException`	Caller made a programming error
`Base class per module`	Callers catch a whole family
`Standard exception`	It already describes the problem

STANDARD CONSTRUCTORS

CONSTRUCTOR	PURPOSE
`()`	No message
`(String message)`	Describe the problem
`(String message, Throwable cause)`	Wrap another exception
`(Throwable cause)`	Wrap, reuse cause message

⚠ COMMON MISTAKES

- Extending `Throwable` or `Error` directly — extend `Exception` or `RuntimeException`
- Omitting the cause constructor, so wrapped errors lose their stack trace
- Creating a new exception class for every method when one domain type would do

✓ TIPS & BEST PRACTICES

- Only create custom types when callers need to catch them specifically
- Keep messages human-readable and include key values
- Make it a `record`-like immutable object: final fields, no setters

QUICK SUMMARY

- `extends Exception` — checked
- `extends RuntimeException` — unchecked
- Constructors: message + cause
- Add fields for context

Finally & Try-with-Resources

GUARANTEED CLEANUP

A `finally` block runs after `try`/`catch` **no matter what**: normal completion, exception, or `return`. Java 7 added **try-with-resources**, which declares resources in the `try` header and closes them automatically in reverse order. Any class implementing `AutoCloseable` works. It replaces almost every manual `finally { close(); }`.

NOTES

- `finally` is skipped only by `System.exit()` or a JVM crash
- Resources close in **reverse** declaration order
- Close-time exceptions are attached as **suppressed**

FINALLY VS TRY-WITH-RESOURCES

```
// Old style
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("data.txt"));
    return br.readLine();
} catch (IOException e) {
    return null;
} finally {
    if (br != null) try { br.close(); } catch (IOException ignored) { }
}

// Modern: try-with-resources
try (var br = new BufferedReader(new FileReader("data.txt"));
     var out = new PrintWriter("copy.txt")) {
    out.println(br.readLine());
}   // out closes, then br — automatically

// Your own resource
class Timer implements AutoCloseable {
    long start = System.nanoTime();
    public void close() {
        System.out.println((System.nanoTime() - start) + " ns");
    }
}
try (var t = new Timer()) { work(); }   // prints elapsed time
```

NOTES

- → Resources are separated by `;` in the header
- → No `catch` is required with try-with-resources
- → `close()` runs even when `work()` throws
- → Get hidden close errors with `e.getSuppressed()`

EXECUTION ORDER

SCENARIO	ORDER
`No exception`	try → finally
`Caught exception`	try → catch → finally
`Uncaught exception`	try → finally → propagate
`return in try`	try (value saved) → finally → return

COMMON AUTOCLOSEABLES

CLASS	PACKAGE
`BufferedReader / Writer`	java.io
`Scanner`	java.util
`Connection / Statement`	java.sql
`Stream from Files.lines`	java.util.stream

⚠ COMMON MISTAKES

- Returning from `finally` — it overrides the `try` result and swallows exceptions
- Forgetting to close a `Files.lines()` stream; wrap it in try-with-resources
- Using try-with-resources on `System.in` Scanners closes standard input permanently

✓ TIPS & BEST PRACTICES

- Use try-with-resources for anything with a `close()` method
- Keep `finally` for non-resource cleanup like unlocking (`lock.unlock()`)
- Implement `AutoCloseable` on your own classes that hold resources

QUICK SUMMARY

- `finally` — always runs
- `try (res)` — auto-close
- Reverse close order
- `AutoCloseable` — opt in

Defining & Calling Methods

WHAT A METHOD IS

A **method** is a named block of code that belongs to a class. Its **signature** is the name plus the parameter types; its declaration also states access level, whether it's `static`, and the **return type** (`void` if nothing is returned). Methods make code reusable, testable, and readable by giving a name to each step.

NOTES

- Every method lives inside a class; Java has no free functions
- Names are verbs in `camelCase`: `calculateTotal`
- `static` methods are called on the class; instance methods on an object
- A non-`void` method must return a value on every path

DECLARING & CALLING

```
public class Calc {
    // access  static  return  name   parameters
    public static int  add(int a, int b) {
        return a + b;
    }

    static void greet(String name) {          // no return value
        System.out.println("Hello, " + name);
    }

    double area(double r) {                   // instance method
        return Math.PI * r * r;
    }

    public static void main(String[] args) {
        int sum = add(2, 3);                  // 5
        greet("Ada");                         // Hello, Ada
        Calc c = new Calc();
        double a = c.area(2);                 // 12.566…
    }
}
```

ANATOMY

- → `public` — who can call it
- → `static` — no object needed
- → `int` — type of the returned value
- → `(int a, int b)` — parameter list
- → `return` — hands the value back

DECLARATION PARTS

PART	REQUIRED	EXAMPLES
`Access modifier`	No	public, private, protected
`static`	No	static
`Return type`	Yes	void, int, String, List<T>
`Name`	Yes	calculateTax
`Parameters`	Yes (may be empty)	(), (int x)

CALLING FORMS

CALL	WHEN
`add(1, 2)`	Same class, static or instance context
`Calc.add(1, 2)`	Static method from another class
`obj.area(2)`	Instance method on an object

⚠ COMMON MISTAKES

- Calling an instance method from `static main` without an object: "non-static method cannot be referenced"
- Declaring a return type but missing `return` on some branch
- Ignoring a returned value, like `s.trim();` with no assignment

✓ TIPS & BEST PRACTICES

- Keep methods short and doing one thing
- Make helpers `private` unless other classes need them

QUICK SUMMARY

- `modifiers returnType name(params)`
- `void` — no result
- `return` — send a value back
- `static` — call on the class

Parameters & Arguments

PASSING DATA IN

Parameters are the variables in a method declaration; **arguments** are the values you pass when calling it. Java is **always pass-by-value**. For primitives, the method gets a copy of the number. For objects, it gets a copy of the **reference**, so it can modify the object's contents but cannot make the caller's variable point to a new object.

NOTES

- Arguments must match parameter count, order, and types
- Java has no named or default parameters; use overloading or builders
- Mark parameters `final` to prevent reassignment
- Strings are immutable, so methods can't change a caller's String

PASS-BY-VALUE IN ACTION

```
static void bump(int n) { n++; }
int x = 5;
bump(x);
System.out.println(x);            // 5  (copy was changed)

static void addItem(List<String> list) { list.add("new"); }
List<String> items = new ArrayList<>();
addItem(items);
System.out.println(items);        // [new]  (same object)

static void replace(List<String> list) {
    list = new ArrayList<>();     // reassigns the local copy only
    list.add("lost");
}
replace(items);
System.out.println(items);        // [new]  (caller unaffected)

static void rename(String s) { s = s.toUpperCase(); }
String name = "ada";
rename(name);                     // name is still "ada"
```

NOTES

- → `bump` changes its own copy of `5`
- → `addItem` follows the reference to the shared list
- → `replace` points its copy elsewhere; the caller still sees the original
- → Return a value instead of trying to modify primitives

WHAT A METHOD CAN CHANGE

ARGUMENT	CHANGE CONTENTS?	REASSIGN CALLER'S VARIABLE?
`int / double`	No (copy)	No
`String`	No (immutable)	No
`List / array`	Yes	No
`Custom object`	Yes (via setters)	No

SIMULATING OPTIONAL PARAMS

TECHNIQUE	EXAMPLE
`Overloading`	greet() / greet(String name)
`Varargs`	sum(int... nums)
`Builder`	new Pizza.Builder().size(12).build()
`Nullable + default`	greet(String n) { n = n == null ? "you" : n; }

⚠ COMMON MISTAKES

- Expecting a method to swap two `int` variables — it can't; return the values instead
- Surprising callers by mutating a list they passed in
- Long parameter lists of the same type (`int, int, int`) that are easy to pass in the wrong order

✓ TIPS & BEST PRACTICES

- Copy defensively when storing caller-provided collections: `List.copyOf(list)`
- Keep parameter lists under ~4; group the rest into an object or record
- Validate arguments at the top with `Objects.requireNonNull`

QUICK SUMMARY

- Always pass-by-value
- Primitives: copy of the value
- Objects: copy of the reference
- No named/default params

Method Overloading

SAME NAME, DIFFERENT PARAMETERS

Overloading means defining several methods with the same name but different **parameter lists** (count, types, or order). The compiler chooses which one to call at **compile time** based on the arguments. It's how `println` accepts `int`, `String`, `double`, and more. Return type alone cannot distinguish overloads.

NOTES

- Differing only by return type is a compile error
- Parameter **names** don't count toward the signature
- The compiler picks the most specific match
- Constructors are commonly overloaded

OVERLOAD SET

```
static int    max(int a, int b)          { return a > b ? a : b; }
static double max(double a, double b)    { return a > b ? a : b; }
static int    max(int a, int b, int c)   { return max(max(a, b), c); }
static String max(String a, String b)    { return a.compareTo(b) >= 0 ? a : b; }

max(3, 7);           // 7       → max(int, int)
max(2.5, 1.0);       // 2.5     → max(double, double)
max(3, 9, 4);        // 9       → max(int, int, int)
max(3, 2.5);         // 3.0     → int widened to double
max("pear", "fig");  // "pear"

// Default-argument style
static void log(String msg)            { log(msg, "INFO"); }
static void log(String msg, String lvl){ System.out.println(lvl + ": " + msg); }
```

NOTES

→ `max(3, 2.5)` widens the `int`; no exact match exists

→ One overload delegating to another simulates default values

→ Overloads should do the same thing conceptually

RESOLUTION ORDER

PHASE	COMPILER TRIES
1	Exact match / widening (no boxing)
2	Boxing and unboxing
3	Varargs

VALID VS INVALID OVERLOADS

PAIR	VALID?
`f(int) / f(long)`	Yes
`f(int, String) / f(String, int)`	Yes, order differs
`f(int) / int f(int) returning other type`	No
`f(int a) / f(int b)`	No, same signature
`f(List<String>) / f(List<Integer>)`	No, erasure clash

⚠ COMMON MISTAKES

- `f(null)` with overloads `f(String)` and `f(Integer)` is ambiguous — cast the null
- Overloading `f(Object)` and `f(String)` and expecting runtime type to decide — the compiler uses the **declared** type
- Making overloads with different behavior; readers assume they're equivalent

✓ TIPS & BEST PRACTICES

- Delegate to the most complete overload to avoid duplicated logic
- Avoid overloads with the same number of parameters of similar types
- Overloading is compile-time; overriding (Sheet 64) is runtime

QUICK SUMMARY

- Same name, different params
- Chosen at compile time
- Return type alone isn't enough
- Delegate for default values

Return Values

GETTING RESULTS BACK

`return` ends a method and, for non-`void` methods, sends a value back to the caller. The returned value must match (or be assignable to) the declared return type. A Java method returns **one** value, but that value can be an array, collection, record, or `Optional` when you need more than a single number.

NOTES

- `void` methods may use a bare `return;` to exit early
- Code directly after `return` is unreachable (compile error)
- Every path in a non-void method must return or throw
- Prefer empty collections over `null`

RETURN PATTERNS

```
static boolean isEven(int n) { return n % 2 == 0; }   // expression

static String grade(int s) {                    // early returns
    if (s >= 90) return "A";
    if (s >= 80) return "B";
    return "C";
}

record MinMax(int min, int max) { }             // multiple values
static MinMax range(int[] a) {
    return new MinMax(Arrays.stream(a).min().orElseThrow(),
                      Arrays.stream(a).max().orElseThrow());
}
MinMax r = range(new int[]{4, 1, 9});
r.min(); r.max();                               // 1, 9

static Optional<User> find(String id) {         // maybe-absent
    return Optional.ofNullable(db.get(id));
}
static List<String> tags() { return List.of(); } // never null
```

NOTES

- → Returning a boolean expression beats `if … return true; else return false;`
- → A `record` names each returned value
- → `Optional` makes "not found" explicit

RETURN TYPE CHOICES

NEED	RETURN
Nothing	void
Yes / no	boolean
Single value	int, String, …
Several values	record or class
Many items	List<T>
Maybe nothing	Optional<T>

RULES

RULE	EXAMPLE
Must match type	int f() { return "x"; } ✖
All paths return	if-only branches ✖
Widening allowed	double f() { return 1; } ✔

⚠ COMMON MISTAKES

- `if (x > 0) return true; else return false;` — just `return x > 0;`
- Returning `null` for "no results" and causing NPEs downstream
- Returning an internal mutable list that callers can modify; return `List.copyOf` or an unmodifiable view

✓ TIPS & BEST PRACTICES

- Use guard-clause early returns to reduce nesting
- Prefer records over arrays like `int[]{min, max}` for multiple values
- Don't return `Optional` from getters of fields or for collections

QUICK SUMMARY

- `return value;` — send result
- `return;` — exit a void method
- Many values → record
- Absent value → `Optional`

Variable Scope

WHERE VARIABLES LIVE

Scope is the region of code where a name can be used. Local variables exist from their declaration to the end of the enclosing **block** `{ }`. Fields belong to an object (instance) or the class (`static`) and are visible throughout the class. An inner name with the same identifier **shadows** the outer one.

NOTES

- Java forbids redeclaring a local in a nested block
- A local can shadow a field; use `this.name` to reach the field
- Lambdas and inner classes can only capture **effectively final** locals
- Loop variables die when the loop ends

SCOPE LEVELS

```
public class Counter {
    static int created = 0;        // class scope: shared
    private int count = 0;         // instance scope: per object

    void add(int count) {          // parameter shadows field
        this.count += count;       // field via this
        created++;
    }

    void demo() {
        int total = 0;             // method scope
        for (int i = 0; i < 3; i++) {   // i: loop scope
            int sq = i * i;        // block scope
            total += sq;
        }
        // System.out.println(i);   ✖ i out of scope
        int base = 10;             // effectively final
        Runnable r = () -> System.out.println(base);   // ✔ captured
        // base = 11;              ✖ would break capture
    }
}
```

NOTES

- → `created` is shared by all `Counter` objects
- → `this.count` vs `count` resolves the shadowing
- → Effectively final = never reassigned after initialization

SCOPE LEVELS

KIND	VISIBLE IN	LIFETIME
`Block local`	Its { } block	Until block ends
`Method local / param`	The method	One call
`Loop variable`	The loop	Loop duration
`Instance field`	Whole class	As long as the object
`Static field`	Whole class (+ others if public)	Until class unloads

DEFAULT VALUES

KIND	GETS DEFAULT?
`Local variable`	No — must assign
`Instance field`	Yes (0, false, null)
`Static field`	Yes
`Array element`	Yes

⚠ COMMON MISTAKES

- Forgetting `this.` in a setter: `name = name;` assigns the parameter to itself
- Declaring a variable inside a loop when you need it after the loop
- Trying to modify a captured local inside a lambda

✓ TIPS & BEST PRACTICES

- Declare variables in the smallest scope that works
- Use different parameter names or `this.` consistently
- Avoid mutable `static` fields; they act like global state

QUICK SUMMARY

- Locals end at their `}`
- Fields span the class
- `this.x` — field when shadowed
- Lambdas capture effectively final locals

Static vs Instance Methods

CLASS-LEVEL VS OBJECT-LEVEL

An **instance method** runs on a specific object and can read and change that object's fields through `this`. A **static method** belongs to the class itself: it runs without an object and cannot touch instance fields directly. Use static for stateless utilities and factories; use instance methods for behavior that depends on an object's state.

NOTES

- Static methods have no `this`
- Instance methods can call static ones, not the other way around without an object
- Static methods can't be overridden, only hidden
- `Math.max`, `Integer.parseInt`, `List.of` are static

SIDE BY SIDE

```
public class Temperature {
    private final double celsius;

    private Temperature(double c) { this.celsius = c; }

    // static factory: no object yet
    public static Temperature ofFahrenheit(double f) {
        return new Temperature((f - 32) * 5 / 9);
    }

    // static utility: no state needed
    public static double cToF(double c) { return c * 9 / 5 + 32; }

    // instance: uses this object's field
    public boolean isFreezing() { return celsius <= 0; }

    public double inFahrenheit() { return cToF(celsius); }  // calls static
}

Temperature t = Temperature.ofFahrenheit(212);   // static call
t.isFreezing();                                  // false (instance)
Temperature.cToF(100);                           // 212.0
```

NOTES

→ `ofFahrenheit` — named constructor alternative

→ `cToF` — pure function: same input, same output

→ `isFreezing` — needs `celsius`, so it's instance

COMPARISON

ASPECT	STATIC	INSTANCE
Called on	Class	Object
Has this	No	Yes
Reads instance fields	No (needs object)	Yes
Overridable	No	Yes
Typical use	Utilities, factories	Object behavior

UTILITY CLASS PATTERN

ELEMENT	CODE
final class	public final class StringUtils
Private constructor	private StringUtils() { }
Static methods only	public static boolean isBlank(…)
Examples in JDK	Math, Arrays, Collections, Objects

⚠ COMMON MISTAKES

- Making everything `static` to avoid creating objects — you lose polymorphism and testability
- Calling a static method through an instance: `t.cToF(5)` compiles but misleads readers
- Storing per-user data in `static` fields, which then leak across users and threads

✓ TIPS & BEST PRACTICES

- If a method doesn't use `this`, consider making it `static`
- Prefer static factories with clear names over many constructors
- Give utility classes a private constructor

QUICK SUMMARY

- `static` — belongs to class
- Instance — belongs to object
- No `this` in static methods
- Static for pure helpers & factories

Lambda Expressions

FUNCTIONS AS VALUES

A **lambda** is a short anonymous function written as `(params) -> body`. It implements a **functional interface**, which is any interface with exactly one abstract method. Lambdas let you pass behavior to methods like `sort`, `forEach`, `removeIf`, and stream operations without writing a full class.

NOTES

- Parameter types are usually inferred
- One parameter: parentheses optional `x -> x * 2`
- Multi-statement bodies need `{ }` and an explicit `return`
- Captured locals must be effectively final

LAMBDA SYNTAX

```
Runnable hello = () -> System.out.println("hi");
Function<Integer, Integer> square = x -> x * x;
BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;
Predicate<String> empty = s -> s.isEmpty();
Consumer<String> shout = s -> System.out.println(s.toUpperCase());
Supplier<List<String>> maker = () -> new ArrayList<>();

square.apply(5);              // 25
add.apply(2, 3);              // 5
empty.test("");               // true
shout.accept("hey");          // HEY

Comparator<String> byLen = (a, b) -> {
    int d = Integer.compare(a.length(), b.length());
    return d != 0 ? d : a.compareTo(b);
};
names.sort(byLen);
names.removeIf(s -> s.startsWith("x"));
```

NOTES

- → Each functional interface has its own method name: `apply`, `test`, `accept`, `get`
- → Predicates combine with `and`, `or`, `negate`
- → `@FunctionalInterface` makes the compiler enforce one abstract method

CORE FUNCTIONAL INTERFACES (JAVA.UTIL.FUNCTION)

INTERFACE	METHOD	SHAPE
`Predicate<T>`	test	T → boolean
`Function<T,R>`	apply	T → R
`Consumer<T>`	accept	T → void
`Supplier<T>`	get	() → T
`UnaryOperator<T>`	apply	T → T
`BinaryOperator<T>`	apply	(T, T) → T
`BiFunction<T,U,R>`	apply	(T, U) → R

⚠ COMMON MISTAKES

- Modifying a captured local: `count++` inside a lambda won't compile
- Writing long, multi-line lambdas — extract a method and use a method reference
- Using `Function<Integer,Integer>` in hot loops; `IntUnaryOperator` avoids boxing

✓ TIPS & BEST PRACTICES

- Prefer method references when they read better (Sheet 41)
- Use primitive specializations: `IntPredicate`, `ToIntFunction`
- Name reusable lambdas as variables to document intent

QUICK SUMMARY

- `(args) -> expr` — lambda
- Implements a functional interface
- `Predicate` `Function` `Consumer` `Supplier`
- Captures effectively final vars

Method References

:: · static reference · bound instance · unbound instance · constructor reference · Comparator.comparing

NAMING AN EXISTING METHOD

A **method reference** (`Class::method`) is shorthand for a lambda that just calls an existing method. `String::toUpperCase` means `s -> s.toUpperCase()`. It's shorter and often clearer, and works anywhere a lambda of a compatible functional interface is expected.

NOTES

- Four kinds: static, bound instance, unbound instance, constructor
- No parentheses after the method name
- Overloaded methods are resolved by the target interface
- Use a lambda when you need extra arguments or logic

THE FOUR KINDS

```
// 1. Static method          x -> Integer.parseInt(x)
Function<String, Integer> parse = Integer::parseInt;

// 2. Bound instance (specific object)   s -> System.out.println(s)
Consumer<String> print = System.out::println;
String prefix = "Hi ";
Function<String, String> greet = prefix::concat;   // "Hi " + s

// 3. Unbound instance (first arg is the object)  s -> s.length()
Function<String, Integer> len = String::length;
BiFunction<String, String, Boolean> eq = String::equalsIgnoreCase;

// 4. Constructor            () -> new ArrayList<>()
Supplier<List<String>> mk = ArrayList::new;
Function<String, File> toFile = File::new;

names.forEach(System.out::println);
names.stream().map(String::trim)
     .filter(Predicate.not(String::isBlank)).toList();
people.sort(Comparator.comparing(Person::lastName)
                      .thenComparing(Person::firstName));
```

NOTES

- → Bound: the object is fixed when the reference is created
- → Unbound: the object is supplied as the first argument
- → `Predicate.not` (Java 11) negates a method reference
- → `Comparator.comparing` takes a key-extractor reference

REFERENCE → LAMBDA

REFERENCE	EQUIVALENT LAMBDA
`Math::abs`	x -> Math.abs(x)
`String::toLowerCase`	s -> s.toLowerCase()
`list::add`	x -> list.add(x)
`String::compareTo`	(a, b) -> a.compareTo(b)
`Person::new`	(n) -> new Person(n)

⚠ COMMON MISTAKES

- Writing `String::length()` with parentheses — compile error
- Using an unbound reference where the interface has no "object" parameter
- Ambiguity with overloaded methods like `Integer::toString` (static and instance) — use a lambda

✓ TIPS & BEST PRACTICES

- Use references when the lambda only forwards its arguments
- `toArray(String[]::new)` creates a correctly typed array from a stream
- Combine with `Comparator.comparing` for readable sorting

QUICK SUMMARY

- `Class::staticMethod`
- `obj::method` — bound
- `Class::method` — unbound
- `Class::new` — constructor

Recursion

METHODS THAT CALL THEMSELVES

A **recursive** method solves a problem by calling itself on a smaller version of the same problem. Every recursive method needs a **base case** that returns without recursing, and a **recursive case** that moves toward it. Each call adds a frame to the **call stack**, so very deep recursion throws `StackOverflowError`.

NOTES

- Java does **not** optimize tail calls
- Default stack handles roughly a few thousand to tens of thousands of frames
- Recursion suits trees, nested data, and divide-and-conquer
- Naive Fibonacci is exponential; memoize or iterate

CLASSIC EXAMPLES

```
static long factorial(int n) {
    if (n <= 1) return 1;                // base case
    return n * factorial(n - 1);         // recursive case
}
factorial(5);                            // 120

static int sumDigits(int n) {
    return n < 10 ? n : n % 10 + sumDigits(n / 10);
}
sumDigits(1234);                         // 10

static long fib(int n) {                 // O(2^n) — slow
    return n < 2 ? n : fib(n - 1) + fib(n - 2);
}

static Map<Integer, Long> memo = new HashMap<>();
static long fastFib(int n) {             // O(n) with memoization
    if (n < 2) return n;
    Long cached = memo.get(n);
    if (cached != null) return cached;
    long r = fastFib(n - 1) + fastFib(n - 2);
    memo.put(n, r);
    return r;
}
```

NOTES

- → `factorial(5)` = 5 × 4 × 3 × 2 × 1
- → Each call waits on the next; results unwind back up
- → Don't memoize with `HashMap.computeIfAbsent` recursively; it throws `ConcurrentModificationException`

CALL TRACE: FACTORIAL(4)

CALL	RETURNS
`factorial(4)`	4 × factorial(3) = 24
`factorial(3)`	3 × factorial(2) = 6
`factorial(2)`	2 × factorial(1) = 2
`factorial(1)`	1 (base case)

RECURSION VS ITERATION

ASPECT	RECURSION	ITERATION
`Readability`	Great for trees	Great for sequences
`Memory`	O(depth) stack	O(1) usually
`Stack overflow risk`	Yes	No
`Speed`	Call overhead	Faster

⚠ COMMON MISTAKES

- Missing or unreachable base case → `StackOverflowError`
- Recursive step that doesn't shrink the problem (`f(n)` calling `f(n)`)
- Using recursion for simple loops over large ranges

✓ TIPS & BEST PRACTICES

- Write the base case first
- Memoize overlapping subproblems or convert to a loop
- Use recursion for trees, directories, and parsing nested structures

QUICK SUMMARY

- Base case — stops recursion
- Recursive case — smaller input
- Each call uses stack space
- Memoize repeated work

Varargs

VARIABLE-LENGTH ARGUMENTS

Varargs (`Type... name`) lets a method accept zero or more arguments of a type. Inside the method, the parameter is an ordinary **array**. Callers can pass separate values, nothing at all, or an existing array. `String.format`, `List.of`, and `Arrays.asList` all use varargs.

NOTES

- Only one varargs parameter, and it must be **last**
- Varargs are chosen **last** in overload resolution
- Passing `null` alone is ambiguous with an array
- Generic varargs trigger "heap pollution" warnings

VARARGS IN USE

```
static int sum(int... nums) {          // nums is int[]
    int total = 0;
    for (int n : nums) total += n;
    return total;
}
sum();                    // 0
sum(5);                   // 5
sum(1, 2, 3);             // 6
sum(new int[]{4, 5});     // 9  (array passes directly)

static String label(String prefix, Object... parts) {   // must be last
    return prefix + ": " + Arrays.toString(parts);
}
label("vals", 1, "two", 3.0);   // "vals: [1, two, 3.0]"

static int max(int first, int... rest) {   // require at least one
    int m = first;
    for (int r : rest) m = Math.max(m, r);
    return m;
}
max(7);                   // 7
// max();                 ✖ compile error: needs one argument

@SafeVarargs
static <T> List<T> listOf(T... items) { return List.of(items); }
```

NOTES

- → `sum()` passes an empty array, never `null`
- → `first, int... rest` enforces a minimum count
- → `@SafeVarargs` asserts generic varargs are handled safely

RULES

RULE	EXAMPLE
`Must be last`	f(String s, int… n) ✔
`Only one`	f(int… a, int… b) ✖
`Empty call allowed`	sum() → empty array
`Array accepted`	sum(arr)

JDK VARARGS METHODS

METHOD	EXAMPLE
`String.format`	format("%s=%d", k, v)
`List.of`	List.of(1, 2, 3)
`Arrays.asList`	Arrays.asList("a", "b")
`Path.of`	Path.of("a", "b", "c.txt")
`EnumSet.of`	EnumSet.of(MON, TUE)

⚠ COMMON MISTAKES

- Overloading `f(int...)` and `f(Integer...)`, which causes ambiguous calls
- Calling `f(null)` — passes a null array, not an array containing null
- Using varargs in performance-critical code; every call allocates an array

✓ TIPS & BEST PRACTICES

- Use `first, rest...` to require at least one argument
- Provide fixed-arity overloads for common counts if performance matters (as `List.of` does)
- Prefer `List<T>` parameters when callers already have collections

QUICK SUMMARY

- `Type... name` — 0 or more args
- Arrives as an array
- Must be the last parameter
- Lowest overload priority

Arrays — 2D & Multi

ARRAYS OF ARRAYS

Java has no true matrix type. A 2D array is an **array of arrays**: `grid[r]` is a row (itself an array), and `grid[r][c]` is one element. Because rows are separate objects, they can have **different lengths**, which is called a **jagged** array. The same idea extends to 3D and beyond.

NOTES

- `grid.length` = number of rows
- `grid[r].length` = columns in row `r`
- `Arrays.deepToString` prints nested arrays
- Row-by-row iteration is fastest (cache-friendly)

CREATING & TRAVERSING

```
int[][] grid = new int[3][4];            // 3 rows × 4 cols, all 0
int[][] m = {
    {1, 2, 3},
    {4, 5, 6}
};
m[1][2];                                 // 6
m.length;                                // 2 rows
m[0].length;                             // 3 cols

for (int r = 0; r < m.length; r++) {
    for (int c = 0; c < m[r].length; c++) {
        System.out.print(m[r][c] + " ");
    }
    System.out.println();
}
for (int[] row : m)                      // for-each version
    for (int v : row) total += v;

int[][] jag = new int[3][];              // rows sized later
jag[0] = new int[]{1};
jag[1] = new int[]{1, 2};
jag[2] = new int[]{1, 2, 3};
System.out.println(Arrays.deepToString(jag));   // [[1], [1, 2], [1, 2, 3]]
```

NOTES

- → Always use `m[r].length` for the inner bound (safe for jagged)
- → `new int[3][]` leaves rows `null` until assigned
- → `Arrays.toString(m)` prints row addresses, not values

COMMON OPERATIONS

TASK	CODE
Rows / cols	m.length / m[0].length
Get cell	m[r][c]
Copy a row	m[r].clone()
Fill all	for (int[] row : m) Arrays.fill(row, -1)
Print	Arrays.deepToString(m)
Compare	Arrays.deepEquals(a, b)

MATRIX PATTERNS

PATTERN	INDEX MATH
Transpose	t[c][r] = m[r][c]
Main diagonal	m[i][i]
Anti-diagonal	m[i][n - 1 - i]
4 neighbors	(r±1, c), (r, c±1)
3D array	int[][][] cube = new int[2][3][4]

⚠ COMMON MISTAKES

- Using `m[0].length` for every row of a jagged array → index errors
- `int[][] copy = m.clone();` copies only the outer array; rows are still shared
- Swapping indexes: `m[c][r]` instead of `m[r][c]`

✓ TIPS & BEST PRACTICES

- Use `r` / `c` (or `row` / `col`) names, never `i` / `j`, for grids
- Check bounds before reading neighbors: `r >= 0 && r < m.length`
- For real linear algebra, use a library (EJML, Apache Commons Math)

QUICK SUMMARY

- `type[][]` — array of arrays
- `m[r][c]` — row, then column
- Rows may differ in length
- `deepToString` — print

ArrayList

RESIZABLE LISTS

`ArrayList<E>` is the go-to `List` implementation: a **resizable array** that grows automatically. It gives fast index access (`get`, `set`) and fast appends, but inserting or removing in the middle shifts elements. Declare variables as the interface type `List<E>` so you can swap implementations later.

NOTES

- Holds objects only: `List<Integer>`, not `List<int>`
- `List.of(...)` creates an **immutable** list
- `remove(int)` removes by index; `remove(Object)` by value
- Not thread-safe

CORE OPERATIONS

```
List<String> list = new ArrayList<>();
list.add("Ada");                    // [Ada]
list.add("Grace");                  // [Ada, Grace]
list.add(1, "Alan");                // [Ada, Alan, Grace]
list.get(0);                        // "Ada"
list.set(0, "Lovelace");            // [Lovelace, Alan, Grace]
list.size();                        // 3
list.contains("Alan");              // true
list.indexOf("Grace");              // 2
list.remove("Alan");                // by value → true
list.remove(0);                     // by index → "Lovelace"
list.isEmpty();                     // false

List<Integer> nums = new ArrayList<>(List.of(5, 3, 9));
nums.remove(Integer.valueOf(3));    // value 3, not index 3
nums.addAll(List.of(1, 2));         // [5, 9, 1, 2]
nums.sort(null);                    // [1, 2, 5, 9]
nums.removeIf(n -> n > 4);          // [1, 2]
List<Integer> fixed = List.of(1, 2);
fixed.add(3);                       // ✖ UnsupportedOperationException
```

NOTES

- → `<>` (diamond) infers the type argument
- → Wrap `List.of` in `new ArrayList<>(…)` to get a mutable copy
- → `sort(null)` uses natural order

METHODS

METHOD	PURPOSE	BIG-O
`add(e)`	Append	O(1)*
`add(i, e)`	Insert at index	O(n)
`get(i) / set(i, e)`	Read / replace	O(1)
`remove(i)`	Remove at index	O(n)
`contains(o)`	Linear search	O(n)
`clear()`	Remove all	O(n)

ARRAY VS ARRAYLIST

FEATURE	ARRAY	ARRAYLIST
`Size`	Fixed	Grows
`Primitives`	Yes	No (boxed)
`Length`	a.length	list.size()
`Access`	a[i]	list.get(i)
`Utilities`	Arrays.*	Built-in + Collections.*

⚠ COMMON MISTAKES

- `nums.remove(3)` on a `List<Integer>` removes **index** 3, not the value 3
- Modifying an immutable `List.of` list
- Removing items inside a for-each loop → `ConcurrentModificationException`

✓ TIPS & BEST PRACTICES

- Declare as `List<T>`; construct as `new ArrayList<>()`
- Pre-size big lists: `new ArrayList<>(10_000)`
- Use `removeIf` and `replaceAll` instead of manual loops

QUICK SUMMARY

- `add` `get` `set` `remove` `size`
- Resizable, index-based
- `List.of` — immutable
- Declare by interface `List`

LinkedList

DOUBLY LINKED LIST

`LinkedList<E>` stores elements in **nodes** linked forward and backward. Adding or removing at either **end** is O(1), but reaching index `i` means walking node by node, so `get(i)` is O(n). It implements both `List` and `Deque`, so it can act as a list, queue, or stack.

NOTES

- Each element carries two extra references: more memory than `ArrayList`
- In practice `ArrayList` and `ArrayDeque` are faster for most work
- Allows `null` elements (unlike `ArrayDeque`)
- Iterator removal is O(1) per element

USAGE

```
LinkedList<String> ll = new LinkedList<>();
ll.add("B");                  // [B]
ll.addFirst("A");             // [A, B]
ll.addLast("C");              // [A, B, C]
ll.getFirst();                // "A"
ll.getLast();                 // "C"
ll.removeFirst();             // "A"  → [B, C]
ll.removeLast();              // "C"  → [B]

ll.offer("X");                // queue: add to tail
ll.peek();                    // "B"  (head, no remove)
ll.poll();                    // "B"  (head, removes)
ll.poll();                    // "X"
ll.poll();                    // null (empty, no exception)
// ll.removeFirst();          // ✖ NoSuchElementException when empty

ListIterator<String> it = names.listIterator();
while (it.hasNext()) {
    if (it.next().startsWith("x")) it.remove();   // O(1) unlink
}
```

NOTES

- → `poll` / `peek` return `null` when empty
- → `remove` / `get…` methods throw when empty
- → `ListIterator` can also `add` and `set` mid-walk

THROW VS RETURN-NULL METHODS

ACTION	THROWS IF EMPTY	RETURNS NULL
Read head	getFirst() / element()	peekFirst() / peek()
Remove head	removeFirst() / remove()	pollFirst() / poll()
Read tail	getLast()	peekLast()
Remove tail	removeLast()	pollLast()

LINKEDLIST VS ARRAYLIST

OPERATION	LINKEDLIST	ARRAYLIST
get(i)	O(n)	O(1)
Add/remove at ends	O(1)	O(1) end, O(n) front
Insert mid (with iterator)	O(1)	O(n)
Memory per element	High	Low

⚠ COMMON MISTAKES

- Looping `for (i…) ll.get(i)` — O(n²) overall
- Choosing `LinkedList` "for fast inserts" without measuring; `ArrayList` usually wins
- Calling `removeFirst()` on an empty list instead of `pollFirst()`

✓ TIPS & BEST PRACTICES

- Need a queue or stack? Prefer `ArrayDeque` (Sheet 51)
- Use `LinkedList` when you need `null` elements in a queue
- Always iterate with for-each or an iterator, never by index

QUICK SUMMARY

- Doubly linked nodes
- O(1) at ends, O(n) by index
- `List` + `Deque`
- `poll` / `peek` → null if empty

HashMap

KEY → VALUE LOOKUP

`HashMap<K, V>` stores **key-value pairs** and finds values by key in about O(1) time using each key's `hashCode()` and `equals()`. Keys are unique; putting an existing key replaces its value. Iteration order is **not guaranteed**. Use `LinkedHashMap` for insertion order or `TreeMap` for sorted keys.

NOTES

- Allows one `null` key and many `null` values
- `get` returns `null` for missing keys
- Custom key classes must override `equals` **and** `hashCode`
- `Map.of(...)` is immutable and rejects nulls

MAP OPERATIONS

```
Map<String, Integer> stock = new HashMap<>();
stock.put("apple", 5);                    // {apple=5}
stock.put("pear", 2);
stock.put("apple", 7);                    // replaces → 7
stock.get("apple");                       // 7
stock.get("kiwi");                        // null
stock.getOrDefault("kiwi", 0);            // 0
stock.containsKey("pear");                // true
stock.putIfAbsent("pear", 99);            // stays 2
stock.remove("pear");                     // 2
stock.size();                             // 1

Map<String, Integer> counts = new HashMap<>();
for (String w : words)
    counts.merge(w, 1, Integer::sum);     // word count

Map<String, List<String>> byDept = new HashMap<>();
byDept.computeIfAbsent("eng", k -> new ArrayList<>()).add("Ada");

for (var e : stock.entrySet())
    System.out.println(e.getKey() + "=" + e.getValue());
```

NOTES

- → `merge(k, 1, sum)` — insert 1 or add 1
- → `computeIfAbsent` — create the bucket list on first use
- → `entrySet()` — key and value without extra lookups

METHODS

METHOD	RETURNS
`put(k, v)`	Previous value or null
`get(k)`	Value or null
`getOrDefault(k, d)`	Value or d
`remove(k)`	Removed value
`keySet() / values()`	Views of keys / values
`compute / merge`	Update in place

MAP IMPLEMENTATIONS

TYPE	ORDER	NULL KEYS	GET/PUT
`HashMap`	None	One	O(1)
`LinkedHashMap`	Insertion	One	O(1)
`TreeMap`	Sorted	No	O(log n)
`ConcurrentHashMap`	None	No	O(1), thread-safe

⚠ COMMON MISTAKES

- `int c = map.get(k);` NPE when the key is missing — use `getOrDefault`
- Mutating a key object after inserting it; its hash changes and it's lost
- Assuming `HashMap` iteration order is stable or sorted

✓ TIPS & BEST PRACTICES

- Use `merge` for counters and `computeIfAbsent` for grouping
- Use `EnumMap` when keys are enum constants
- Use `Map.entry(k, v)` and `Map.ofEntries` for small constant maps

QUICK SUMMARY

- `put` / `get` — O(1) lookup
- Unique keys, no order
- `getOrDefault` `merge` `computeIfAbsent`
- Keys need `equals` + `hashCode`

HashSet

UNIQUE ELEMENTS

A `Set` holds **no duplicates**. `HashSet<E>` is backed by a `HashMap`, so `add`, `remove`, and `contains` are about O(1). `add` returns `false` if the element was already present. Sets are ideal for membership tests, removing duplicates, and math-style unions and intersections.

NOTES

- No index: there is no `get(i)`
- Iteration order is unpredictable
- Elements need correct `equals` and `hashCode`
- `Set.of(...)` is immutable and throws on duplicate arguments

SET OPERATIONS

```
Set<String> tags = new HashSet<>();
tags.add("java");                       // true
tags.add("java");                       // false (duplicate)
tags.add("sql");
tags.contains("sql");                   // true
tags.remove("sql");                     // true
tags.size();                            // 1

List<Integer> dupes = List.of(3, 1, 3, 2, 1);
Set<Integer> unique = new HashSet<>(dupes);           // [1, 2, 3]
var ordered = new ArrayList<>(new LinkedHashSet<>(dupes));   // [3, 1, 2]

Set<Integer> a = new HashSet<>(Set.of(1, 2, 3, 4));
Set<Integer> b = Set.of(3, 4, 5);
Set<Integer> union = new HashSet<>(a);  union.addAll(b);     // [1,2,3,4,5]
Set<Integer> inter = new HashSet<>(a);  inter.retainAll(b);  // [3, 4]
Set<Integer> diff  = new HashSet<>(a);  diff.removeAll(b);   // [1, 2]
a.containsAll(Set.of(1, 2));                                  // true (subset)
```

NOTES

- → `LinkedHashSet` removes duplicates but keeps first-seen order
- → Set algebra methods **modify** the receiver; copy first
- → `containsAll` checks for a subset

SET ALGEBRA

OPERATION	METHOD	MATH
Union	a.addAll(b)	A ∪ B
Intersection	a.retainAll(b)	A ∩ B
Difference	a.removeAll(b)	A − B
Subset test	a.containsAll(b)	B ⊆ A

SET IMPLEMENTATIONS

TYPE	ORDER	SPEED
HashSet	None	O(1)
LinkedHashSet	Insertion	O(1)
TreeSet	Sorted	O(log n)
EnumSet	Enum declaration	Fastest (bit vector)

⚠ COMMON MISTAKES

- Expecting sorted or insertion order from `HashSet`
- Storing mutable objects and changing them after insertion
- Using `list.contains` in a loop over large data — convert to a `Set` first

✓ TIPS & BEST PRACTICES

- Convert lists to sets for O(1) membership checks
- Use `LinkedHashSet` to dedupe while keeping order
- Use `Set.copyOf(collection)` for an immutable snapshot

QUICK SUMMARY

- No duplicates
- `add` returns false if present
- O(1) `contains`
- `addAll` `retainAll` `removeAll` — set math

TreeMap & TreeSet

SORTED COLLECTIONS

`TreeMap` and `TreeSet` keep elements **sorted** at all times using a red-black tree, so operations are O(log n). Keys are ordered by their natural order (`Comparable`) or a `Comparator` you supply. They also implement `NavigableMap`/`NavigableSet`, adding range queries and nearest-match lookups.

NOTES

- No `null` keys (they can't be compared)
- Keys must be `Comparable` or you must pass a `Comparator`
- Views like `headMap` are **live**: changes flow both ways
- Slower than hash versions; use only when order matters

NAVIGABLE OPERATIONS

```
TreeMap<Integer, String> grades = new TreeMap<>(Map.of(
    90, "A", 80, "B", 70, "C", 60, "D"));
grades.firstKey();              // 60
grades.lastKey();               // 90
grades.floorKey(85);            // 80   (≤ 85)
grades.ceilingKey(85);          // 90   (≥ 85)
grades.floorEntry(77).getValue();   // "C"  → grade lookup
grades.headMap(80);             // {60=D, 70=C}       (< 80)
grades.tailMap(80, true);       // {80=B, 90=A}       (≥ 80)
grades.descendingMap();         // {90=A, 80=B, 70=C, 60=D}

TreeSet<String> words = new TreeSet<>(List.of("pear", "apple", "fig"));
words;                          // [apple, fig, pear]
words.first();                  // "apple"
words.higher("fig");            // "pear"
words.pollFirst();              // removes "apple"

TreeSet<String> byLen = new TreeSet<>(
    Comparator.comparing(String::length).thenComparing(s -> s));
```

NOTES

- → `floorEntry` — perfect for range lookups like grade cutoffs
- → `true` flags make range ends inclusive
- → A comparator returning 0 means "duplicate" to a TreeSet

NAVIGATION METHODS

METHOD	RETURNS
`floor(x)`	Greatest ≤ x
`ceiling(x)`	Smallest ≥ x
`lower(x)`	Greatest < x
`higher(x)`	Smallest > x
`first() / last()`	Ends
`pollFirst() / pollLast()`	Remove ends

RANGE VIEWS

METHOD	RANGE
`headMap(k) / headSet(k)`	< k
`tailMap(k) / tailSet(k)`	≥ k
`subMap(a, b)`	a ≤ key < b
`descendingMap()`	Reverse order

⚠ COMMON MISTAKES

- A comparator that ignores fields — the TreeSet silently drops "equal" items
- Inserting `null` → `NullPointerException`
- Using `TreeMap` where `HashMap` would do, paying O(log n) for nothing

✓ TIPS & BEST PRACTICES

- Use `floorEntry` / `ceilingEntry` for thresholds and time-based lookups
- Chain `thenComparing` so the comparator never ties unintentionally
- `new TreeMap<>(hashMap)` gives a sorted snapshot

QUICK SUMMARY

- Always sorted, O(log n)
- `floor` `ceiling` `higher` `lower`
- `headMap` `tailMap` — ranges
- Needs `Comparable` or `Comparator`

Stack & Queue

LIFO AND FIFO

A **stack** is last-in, first-out (LIFO): `push` adds to the top and `pop` takes from the top. A **queue** is first-in, first-out (FIFO): `offer` adds to the back and `poll` takes from the front. Java's legacy `Stack` class still works, but `Deque` (Sheet 51) is the recommended implementation for both.

NOTES

- `Stack` extends `Vector`, so it's synchronized and slower
- `Queue` is an interface; use `ArrayDeque` or `LinkedList`
- `PriorityQueue` polls the **smallest** element first
- `offer` / `poll` / `peek` don't throw on full or empty

STACK & QUEUE USAGE

```
Deque<Integer> stack = new ArrayDeque<>();   // preferred stack
stack.push(1); stack.push(2); stack.push(3);
stack.peek();               // 3
stack.pop();                // 3
stack.pop();                // 2

Queue<String> q = new ArrayDeque<>();        // FIFO
q.offer("a"); q.offer("b"); q.offer("c");
q.peek();                   // "a"
q.poll();                   // "a"
q.size();                   // 2

PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 4));
pq.poll();                  // 1 (smallest)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

Stack<Character> legacy = new Stack<>();     // works, but legacy
legacy.push('x');
legacy.pop();               // 'x'
```

NOTES

- Use `Deque` methods `push` / `pop` for stacks
- `PriorityQueue` iteration order is **not** sorted; only `poll` order is
- `reverseOrder()` turns it into a max-heap

QUEUE METHODS

ACTION	THROWS	RETURNS SPECIAL VALUE
Add	add(e)	offer(e) → false
Remove head	remove()	poll() → null
Read head	element()	peek() → null

CHOOSING AN IMPLEMENTATION

NEED	USE
Stack	ArrayDeque (push/pop)
FIFO queue	ArrayDeque (offer/poll)
Priority order	PriorityQueue
Thread-safe queue	ArrayBlockingQueue / ConcurrentLinkedQueue
Legacy code	Stack

⚠ COMMON MISTAKES

- Using `Stack` in new code; it's synchronized and inherits index methods that break LIFO
- Printing a `PriorityQueue` and expecting sorted output
- Calling `pop()` on an empty `ArrayDeque` → `NoSuchElementException`; check `isEmpty()` or use `poll`

✓ TIPS & BEST PRACTICES

- Declare stacks and queues as `Deque<T>` / `Queue<T>`
- Use a stack for undo, bracket matching, and DFS
- Use a queue for BFS, task processing, and buffering

QUICK SUMMARY

- Stack — LIFO: `push` `pop` `peek`
- Queue — FIFO: `offer` `poll` `peek`
- `PriorityQueue` — smallest first
- Prefer `ArrayDeque`

Deque

DOUBLE-ENDED QUEUE

A Deque ("deck") supports adding and removing at **both ends** in O(1). ArrayDeque is a resizable circular array and is the best general-purpose stack *and* queue in Java. Methods come in pairs for the first and last positions, each with a throwing and a non-throwing version.

NOTES

- `ArrayDeque` rejects `null` elements
- Faster than `Stack` and usually faster than `LinkedList`
- Not thread-safe; use `LinkedBlockingDeque` for concurrency
- `descendingIterator()` walks tail → head

DEQUE OPERATIONS

```
Deque<Integer> d = new ArrayDeque<>();
d.addFirst(2);              // [2]
d.addFirst(1);              // [1, 2]
d.addLast(3);               // [1, 2, 3]
d.offerLast(4);             // [1, 2, 3, 4]
d.peekFirst();              // 1
d.peekLast();               // 4
d.pollFirst();              // 1 → [2, 3, 4]
d.pollLast();               // 4 → [2, 3]

// Palindrome check
Deque<Character> chars = new ArrayDeque<>();
for (char c : "racecar".toCharArray()) chars.addLast(c);
boolean pal = true;
while (chars.size() > 1)
    if (!chars.pollFirst().equals(chars.pollLast())) pal = false;

// Sliding-window maximum (indices kept in decreasing value order)
Deque<Integer> win = new ArrayDeque<>();
for (int i = 0; i < a.length; i++) {
    while (!win.isEmpty() && a[win.peekLast()] <= a[i]) win.pollLast();
    win.addLast(i);
    if (win.peekFirst() <= i - k) win.pollFirst();
}
```

NOTES

- → Comparing `Character` objects needs `.equals`
- → The window deque stores **indexes**, not values
- → `a[win.peekFirst()]` is the current window max

METHOD PAIRS

OPERATION	FIRST (HEAD)	LAST (TAIL)
Insert (throws)	addFirst	addLast
Insert (returns false)	offerFirst	offerLast
Remove (throws)	removeFirst	removeLast
Remove (null)	pollFirst	pollLast
Examine (null)	peekFirst	peekLast

DEQUE AS STACK / QUEUE

ROLE	ADD	REMOVE	PEEK
Stack	push = addFirst	pop = removeFirst	peek
Queue	offer = offerLast	poll = pollFirst	peek

⚠ COMMON MISTAKES

- Adding `null` to an `ArrayDeque` → `NullPointerException`
- Mixing `push` and `offer` on the same deque and confusing which end is which
- Comparing boxed `Character` / `Integer` values with `==`

✓ TIPS & BEST PRACTICES

- Use `ArrayDeque` as your default stack and queue
- Stick to one naming style: either stack methods or queue methods
- Monotonic deques solve many "window" problems in O(n)

QUICK SUMMARY

- O(1) at both ends
- `…First` / `…Last` method pairs
- `poll` / `peek` → null when empty
- Best stack + queue: `ArrayDeque`

Generics

TYPE-SAFE REUSABLE CODE

Generics let classes and methods work with any type while keeping compile-time type checking. `List<String>` only accepts strings, so no casts are needed when reading. You declare **type parameters** like <T>, can **bound** them (`<T extends Number>`), and can use **wildcards** (`?`) in parameters for flexible APIs.

NOTES

- Generics work with reference types only: `Box<Integer>`, not `Box<int>`
- Types are **erased** at runtime: `List<String>` and `List<Integer>` share one class
- You can't `new T()` or create `new T[n]`
- PECS: Producer `extends`, Consumer `super`

CLASSES, METHODS, BOUNDS

```
class Box<T> {                                  // generic class
    private T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
}
Box<String> b = new Box<>("hi");
String s = b.get();                             // no cast

record Pair<K, V>(K key, V value) { }           // two params
var p = new Pair<>("age", 36);

static <T> T first(List<T> list) { return list.get(0); }   // generic method
static <T extends Comparable<T>> T max(T a, T b) {         // bounded
    return a.compareTo(b) >= 0 ? a : b;
}
max(3, 7);                                      // 7
max("fig", "kiwi");                             // "kiwi"

static double sum(List<? extends Number> nums) { // reads Numbers
    double t = 0;
    for (Number n : nums) t += n.doubleValue();
    return t;
}
static void fill(List<? super Integer> out) { out.add(42); }  // writes Integers
```

NOTES

- → `<T>` before the return type declares a generic method
- → `T extends Comparable<T>` guarantees `compareTo` exists
- → `? extends Number` accepts `List<Integer>`, `List<Double>`
- → `? super Integer` accepts `List<Number>`, `List<Object>`

TYPE PARAMETER CONVENTIONS

LETTER	MEANING
`T`	Type
`E`	Element (collections)
`K / V`	Key / Value
`R`	Return type

WILDCARDS

SYNTAX	ACCEPTS	CAN READ	CAN ADD
`List<?>`	Any list	Object	Only null
`List<? extends T>`	T or subtypes	T	No
`List<? super T>`	T or supertypes	Object	T
`List<T>`	Exactly T	T	T

⚠ COMMON MISTAKES

- Assuming `List<Integer>` is a `List<Number>` — it isn't; use `List<? extends Number>`
- Using raw types (`List list`) which disable type checking
- `if (obj instanceof List<String>)` — not allowed due to erasure

✓ TIPS & BEST PRACTICES

- Follow PECS when designing method parameters
- Let the diamond `<>` and `var` infer types
- Add bounds when you need methods of the bound type

QUICK SUMMARY

- `<T>` — type parameter
- `<T extends X>` — bounded
- `? extends` read, `? super` write
- Erased at runtime

Streams

DECLARATIVE DATA PIPELINES

A **stream** is a sequence of elements processed through a **pipeline**: a source (a collection, array, or generator), zero or more **intermediate** operations (`filter`, `map`, `sorted`) that build a recipe, and one **terminal** operation (`toList`, `count`, `forEach`) that runs it. Streams are **lazy** and **single-use**, and they never modify the source.

NOTES

- Nothing executes until a terminal operation is called
- A stream can be consumed only once
- `IntStream` / `LongStream` / `DoubleStream` avoid boxing
- `.toList()` (Java 16+) returns an unmodifiable list

BUILDING PIPELINES

```
List<String> names = List.of("Ada", "alan", "Grace", "Al");

List<String> result = names.stream()          // source
    .filter(n -> n.length() > 2)              // intermediate
    .map(String::toUpperCase)                 // intermediate
    .sorted()                                 // intermediate
    .toList();                                // terminal → [ADA, ALAN, GRACE]

long count = names.stream().filter(n -> n.startsWith("A")).count();  // 2
Stream.of(3, 1, 2).sorted().toList();         // [1, 2, 3]
Arrays.stream(new int[]{4, 5, 6}).sum();      // 15
IntStream.rangeClosed(1, 5).map(i -> i * i).toArray();   // [1,4,9,16,25]
Stream.iterate(1, x -> x * 2).limit(5).toList();         // [1,2,4,8,16]

Stream<String> s = names.stream();
s.count();
s.count();                                    // ✖ IllegalStateException

names.stream().peek(System.out::println)      // lazy: prints nothing…
     .map(String::length);                    // …no terminal op
```

NOTES

- → Read top to bottom: what, then how, then collect
- → `rangeClosed` includes the upper bound; `range` excludes it
- → `iterate` + `limit` builds finite sequences
- → `peek` is for debugging only

OPERATION TYPES

INTERMEDIATE (LAZY)	TERMINAL (RUNS PIPELINE)
filter, map, flatMap	toList, collect
sorted, distinct	forEach
limit, skip	count, sum, average
peek, mapToInt	reduce, min, max
takeWhile, dropWhile	anyMatch, findFirst

STREAM SOURCES

SOURCE	CODE
Collection	list.stream()
Array	Arrays.stream(arr)
Values	Stream.of(a, b, c)
Range	IntStream.range(0, n)
File lines	Files.lines(path)
Generator	Stream.generate(Math::random)

⚠ COMMON MISTAKES

- Reusing a stream after a terminal operation
- Forgetting a terminal operation and wondering why nothing happens
- Using `forEach` to add to an external list; use `collect` / `toList` instead

✓ TIPS & BEST PRACTICES

- Use `mapToInt(...).sum()` instead of `reduce` for numeric totals
- Keep lambdas short; extract complex logic to methods
- Use a plain loop when it's clearer, especially with early exits or checked exceptions

QUICK SUMMARY

- Source → intermediate → terminal
- Lazy, single-use
- Never modifies source
- `toList()` — collect results

map & filter

TRANSFORMING AND SELECTING

`filter(predicate)` keeps only elements that pass a test. `map(function)` transforms each element into a new value, possibly of a different type. `flatMap` maps each element to a **stream** and flattens the results into one stream, which is how you handle nested lists. Chaining these builds most real-world pipelines.

NOTES

- `filter` never changes elements, only drops some
- `map` produces exactly one output per input
- `flatMap` produces zero or more outputs per input
- Order of operations affects performance: filter early

EXAMPLES

```
record Emp(String name, String dept, double salary, List<String> skills) { }

List<String> engNames = emps.stream()
    .filter(e -> e.dept().equals("eng"))
    .map(Emp::name)
    .toList();                                  // ["Ada", "Alan"]

double total = emps.stream()
    .mapToDouble(Emp::salary).sum();            // DoubleStream → sum

List<String> allSkills = emps.stream()
    .flatMap(e -> e.skills().stream())          // List<List> → flat
    .distinct()
    .sorted()
    .toList();                                  // [java, sql, …]

Stream.of(1, 2, 3, 10, 4).takeWhile(n -> n < 5).toList();  // [1, 2, 3]
Stream.of(1, 2, 3, 4, 5).skip(1).limit(3).toList();        // [2, 3, 4]
List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3));
nested.stream().flatMap(List::stream).toList();            // [1, 2, 3]
```

NOTES

- → `Emp::name` — record accessor as a method reference
- → `mapToDouble` switches to a primitive stream for `sum`
- → `takeWhile` stops at the first failure (unlike `filter`)

OPERATION SUMMARY

OPERATION	IN → OUT	EXAMPLE
`filter`	n → ≤ n	filter(x -> x > 0)
`map`	n → n	map(String::trim)
`flatMap`	n → many	flatMap(List::stream)
`distinct`	n → ≤ n	removes duplicates
`limit / skip`	n → ≤ n	first / after k
`takeWhile`	n → prefix	until test fails

PRIMITIVE MAPPING

METHOD	PRODUCES
`mapToInt`	IntStream
`mapToLong`	LongStream
`mapToDouble`	DoubleStream
`mapToObj`	Stream<T> (from primitive)

⚠ COMMON MISTAKES

- Using `map` when the function returns a list, producing `Stream<List<T>>` — use `flatMap`
- Calling `filter` after expensive `map` calls that could have been skipped
- Side effects inside `map` (like incrementing counters)

✓ TIPS & BEST PRACTICES

- Filter before mapping to reduce work
- Use `mapToInt` / `mapToDouble` for numeric aggregates
- `Optional::stream` in `flatMap` drops empty Optionals cleanly

QUICK SUMMARY

- `filter` — keep matching
- `map` — transform 1→1
- `flatMap` — 1→many, flattened
- `mapToX` — primitive streams

reduce & Collectors

AGGREGATING RESULTS

Terminal operations turn a stream into a result. `reduce` folds elements into one value with an accumulator. `collect` uses a **Collector** to build a container: lists, sets, maps, joined strings, or grouped summaries. The `Collectors` class provides ready-made collectors, and many can be nested.

NOTES

- `reduce(identity, op)` returns a value; without an identity it returns `Optional`
- `groupingBy` returns `Map<K, List<V>>` by default
- `toMap` throws on duplicate keys unless you pass a merge function

COLLECTING

```
int sum = Stream.of(1, 2, 3, 4).reduce(0, Integer::sum);        // 10
Optional<Integer> prod =
    Stream.of(2, 3, 4).reduce((a, b) -> a * b);                 // 24

String csv = names.stream()
    .collect(Collectors.joining(", ", "[", "]"));       // "[Ada, Alan]"
Map<String, List<Emp>> byDept = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept));

Map<String, Long> countByDept = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept, Collectors.counting()));
                                                        // {eng=2, ops=1}
Map<String, Double> avgPay = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept,
             Collectors.averagingDouble(Emp::salary)));

var evenOdd = IntStream.range(0, 6).boxed()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));  // {false=[1,3,5], …}
Map<String, Double> pay = emps.stream()
    .collect(Collectors.toMap(Emp::name, Emp::salary, (a, b) -> a));
```

NOTES

- `joining(sep, prefix, suffix)`
- The second `groupingBy` argument is a **downstream** collector
- `partitioningBy` always has both `true` and `false` keys
- `(a, b) -> a` keeps the first value on duplicate keys

COLLECTORS

COLLECTOR	RESULT
`toList() / toSet()`	List / Set
`toMap(k, v)`	Map
`joining(sep)`	String
`groupingBy(f)`	Map<K, List<T>>
`partitioningBy(p)`	Map<Boolean, List<T>>
`counting()`	Long

DOWNSTREAM HELPERS

COLLECTOR	COMPUTES
`summingInt(f)`	Total
`maxBy(cmp)`	Optional max
`mapping(f, toList())`	Transformed list

⚠ COMMON MISTAKES

- `toMap` with duplicate keys → `IllegalStateException`; supply a merge function
- Using `reduce` to build collections — use `collect`
- Expecting `groupingBy` to keep insertion order; pass `LinkedHashMap::new` or `TreeMap::new`

✓ TIPS & BEST PRACTICES

- Use `stream.toList()` for plain lists; `Collectors` for everything else
- Nest `mapping` inside `groupingBy` for "names by dept"
- `IntSummaryStatistics` gives all numeric stats in one pass

QUICK SUMMARY

- `reduce` — fold to one value
- `collect` — build a container
- `groupingBy` + downstream
- `joining` `toMap` `partitioningBy`

Sorting & Searching

ORDERING AND FINDING DATA

Java sorts arrays with `Arrays.sort` and lists with `list.sort(comparator)` or `Collections.sort`. Without a comparator, elements use their **natural order** (`Comparable`). Object sorts are **stable**, so equal elements keep their relative order. Once sorted, `binarySearch` finds elements in O(log n).

NOTES

- Primitives sort with dual-pivot quicksort; objects with TimSort (stable)
- `binarySearch` on unsorted data gives meaningless results
- Not found → `binarySearch` returns `-(insertionPoint) - 1`
- Defining `Comparable` and `Comparator` is covered on Sheet 101

SORTING EXAMPLES

```
int[] nums = {5, 2, 9, 1};
Arrays.sort(nums);                                  // [1, 2, 5, 9]
Arrays.sort(nums, 1, 3);                            // sort a range only

List<String> words = new ArrayList<>(List.of("pear", "Fig", "apple"));
Collections.sort(words);                            // [Fig, apple, pear]
words.sort(String.CASE_INSENSITIVE_ORDER);          // [apple, Fig, pear]
words.sort(Comparator.reverseOrder());              // [pear, apple, Fig]
words.sort(Comparator.comparing(String::length));   // [Fig, pear, apple]

Integer[] boxed = {3, 1, 2};
Arrays.sort(boxed, Collections.reverseOrder());     // [3, 2, 1]

emps.sort(Comparator.comparing(Emp::dept)
          .thenComparing(Emp::salary, Comparator.reverseOrder()));

var sortedCopy = words.stream().sorted().toList();  // source unchanged
int[] sorted = {1, 3, 5, 7};
Arrays.binarySearch(sorted, 5);                     // 2
Arrays.binarySearch(sorted, 4);                     // -3 (would insert at 2)
Collections.binarySearch(List.of("a", "c"), "c");   // 1
```

NOTES

- → Uppercase sorts before lowercase in natural order
- → `reverseOrder` needs objects, not `int[]`
- → `thenComparing` breaks ties
- → `stream().sorted()` leaves the original list as-is

SORT METHODS

TARGET	METHOD
`int[] / double[]`	Arrays.sort(a)
`Object[]`	Arrays.sort(a, cmp)
`List`	list.sort(cmp)
`List (natural)`	Collections.sort(list)
`New sorted list`	stream().sorted().toList()

SEARCH METHODS

METHOD	REQUIRES	COST
`list.indexOf(x)`	Nothing	O(n)
`list.contains(x)`	Nothing	O(n)
`Arrays.binarySearch`	Sorted array	O(log n)
`Collections.binarySearch`	Sorted list	O(log n)
`set.contains(x)`	HashSet	O(1)

⚠ COMMON MISTAKES

- Calling `binarySearch` on an unsorted list
- Trying `Arrays.sort(intArray, Collections.reverseOrder())` — doesn't compile for primitives
- Sorting an immutable `List.of` in place → `UnsupportedOperationException`

✓ TIPS & BEST PRACTICES

- Sort descending primitives by sorting, then reversing, or use `boxed()` streams
- Build comparators with `comparing` and `thenComparing` rather than subtraction
- Use a `TreeMap` / `TreeSet` if you need data sorted continuously

QUICK SUMMARY

- `Arrays.sort` / `list.sort`
- Natural order or `Comparator`
- Object sorts are stable
- `binarySearch` — sorted data only

Iterators & Iterables

WALKING A COLLECTION

An `Iterator` steps through a collection one element at a time with `hasNext()` and `next()`, and can safely **remove** the last element returned. Any class that implements `Iterable` (one method: `iterator()`) works with the enhanced `for` loop. Collection iterators are **fail-fast**: changing the collection behind their back throws `ConcurrentModificationException`.

NOTES

- `next()` past the end throws `NoSuchElementException`
- `remove()` may be called once per `next()`
- `ListIterator` moves both ways and supports `set` and `add`
- Implement `Iterable<T>` to make your own types for-each friendly

USING & IMPLEMENTING

```
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String n = it.next();
    if (n.isBlank()) it.remove();          // safe removal
}

ListIterator<String> li = names.listIterator();
while (li.hasNext()) {
    li.set(li.next().toUpperCase());       // replace in place
}

class Range implements Iterable<Integer> {   // custom iterable
    private final int start, end;
    Range(int s, int e) { start = s; end = e; }
    public Iterator<Integer> iterator() {
        return new Iterator<>() {
            int cur = start;
            public boolean hasNext() { return cur < end; }
            public Integer next() {
                if (!hasNext()) throw new NoSuchElementException();
                return cur++;
            }
        };
    }
}
for (int i : new Range(1, 4)) System.out.print(i);   // 123
```

NOTES

- → `it.remove()` is the only safe removal during iteration
- → `set` replaces the element last returned by `next`
- → An anonymous class implements the iterator inline
- → `Range` now works in any for-each loop

ITERATOR METHODS

METHOD	PURPOSE
`hasNext()`	More elements?
`next()`	Return next element
`remove()`	Remove last returned

LISTITERATOR EXTRAS

METHOD	PURPOSE
`hasPrevious() / previous()`	Move backward
`nextIndex() / previousIndex()`	Current position
`set(e)`	Replace last returned

⚠ COMMON MISTAKES

- Calling `next()` twice per loop pass and skipping elements
- Calling `list.remove(x)` instead of `it.remove()` during iteration
- Calling `remove()` twice without an intervening `next()` → `IllegalStateException`

✓ TIPS & BEST PRACTICES

- Prefer `removeIf` for simple conditional removal
- Implement `Iterable` instead of exposing internal lists
- Use `Spliterator` / streams if you need parallel traversal

QUICK SUMMARY

- `hasNext()` / `next()` — step through
- `it.remove()` — safe delete
- `ListIterator` — two-way + `set`
- `Iterable` — enables for-each

Classes & Objects

BLUEPRINTS AND INSTANCES

A **class** is a blueprint that defines **fields** (state) and **methods** (behavior). An **object** is one instance of that class, created with `new`. Variables of a class type hold a **reference** to an object, not the object itself, so two variables can point to the same object, and a variable can be `null`.

NOTES

- Every class implicitly extends `Object`
- Objects live on the heap; the GC frees unreachable ones
- Override `toString()` for readable printing
- Use a `record` (Sheet 72) for simple data carriers

A COMPLETE SMALL CLASS

```
public class Dog {
    private String name;                // fields (state)
    private int age;

    public Dog(String name, int age) {   // constructor
        this.name = name;
        this.age = age;
    }

    public void bark() {                // method (behavior)
        System.out.println(name + " says woof");
    }
    public int getAge() { return age; }

    @Override
    public String toString() { return "Dog[" + name + ", " + age + "]"; }
}

Dog rex = new Dog("Rex", 3);   // create an object
rex.bark();                    // Rex says woof
Dog alias = rex;               // same object, two references
System.out.println(alias);     // Dog[Rex, 3]
Dog none = null;
none.bark();                   // ✖ NullPointerException
```

NOTES

- → `new` allocates the object and runs the constructor
- → `rex.bark()` — dot operator calls a method
- → `alias` and `rex` point to one Dog
- → `println(obj)` calls `obj.toString()`

CLASS MEMBERS

MEMBER	PURPOSE	EXAMPLE
`Field`	Holds state	private int age;
`Constructor`	Initializes object	Dog(String n, int a)
`Method`	Behavior	void bark()
`Static member`	Shared by class	static int count;
`Nested class`	Helper type	static class Builder

METHODS INHERITED FROM OBJECT

METHOD	DEFAULT BEHAVIOR
`toString()`	ClassName@hash
`equals(o)`	Same reference
`hashCode()`	Identity-based
`getClass()`	Runtime class

⚠ COMMON MISTAKES

- Calling methods on a `null` reference → `NullPointerException`
- Making fields `public` instead of using methods to control access
- Forgetting `@Override` on `toString`, so a typo like `tostring()` silently doesn't override

✓ TIPS & BEST PRACTICES

- One public class per file; name the file after it
- Keep classes focused on one responsibility
- Override `toString` for every class you'll debug or log

QUICK SUMMARY

- Class — blueprint
- Object — `new` instance
- Variables hold references
- Fields + constructors + methods

Constructors

INITIALIZING OBJECTS

A **constructor** runs when you create an object with new. It has the **same name as the class** and **no return type**. If you write no constructor, Java supplies a no-argument **default constructor**; as soon as you write any constructor, that default disappears. Constructors can be overloaded and can call each other with `this(...)`.

NOTES

- `this(...)` or `super(...)` must be the **first** statement
- If omitted, `super()` is inserted automatically
- `private` constructors prevent outside instantiation

CONSTRUCTOR FORMS

```
public class Account {
    private final String id;
    private double balance;
    private final String currency;

    public Account(String id, double balance, String currency) {  // full
        if (balance < 0) throw new IllegalArgumentException("negative");
        this.id = id;
        this.balance = balance;
        this.currency = currency;
    }
    public Account(String id, double balance) {   // chain with defaults
        this(id, balance, "USD");
    }
    public Account(String id) {
        this(id, 0);
    }
}

new Account("A1");                  // balance 0, USD
new Account("A2", 50);              // USD
new Account("A3", 10, "EUR");

class Savings extends Account {
    Savings(String id) { super(id, 100); }   // call parent constructor
}
```

NOTES

- → Put validation in the most complete constructor
- → Shorter constructors delegate with `this(...)`
- → `final` fields must be assigned exactly once in every constructor
- → `super(...)` picks which parent constructor runs

INITIALIZATION ORDER

STEP	WHAT RUNS
1	Static fields & static blocks (once per class)
2	Parent constructor (super)
3	Instance field initializers & instance blocks
4	Constructor body

CONSTRUCTOR TYPES

TYPE	EXAMPLE
Default (generated)	Only when none are declared
No-arg	public Dog() { }
Parameterized	public Dog(String name)
Copy	public Dog(Dog other)

⚠ COMMON MISTAKES

- Adding a return type (`public void Dog()`) — that's a method, not a constructor
- Adding a parameterized constructor and breaking code that relied on the default one
- Calling overridable methods from a constructor; subclasses may see half-built state

✓ TIPS & BEST PRACTICES

- Validate arguments in constructors so objects are always valid
- Use static factories or a builder when there are many optional params
- Chain constructors with `this(...)` to avoid duplicate code

QUICK SUMMARY

- Same name as class, no return type
- `this(...)` — call another constructor
- `super(...)` — call parent's
- Default constructor vanishes once you declare one

The this Keyword

REFERRING TO THE CURRENT OBJECT

Inside an instance method or constructor, `this` is a reference to **the object the method was called on**. It's used to reach fields hidden by same-named parameters, to call another constructor with `this(...)`, to return the current object for **method chaining**, and to pass the current object to other methods.

NOTES

- `this` doesn't exist in `static` methods
- `this(...)` must be the first line of a constructor
- `this` can't be reassigned
- Inner classes reach the outer object with `Outer.this`

FOUR USES OF THIS

```
public class Point {
    private int x, y;

    public Point(int x, int y) {
        this.x = x;                  // 1. field vs parameter
        this.y = y;
    }
    public Point() {
        this(0, 0);                  // 2. call another constructor
    }

    public Point moveX(int dx) { x += dx; return this; }   // 3. chaining
    public Point moveY(int dy) { y += dy; return this; }

    public void register(Registry r) {
        r.add(this);                 // 4. pass self
    }
}

Point p = new Point().moveX(3).moveY(4);   // fluent: (3, 4)

StringBuilder sb = new StringBuilder()
    .append("a").append("b");               // JDK uses the same pattern
```

NOTES

- → Without `this.`, `x = x` assigns the parameter to itself
- → `return this` enables fluent chaining
- → Builders and `StringBuilder` rely on chaining

USES AT A GLANCE

USE	SYNTAX
Access field	this.name
Call method	this.helper()
Call constructor	this(args)
Return self	return this;
Outer instance	Outer.this.field

THIS VS SUPER

KEYWORD	REFERS TO
this	Current object (this class's members)
super	Parent-class members of current object
this()	Constructor in same class
super()	Constructor in parent class

⚠ COMMON MISTAKES

- `name = name;` in a setter does nothing; write `this.name = name;`
- Using `this` inside a `static` method — compile error
- Leaking `this` from a constructor (e.g., registering a listener) before the object is fully built

✓ TIPS & BEST PRACTICES

- Use `this.` consistently in constructors and setters
- Return `this` for builder-style APIs
- Inside lambdas, `this` refers to the enclosing object (unlike anonymous classes)

QUICK SUMMARY

- `this` — current object
- `this.x` — field when shadowed
- `this(...)` — chain constructors
- `return this` — fluent chaining

Instance vs Static Members

ONE PER OBJECT VS ONE PER CLASS

Instance fields exist once per object, so each object has its own copy. **Static** fields exist once per class and are shared by every instance. Static **initializer blocks** run once when the class is first loaded, which is useful for setting up complex static data.

NOTES

- Access static members through the class name: `Counter.total`
- Static fields are initialized before any instance is created
- `static final` primitives and strings are compile-time constants
- Mutable static state is shared across threads

COUNTING INSTANCES

```
public class Ticket {
    private static int issued = 0;              // shared by all tickets
    private static final Map<String, Double> PRICES;

    static {                                    // runs once at class load
        PRICES = Map.of("adult", 12.0, "child", 6.0);
    }

    private final int number;                   // per object
    private final String type;

    public Ticket(String type) {
        this.type = type;
        this.number = ++issued;                 // update shared counter
    }
    public double price() { return PRICES.get(type); }
    public static int issuedCount() { return issued; }
}

Ticket a = new Ticket("adult");   // number 1
Ticket b = new Ticket("child");   // number 2
Ticket.issuedCount();             // 2
b.price();                        // 6.0
```

NOTES

- → `issued` is one counter for the whole class
- → `number` differs per ticket
- → `static {}` handles setup too complex for one line
- → Instance methods can read static fields; not vice versa

COMPARISON

ASPECT	INSTANCE MEMBER	STATIC MEMBER
Copies	One per object	One per class
Access	obj.field	ClassName.field
Created	With new	At class load
Can use this	Yes	No
Typical use	Object state	Counters, constants, caches

INITIALIZATION BLOCKS

BLOCK	RUNS
`static { }`	Once, when class loads
`{ } (instance)`	Before each constructor body
`Field initializer`	In declaration order

⚠ COMMON MISTAKES

- Making a field `static` by accident — every object then shares one value
- Incrementing a static counter from multiple threads without synchronization
- Accessing static members through instances: `a.issuedCount()` compiles but misleads

✓ TIPS & BEST PRACTICES

- Use `static final` for constants, rarely mutable statics
- Use `AtomicInteger` for shared counters in concurrent code
- Prefer dependency injection over static singletons for testability

QUICK SUMMARY

- Instance — per object
- Static — per class, shared
- `static { }` — one-time setup
- Access statics via class name

Inheritance

REUSING AND EXTENDING CLASSES

Inheritance lets a class (subclass) acquire the fields and methods of another (superclass) with `extends`. The subclass can add members and **override** behavior. Java allows **single** class inheritance only, and every class ultimately extends `Object`. Use inheritance for genuine **is-a** relationships.

NOTES

- Private members are inherited but not directly accessible
- `protected` members are visible to subclasses
- `final class` cannot be extended (e.g., `String`)
- Prefer composition ("has-a") when "is-a" is doubtful

CLASS HIERARCHY

```
public class Vehicle {
    protected final String make;
    private int speed;

    public Vehicle(String make) { this.make = make; }
    public void accelerate(int amt) { speed += amt; }
    public int getSpeed() { return speed; }
    public String describe() { return make + " vehicle"; }
}public class Car extends Vehicle {               // Car is-a Vehicle
    private final int doors;

    public Car(String make, int doors) {
        super(make);                              // parent constructor
        this.doors = doors;
    }
    @Override
    public String describe() { return make + " car, " + doors + " doors"; }
}

Car c = new Car("Volvo", 4);
c.accelerate(30);                 // inherited
c.getSpeed();                     // 30
c.describe();                     // "Volvo car, 4 doors"
Vehicle v = c;                    // a Car is a Vehicle
```

NOTES

- → `make` is `protected`, so `Car` can read it
- → `speed` is private: use `getSpeed()`
- → `super(make)` must run first
- → A `Car` can be stored in a `Vehicle` variable

WHAT SUBCLASSES GET

MEMBER	INHERITED?	ACCESSIBLE?
`public`	Yes	Yes
`protected`	Yes	Yes
`package-private`	Yes	Same package only
`private`	Yes (exists)	No

INHERITANCE VS COMPOSITION

USE	WHEN
`extends`	True is-a, shared behavior
`Composition`	has-a, reuse without coupling
`Interface`	Shared capability across unrelated classes

⚠ COMMON MISTAKES

- Using inheritance just to reuse code (e.g., `Stack extends Vector`)
- Forgetting `super(...)` when the parent has no no-arg constructor — compile error

✓ TIPS & BEST PRACTICES

- Favor composition over inheritance by default
- Mark classes `final` unless designed for extension
- Keep shared state `private` with `protected` accessors if needed

QUICK SUMMARY

- `extends` — single inheritance
- Subclass inherits members
- `super(...)` — parent constructor
- Use for real is-a relationships

The super Keyword

REACHING THE PARENT

`super` refers to the **parent-class part** of the current object. `super(...)` calls a parent constructor and must be the first statement in a constructor. `super.method()` calls the parent's version of an overridden method, letting a subclass **extend** behavior instead of replacing it entirely.

NOTES

- If you don't call `super(...)`, Java inserts `super()`
- `super.super.method()` is not allowed
- `super` can't be used in static contexts
- Hidden fields can be reached with `super.field`

EXTENDING BEHAVIOR

```
class Employee {
    protected final String name;
    protected double baseSalary;

    Employee(String n, double b) { name = n; baseSalary = b; }
    double pay() { return baseSalary; }
}

class Manager extends Employee {
    private final double bonus;

    Manager(String name, double base, double bonus) {
        super(name, base);                          // 1. parent constructor
        this.bonus = bonus;
    }
    @Override
    double pay() { return super.pay() + bonus; }   // 2. reuse parent logic

}

class Director extends Manager {
    Director(String n) { super(n, 150_000, 30_000); }   // multi-level
    @Override
    double pay() { return super.pay() * 1.1; }          // Manager.pay()
}
new Director("Ada").pay();   // (150000 + 30000) × 1.1 = 198000.0
```

NOTES

- → `super.pay()` calls the **immediate** parent's version
- → `Director` → `Manager` → `Employee`: each layer adds to the last
- → Constructors run top-down: Employee, Manager, Director

SUPER USES

SYNTAX	CALLS
`super()`	Parent no-arg constructor
`super(args)`	Matching parent constructor
`super.method()`	Parent's method version

CONSTRUCTOR CHAIN RULES

RULE	RESULT
`No explicit call`	super() inserted
`Parent lacks no-arg`	Must call super(args)
`this(...) first`	super happens in the target

⚠ COMMON MISTAKES

- Calling `super(...)` after other statements — compile error
- Forgetting `super.method()` when you meant to extend, not replace, behavior
- Declaring a field with the same name as the parent's (field hiding) and confusing which one is used

✓ TIPS & BEST PRACTICES

- Call `super.method()` first or last consistently, and document which
- Avoid field hiding; give subclass fields distinct names
- Keep constructors simple so chains are easy to follow

QUICK SUMMARY

- `super(...)` — parent constructor, first line
- `super.m()` — parent method
- Chains run parent → child
- No `super.super`

Method Overriding

REPLACING INHERITED BEHAVIOR

A subclass **overrides** a method by declaring one with the **same name and parameters** as the parent. At runtime Java calls the version that matches the **actual object type**, not the variable type (dynamic dispatch). Always annotate overrides with `@Override` so the compiler catches signature mistakes.

NOTES

- Can't reduce visibility (`public` → `protected` ✖)
- Can return a **subtype** (covariant return)
- Can't throw broader **checked** exceptions
- `final` , `static` , and `private` methods can't be overridden

OVERRIDING IN PRACTICE

```
class Shape {
    double area() { return 0; }
    Shape copy() { return new Shape(); }
}
class Circle extends Shape {
    final double r;
    Circle(double r) { this.r = r; }

    @Override double area() { return Math.PI * r * r; }
    @Override Circle copy() { return new Circle(r); }   // covariant

    @Override
    public boolean equals(Object o) {
        return o instanceof Circle c && c.r == r;
    }
    @Override
    public int hashCode() { return Double.hashCode(r); }
}

Shape s = new Circle(1);
s.area();                 // 3.14159…  Circle's version runs
// @Override double area(int x)    ✖ compile error: doesn't override
```

NOTES

→ The variable type is `Shape` , but `Circle.area()` runs

→ `@Override` turns a silent overload into a compile error

→ Override `equals` and `hashCode` **together**

OVERRIDE RULES

ASPECT	RULE
`Name + params`	Must match exactly
`Return type`	Same or subtype
`Access`	Same or wider
`Checked exceptions`	Same, narrower, or none
`static methods`	Hidden, not overridden

OVERLOADING VS OVERRIDING

ASPECT	OVERLOADING	OVERRIDING
`Where`	Same class	Subclass
`Params`	Different	Same
`Chosen`	Compile time	Runtime
`@Override`	No	Yes

⚠ COMMON MISTAKES

- `public boolean equals(Circle c)` — overloads instead of overriding `equals(Object)`
- Overriding `equals` without `hashCode` ; `HashSet` and `HashMap` break
- Omitting `@Override` and not noticing a typo in the method name

✓ TIPS & BEST PRACTICES

- Always write `@Override`
- Use `final` on methods subclasses must not change
- Let your IDE or a `record` generate `equals` / `hashCode`

QUICK SUMMARY

- Same signature in subclass
- Runtime type decides
- `@Override` — compiler check
- Covariant returns allowed

Polymorphism

ONE INTERFACE, MANY FORMS

Polymorphism means code written against a parent type or interface works with any subtype. A `List<Shape>` can hold circles and squares, and calling `area()` runs each object's own version. **Upcasting** (child → parent) is automatic; **downcasting** (parent → child) needs a check with `instanceof` or risks `ClassCastException`.

NOTES

- Variable type controls which methods you **can** call
- Object type controls which implementation **runs**
- Java 16+ pattern matching: `if (s instanceof Circle c)`
- Prefer polymorphic methods over chains of `instanceof`

POLYMORPHISM AT WORK

```
interface Shape { double area(); }
record Circle(double r) implements Shape {
    public double area() { return Math.PI * r * r; }
}
record Square(double side) implements Shape {
    public double area() { return side * side; }
}

List<Shape> shapes = List.of(new Circle(1), new Square(2));
double total = 0;
for (Shape s : shapes) total += s.area();    // each runs its own area()

Shape s = new Circle(2);          // upcast (automatic)
// s.r();                         ✖ Shape has no r()
if (s instanceof Circle c) {      // safe downcast + bind
    System.out.println(c.r());    // 2.0
}
Square sq = (Square) s;           // ✖ ClassCastException at runtime

String info = switch (s) {        // Java 21 pattern switch
    case Circle c -> "circle r=" + c.r();
    case Square q -> "square " + q.side();
    default -> "unknown";
};
```

NOTES

→ The loop never checks types; dispatch picks the method

→ `instanceof Circle c` tests and casts in one step

→ An explicit cast compiles but fails if the type is wrong

CASTING

DIRECTION	SYNTAX	SAFE?
Upcast	Shape s = circle;	Always
Downcast	(Circle) s	Only if s is a Circle
Checked downcast	s instanceof Circle c	Yes
Unrelated types	(String) circle	Compile error

KINDS OF POLYMORPHISM

KIND	MECHANISM
Subtype (runtime)	Overriding + dynamic dispatch
Ad hoc (compile time)	Overloading
Parametric	Generics <T>

⚠ COMMON MISTAKES

- Downcasting without checking → `ClassCastException`
- Long `if (x instanceof A) … else if (x instanceof B)` chains instead of an overridden method
- Calling subclass-only methods through a parent-typed variable — compile error

✓ TIPS & BEST PRACTICES

- Program to interfaces: `List<Shape>`, not `ArrayList<Circle>`
- Use sealed types + pattern `switch` for exhaustive type handling
- Put type-specific behavior in the types themselves

QUICK SUMMARY

- Parent type, child behavior
- Upcast: automatic
- Downcast: `instanceof` first
- Dispatch by runtime type

Encapsulation

HIDING INTERNAL STATE

Encapsulation keeps an object's data `private` and exposes only the operations that make sense. Outside code can't put the object into an invalid state because every change goes through methods that enforce the rules (**invariants**). Java's four access levels control who can see each class, field, and method.

NOTES

- Default (no modifier) is **package-private**
- Top-level classes can only be `public` or package-private
- `private` is per-class, not per-object: instances of the same class can read each other's private fields
- Expose the narrowest access that works

ACCESS MODIFIERS

MODIFIER	CLASS	PACKAGE	SUBCLASS	EVERYWHERE
`public`	✔	✔	✔	✔
`protected`	✔	✔	✔	✖
`(none) package-private`	✔	✔	✖	✖
`private`	✔	✖	✖	✖

PROTECTING AN INVARIANT

```
public class BankAccount {
    private double balance;                        // hidden
    private final List<String> history = new ArrayList<>();

    public void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("amount > 0");
        balance += amt;
        history.add("+" + amt);
    }
    public void withdraw(double amt) {
        if (amt > balance) throw new IllegalStateException("insufficient");
        balance -= amt;
        history.add("-" + amt);
    }
    public double getBalance() { return balance; }
    public List<String> getHistory() {
        return List.copyOf(history);               // no outside edits
    }
}

acct.balance = -500;            // ✖ compile error: private
acct.getHistory().clear();      // ✖ UnsupportedOperationException
```

NOTES

- → Rule: balance never goes negative
- → Only `deposit` / `withdraw` change state
- → Returning a copy stops callers from editing internal lists

⚠ COMMON MISTAKES

- Public fields that anyone can set to invalid values
- Returning internal mutable collections or arrays directly
- Adding a getter **and** setter for every field by reflex, which exposes everything anyway

✓ TIPS & BEST PRACTICES

- Start every field as `private final`; relax only when needed
- Return `List.copyOf` or `Collections.unmodifiableList`
- Validate in constructors and mutators

QUICK SUMMARY

- `private` fields, public methods
- Four access levels
- Methods enforce invariants
- Return copies, not internals

Getters & Setters

CONTROLLED ACCESS TO FIELDS

Getters (accessors) return a field's value, and **setters** (mutators) change it. Routing access through methods lets you validate input, compute values, and change the internal representation later without breaking callers. JavaBeans conventions use `getX()`, `isX()` for booleans, and `setX(value)`.

NOTES

- Frameworks (Jackson, JPA, JavaFX) rely on these naming conventions
- Omit setters to make a field read-only
- Records generate accessors named `x()`, not `getX()`
- Lombok can generate them (Sheet 110)

CONVENTIONS & VALIDATION

```java
public class Person {
    private String name;
    private int age;
    private boolean active;

    public String getName() { return name; }
    public void setName(String name) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("name required");
        this.name = name.strip();
    }

    public int getAge() { return age; }
    public void setAge(int age) {
        if (age < 0 || age > 150) throw new IllegalArgumentException("age");
        this.age = age;
    }

    public boolean isActive() { return active; }    // boolean → is
    public void setActive(boolean active) { this.active = active; }

    public boolean isAdult() { return age >= 18; }  // computed, no field
}

record Point(int x, int y) { }      // immutable alternative
new Point(1, 2).x();                // 1
```

NOTES

- → Setters normalize input (`strip()`) as well as validate
- → Computed getters need no backing field
- → Records give read-only accessors for free

NAMING RULES

FIELD	GETTER	SETTER
`name`	getName()	setName(String)
`age`	getAge()	setAge(int)
`active (boolean)`	isActive()	setActive(boolean)
`URL`	getURL()	setURL(String)

MUTABLE VS IMMUTABLE DESIGN

APPROACH	SETTERS?	THREAD-SAFE?
`JavaBean`	Yes	No (needs sync)
`final fields + constructor`	No	Yes
`record`	No	Yes
`Builder → immutable`	No (on result)	Yes

⚠ COMMON MISTAKES

- Setters that don't validate, making encapsulation pointless
- Getter returning a mutable `List` or `Date` that callers can change
- `getIsActive()` naming, which breaks JavaBeans tools — use `isActive()`

✓ TIPS & BEST PRACTICES

- Prefer immutable objects; add setters only when state must change
- Return defensive copies of mutable fields
- Use `with`-style methods (`withAge(31)`) that return new objects

QUICK SUMMARY

- `getX()` / `isX()` — read
- `setX(v)` — validated write
- Omit setters → read-only
- Records: `x()` accessors

Abstract Classes

PARTIALLY IMPLEMENTED BASE CLASSES

An **abstract class** can't be instantiated; it exists to be extended. It may contain **abstract methods** (declared without a body) that every concrete subclass must implement, alongside normal fields, constructors, and implemented methods. Use one when related classes share state and some behavior but differ in specific steps.

NOTES

- Any class with an abstract method must be declared `abstract`
- Abstract classes can have constructors (called via `super`)
- A subclass that doesn't implement all abstract methods must also be abstract
- `abstract` and `final` can't be combined

TEMPLATE METHOD PATTERN

```
public abstract class Report {
    private final String title;                  // shared state
    protected Report(String title) { this.title = title; }

    public final String render() {               // fixed algorithm
        return header() + "\n" + body() + "\n" + footer();
    }
    private String header() { return "== " + title + " =="; }
    protected String footer() { return "-- end --"; }    // optional hook
    protected abstract String body();             // required step
}

public class SalesReport extends Report {
    private final double total;
    public SalesReport(double total) { super("Sales"); this.total = total; }
    @Override
    protected String body() { return "Total: $" + total; }
}

Report r = new SalesReport(1200);
System.out.println(r.render());
// == Sales ==
// Total: $1200.0
// -- end --
```

NOTES

- → `render()` is `final`, so the overall order is locked
- → `body()` must be supplied; `footer()` may be overridden
- → Variables can still have the abstract type: `Report r`

ABSTRACT CLASS CAN HAVE

ELEMENT	ALLOWED
Instance fields	Yes
Constructors	Yes
Concrete methods	Yes
Abstract methods	Yes

ABSTRACT CLASS VS INTERFACE

ASPECT	ABSTRACT CLASS	INTERFACE
Inheritance	Single	Multiple
State (fields)	Yes	Constants only
Constructors	Yes	No
Best for	Shared base with state	Capability / contract

⚠ COMMON MISTAKES

- Making a class abstract when an interface would do — it uses up the single inheritance slot
- Forgetting to implement an abstract method — the compiler forces the subclass to be abstract
- Calling abstract methods from the constructor before subclass fields are set

✓ TIPS & BEST PRACTICES

- Pair an interface with an abstract "skeleton" class (like `AbstractList`)
- Mark the template method `final`
- Use `protected` for hooks meant only for subclasses

QUICK SUMMARY

- `abstract class` — no `new`
- `abstract` methods — no body
- Mix state + implemented methods
- Template method pattern

Interfaces

CONTRACTS BETWEEN TYPES

An **interface** declares what a type can **do** without dictating how. Classes `implement` interfaces and must provide all abstract methods. A class can implement **many** interfaces, which is Java's way of combining capabilities. Since Java 8, interfaces can include `default` and `static` methods, and since Java 9, `private` helpers.

NOTES

- Methods are `public abstract` unless marked otherwise
- Fields are implicitly `public static final`
- A one-method interface is a functional interface (lambda-ready)

DEFINING & IMPLEMENTING

```
public interface Shape {
    double PI_APPROX = 3.14;                    // constant
    double area();                              // abstract
    double perimeter();

    default String describe() {                 // default method
        return "%s with area %.2f".formatted(name(), area());
    }
    private String name() { return getClass().getSimpleName(); }
    static Shape unitSquare() { return new Rect(1, 1); }   // factory
}

public interface Scalable { Shape scale(double f); }

public record Rect(double w, double h) implements Shape, Scalable {
    public double area() { return w * h; }
    public double perimeter() { return 2 * (w + h); }
    public Rect scale(double f) { return new Rect(w * f, h * f); }
}

Shape s = new Rect(2, 3);
s.describe();                 // "Rect with area 6.00"
Shape.unitSquare().area();    // 1.0
```

NOTES

- → `default` methods give implementers free behavior
- → `private` interface methods share code between defaults
- → `static` methods are called on the interface name
- → One class, two interfaces: `Shape, Scalable`

INTERFACE MEMBERS

MEMBER	SINCE	MODIFIER
`Abstract method`	1.0	public abstract (implicit)
`Constant`	1.0	public static final (implicit)
`default method`	8	default
`static method`	8	static
`private method`	9	private

JDK INTERFACES YOU'LL IMPLEMENT

INTERFACE	METHOD
`Comparable<T>`	compareTo
`Runnable`	run
`AutoCloseable`	close
`Iterable<T>`	iterator

⚠ COMMON MISTAKES

- Using interfaces just to hold constants (the "constant interface" anti-pattern)
- Forgetting `public` when implementing — interface methods are public, so narrower access fails
- Two interfaces with the same default method → you must override and choose

✓ TIPS & BEST PRACTICES

- Name interfaces by capability: `Readable`, `Payable`
- Declare variables with interface types
- Add `default` methods to evolve an interface without breaking implementers

QUICK SUMMARY

- `interface` — contract
- `implements` — fulfill it
- Many interfaces per class
- `default` `static` `private` methods

implements vs extends

COMBINING INHERITANCE AND INTERFACES

`extends` creates a subclass of **one** class, inheriting its state and behavior. `implements` promises to fulfill **one or more** interface contracts. A class can do both: `class A extends B implements C, D`. Interfaces themselves use `extends`, and can extend several interfaces at once.

NOTES

- Interfaces extend interfaces with `extends`, never `implements`
- Conflicting `default` methods must be overridden
- `Interface.super.method()` picks a specific default

MIXING BOTH

```
abstract class Animal {
    protected final String name;
    Animal(String name) { this.name = name; }
    abstract String sound();
}
interface Swimmer { default String move() { return "swims"; } }
interface Walker  { default String move() { return "walks"; } }
interface Pet extends Comparable<Pet> { String owner(); }  // iface extends

class Duck extends Animal implements Swimmer, Walker, Pet {
    private final String owner;
    Duck(String n, String o) { super(n); owner = o; }

    String sound() { return "quack"; }
    public String owner() { return owner; }
    public int compareTo(Pet p) { return owner.compareTo(p.owner()); }

    @Override
    public String move() {                    // resolve the conflict
        return Swimmer.super.move() + " and " + Walker.super.move();
    }
}
new Duck("Don", "Ada").move();   // "swims and walks"
```

NOTES

- → One superclass (`Animal`), three interfaces
- → `Pet` inherits `compareTo` from `Comparable`
- → `X.super.move()` calls a specific interface's default

KEYWORD COMBINATIONS

DECLARATION	KEYWORD	HOW MANY
`class → class`	extends	Exactly one
`class → interface`	implements	Any number
`interface → interface`	extends	Any number
`record / enum → interface`	implements	Any number
`record / enum → class`	—	Not allowed

CHOOSING

NEED	USE
`Share fields + code`	extends abstract class
`Add a capability`	implements interface
`Multiple capabilities`	Several interfaces

⚠ COMMON MISTAKES

- `class A implements B extends C` — wrong order, compile error
- `interface X implements Y` — interfaces use `extends`
- Ignoring the diamond conflict error instead of overriding the method

✓ TIPS & BEST PRACTICES

- Keep inheritance shallow; layer behavior with interfaces
- Records and enums can implement interfaces, which is often enough
- Use composition plus an interface for flexible delegation

QUICK SUMMARY

- `extends` — one class
- `implements` — many interfaces
- Interfaces `extends` interfaces
- `X.super.m()` — pick a default

Enums

FIXED SETS OF CONSTANTS

An `enum` defines a type with a fixed set of named instances, like days of the week or order states. Enums are full classes: they can have fields, constructors, methods, and can implement interfaces. They're type-safe, work in `switch`, and compare safely with ==.

NOTES

- Enum constructors are always private
- Each constant is a singleton
- `valueOf("X")` throws `IllegalArgumentException` if no match
- Prefer `EnumMap` / `EnumSet` for enum-keyed data

ENUM FEATURES

```
public enum Planet {
    MERCURY(3.303e23, 2.4397e6),
    EARTH(5.976e24, 6.37814e6);           // constants call the constructor

    private final double mass, radius;
    Planet(double m, double r) { mass = m; radius = r; }
    public double gravity() { return 6.67e-11 * mass / (radius * radius); }
}
Planet.EARTH.gravity();          // ≈ 9.8

enum Status { NEW, PAID, SHIPPED }
Status s = Status.valueOf("PAID");   // PAID
s.name();                            // "PAID"
s.ordinal();                         // 1
Status.values();                     // [NEW, PAID, SHIPPED]
s == Status.PAID;                    // true

String label = switch (s) {          // exhaustive, no default needed
    case NEW -> "Awaiting payment";
    case PAID -> "Preparing";
    case SHIPPED -> "On the way";
};
EnumSet<Status> open = EnumSet.of(Status.NEW, Status.PAID);
EnumMap<Status, Integer> counts = new EnumMap<>(Status.class);
```

NOTES

- → Constants must come first, ending with `;` if more members follow
- → `ordinal()` is the declaration position — don't persist it
- → Switch on enums needs no `default` when every constant is covered

BUILT-IN METHODS

METHOD	RETURNS
`name()`	Constant name as String
`ordinal()`	Position (0-based)
`values()`	Array of all constants
`valueOf(String)`	Constant by name
`compareTo(e)`	Order by ordinal

ENUM COLLECTIONS

TYPE	USE
`EnumSet.of(A, B)`	Fast set of constants
`EnumSet.allOf(E.class)`	All constants
`EnumSet.range(A, C)`	Contiguous range
`EnumMap<E, V>`	Map keyed by enum

⚠ COMMON MISTAKES

- Storing `ordinal()` in a database; adding a constant later shifts every value
- Using `int` or `String` constants instead of an enum, losing type safety
- Calling `valueOf` on user input without handling the exception

✓ TIPS & BEST PRACTICES

- Store `name()` (or a dedicated code field) when persisting
- Put behavior in the enum instead of switching on it everywhere
- Use enums for singletons: `enum Config { INSTANCE; }`

QUICK SUMMARY

- `enum` — fixed set of instances
- Fields, constructors, methods
- `values()` `valueOf` `name`
- `EnumSet` / `EnumMap`

Records (Java 16+)

CONCISE IMMUTABLE DATA CLASSES

A `record` declares a class whose purpose is to carry data. From one line, `record Point(int x, int y)`, Java generates private final fields, a constructor, accessors `x()` and `y()`, and value-based `equals`, `hashCode`, and `toString`. Records are **immutable** and implicitly `final`.

NOTES

- Records can't extend classes, but can implement interfaces
- No extra instance fields allowed beyond the components
- Static fields and methods are allowed
- Shallowly immutable: a `List` component can still be mutated

RECORD FEATURES

```
public record Point(int x, int y) { }

Point p = new Point(3, 4);
p.x();                              // 3   (accessor, no "get")
p.toString();                       // "Point[x=3, y=4]"
p.equals(new Point(3, 4));          // true (value equality)

public record Range(int lo, int hi) {
    public Range {                                  // compact constructor
        if (lo > hi) throw new IllegalArgumentException(lo + " > " + hi);
    }
    public int length() { return hi - lo; }     // extra method
    public static Range of(int n) { return new Range(0, n); }
}

public record Team(String name, List<String> members) {
    public Team { members = List.copyOf(members); }   // defensive copy
}

if (obj instanceof Point(int x, int y)) {    // record pattern (Java 21)
    System.out.println(x + y);
}
```

NOTES

- → Compact constructors validate or normalize without repeating assignments
- → `List.copyOf` makes the component truly immutable
- → Records are ideal map keys and DTOs

WHAT JAVA GENERATES

GENERATED	FOR RECORD POINT(INT X, INT Y)
Fields	private final int x, y
Constructor	Point(int x, int y)
Accessors	x(), y()
equals / hashCode	Based on all components
toString	Point[x=3, y=4]

RECORD VS CLASS VS LOMBOK

NEED	CHOOSE
Immutable data carrier	record
Mutable state	class
Inheritance from a class	class
JPA entity	class (records can't be entities)

⚠ COMMON MISTAKES

- Calling `p.getX()` — record accessors have no `get` prefix
- Assuming list components are immutable without copying them
- Trying to add instance fields or extend another class

✓ TIPS & BEST PRACTICES

- Use records for DTOs, API responses, and multiple return values
- Validate in a compact constructor
- Declare local records inside methods for temporary groupings

QUICK SUMMARY

- `record Name(components)`
- Auto: fields, constructor, accessors
- Value-based `equals` / `hashCode`
- Immutable and final

Sealed Classes (Java 17+)

RESTRICTING WHO CAN EXTEND

A **sealed** class or interface lists exactly which types may extend or implement it with `permits`. Each permitted subtype must be `final`, `sealed`, or `non-sealed`. Because the compiler knows every subtype, a pattern `switch` over a sealed type can be **exhaustive** without a `default`.

NOTES

- Permitted subclasses must be in the same module (or package if unnamed module)
- `permits` can be omitted if subclasses are in the same file
- Records are implicitly `final`, so they pair well with sealed interfaces
- `non-sealed` reopens a branch for anyone to extend

SEALED HIERARCHY

```
public sealed interface Shape permits Circle, Square, Rect { }

public record Circle(double r) implements Shape { }
public record Square(double side) implements Shape { }
public record Rect(double w, double h) implements Shape { }

static double area(Shape s) {
    return switch (s) {                 // exhaustive: no default
        case Circle c -> Math.PI * c.r() * c.r();
        case Square q -> q.side() * q.side();
        case Rect(double w, double h) -> w * h;
    };
}

public sealed abstract class Payment permits Card, Cash, Crypto { }
public final class Card extends Payment { }
public final class Cash extends Payment { }
public non-sealed class Crypto extends Payment { }   // open again
public class Bitcoin extends Crypto { }              // allowed

// class Check extends Payment { }   ✖ not in permits list
```

NOTES

→ Add a new `Shape` and every switch without `default` fails to compile, showing you what to update

→ `Rect(double w, double h)` deconstructs the record

→ `non-sealed` trades exhaustiveness for extensibility

SUBCLASS MODIFIERS

MODIFIER	MEANING
`final`	No further subclasses
`sealed`	Further restricted with its own permits
`non-sealed`	Anyone may extend
`record / enum`	Implicitly final

WHEN TO SEAL

SITUATION	SEAL?
`Closed set of variants (AST nodes, events)`	Yes
`Result type (Success / Failure)`	Yes
`Plugin or extension point`	No
`Public library base class`	Carefully

⚠ COMMON MISTAKES

- Adding `default` to a switch over a sealed type, which hides missing cases
- Forgetting a modifier on a permitted subclass — compile error
- Putting permitted subclasses in another package of an unnamed module

✓ TIPS & BEST PRACTICES

- Model "one of these" data with sealed interface + records
- Let the compiler's exhaustiveness check drive updates
- Pair with record patterns for concise deconstruction

QUICK SUMMARY

- `sealed … permits A, B`
- Subtypes: `final` / `sealed` / `non-sealed`
- Exhaustive pattern switch
- Closed, known hierarchies

Annotations & Reflection

METADATA AND RUNTIME INSPECTION

Annotations attach metadata to code (`@Override`, `@Deprecated`, `@Test`). You can define your own with `@interface`. **Reflection** lets a program inspect classes, fields, methods, and annotations at runtime through `Class<?>` objects. Frameworks like JUnit, Spring, and Jackson combine the two: read annotations via reflection and act on them.

NOTES

- `@Retention(RUNTIME)` is required to read an annotation via reflection
- `@Target` limits where an annotation may appear
- Reflection is slower and bypasses compile-time checks

CUSTOM ANNOTATION + REFLECTION

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Required { String message() default "is required"; }

class SignupForm {
    @Required String email;
    @Required(message = "needs a value") String password;
    String nickname;
}

static List<String> validate(Object obj) throws IllegalAccessException {
    List<String> errors = new ArrayList<>();
    for (Field f : obj.getClass().getDeclaredFields()) {
        Required r = f.getAnnotation(Required.class);
        if (r == null) continue;
        f.setAccessible(true);
        if (f.get(obj) == null) errors.add(f.getName() + " " + r.message());
    }
    return errors;
}
validate(new SignupForm());   // [email is required, password needs a value]

Method m = String.class.getMethod("toUpperCase");
m.invoke("hi");                                  // "HI"
```

NOTES

- → `default` supplies an element value when omitted
- → `getDeclaredFields` includes private fields
- → `invoke` calls a method found at runtime

BUILT-IN ANNOTATIONS

ANNOTATION	PURPOSE
`@Override`	Verify a method overrides
`@Deprecated`	Mark as obsolete
`@SuppressWarnings`	Silence compiler warnings
`@FunctionalInterface`	Enforce one abstract method

REFLECTION API

CALL	RETURNS
`obj.getClass() / X.class`	Class<?>
`getDeclaredFields()`	All fields of this class
`getMethods()`	Public methods incl. inherited
`getConstructor(...).newInstance()`	New object

⚠ COMMON MISTAKES

- Forgetting `@Retention(RUNTIME)` — `getAnnotation` returns `null`
- Using reflection for ordinary logic that an interface would express safely
- Ignoring checked reflection exceptions instead of handling or wrapping them

✓ TIPS & BEST PRACTICES

- Use annotations for declarative config read by tools or frameworks
- Cache `Field` / `Method` lookups; reflection lookups are costly
- Prefer `MethodHandle` or records' accessors for faster dynamic access

QUICK SUMMARY

- `@interface` — define annotation
- `@Retention` `@Target` — meta-annotations
- `Class<?>` — runtime type info
- `getAnnotation` / `invoke`

Inner & Anonymous Classes

CLASSES INSIDE CLASSES

Java lets you declare classes inside other classes. A **static nested** class is an ordinary class scoped to its outer class. An **inner** (non-static) class is tied to an instance of the outer class and can read its fields. **Local** classes live inside a method, and **anonymous** classes are declared and instantiated in a single expression.

NOTES

- Inner classes hold a hidden reference to the outer object
- Prefer `static` nested unless you need the outer instance
- Lambdas replace anonymous classes for functional interfaces

THE FOUR KINDS

```
public class Outer {
    private int count = 5;
    static class Builder { }                    // 1. static nested

    class Counter {                             // 2. inner
        int doubled() { return count * 2; }     //    reads outer field
    }

    void demo() {
        class Local { }                         // 3. local
        Comparator<String> byLen = new Comparator<>() {   // 4. anonymous
            public int compare(String a, String b) {
                return a.length() - b.length(); }
        };
        Comparator<String> lam = (a, b) -> a.length() - b.length();
    }
}

Outer.Builder b = new Outer.Builder();      // no Outer needed
Outer o = new Outer();
Outer.Counter c = o.new Counter();          // needs an Outer
c.doubled();                                // 10
```

NOTES

→ `o.new Counter()` — inner instances need an outer object

→ The lambda equals the anonymous class in one line

COMPARISON

KIND	NEEDS OUTER	TYPICAL USE
`Static nested`	No	Builders, helper types
`Inner`	Yes	Iterators, views over outer data
`Local`	Yes (if in instance method)	One-method helper
`Anonymous`	Yes (if in instance method)	One-off implementations

ANONYMOUS CLASS VS LAMBDA

ASPECT	ANONYMOUS CLASS	LAMBDA
`this means`	The anonymous object	Enclosing object
`Can have state`	Yes (fields)	No

⚠ COMMON MISTAKES

- Inner classes that keep large outer objects alive (memory leaks)
- `new Counter()` from a static method — needs `outerObj.new Counter()`
- Using an anonymous class where a lambda would do

✓ TIPS & BEST PRACTICES

- Default to `static` nested classes
- Use anonymous classes only when you need state or several methods

QUICK SUMMARY

- `static class` — independent
- Inner — bound to outer object
- Local — inside a method
- Anonymous — one-shot implementation

FileReader & FileWriter

BASIC CHARACTER FILE I/O

`FileReader` and `FileWriter` read and write **characters** to files. They're the simplest file classes in `java.io`, reading one character (or a buffer) at a time. On their own they're unbuffered and slow, so they're usually wrapped in `BufferedReader`/`BufferedWriter` (Sheet 77). Java 11 added constructors that take a `Charset`.

NOTES

- `FileWriter(path)` **overwrites**; `FileWriter(path, true)` appends
- Before Java 18, the default charset was platform-dependent
- `read()` returns `-1` at end of file
- Always close streams; use try-with-resources

READING & WRITING CHARACTERS

```
import java.io.*;
import java.nio.charset.StandardCharsets;

try (FileWriter fw = new FileWriter("notes.txt", StandardCharsets.UTF_8)) {
    fw.write("Line one\n");
    fw.write("Line two\n");
}                                           // overwrites file

try (var fw = new FileWriter("notes.txt", StandardCharsets.UTF_8, true)) {
    fw.write("Appended\n");                 // append mode
}

try (FileReader fr = new FileReader("notes.txt", StandardCharsets.UTF_8)) {
    int ch;
    while ((ch = fr.read()) != -1) {        // one char at a time
        System.out.print((char) ch);
    }
}

try (FileReader fr = new FileReader("notes.txt")) {
    char[] buf = new char[1024];
    int n = fr.read(buf);                   // fill a buffer
    String chunk = new String(buf, 0, n);
}
```

NOTES

- → `read()` returns an `int` so `-1` can signal EOF
- → Cast to `char` to print the character
- → Reading into a `char[]` is much faster than single chars
- → Both constructors throw `IOException`

CONSTRUCTORS

CONSTRUCTOR	BEHAVIOR
`FileWriter(String path)`	Create / overwrite
`FileWriter(String path, boolean append)`	Append if true
`FileWriter(String, Charset)`	Explicit encoding (11+)
`FileReader(String path)`	Open for reading
`FileReader(File file)`	From a File object

EXCEPTIONS

EXCEPTION	WHEN
`FileNotFoundException`	Path missing or is a directory
`IOException`	General read/write failure
`SecurityException`	No permission

⚠ COMMON MISTAKES

- Forgetting the `append` flag and wiping the file
- Not closing the writer, so buffered data never reaches disk
- Relying on the default charset on older JVMs — accented characters get garbled

✓ TIPS & BEST PRACTICES

- Wrap in `BufferedReader` / `BufferedWriter` for real work
- For small files, `Files.readString` / `Files.writeString` are simpler (Sheet 78)
- Always specify `StandardCharsets.UTF_8`

QUICK SUMMARY

- `FileReader` — read chars
- `FileWriter` — write chars
- `true` flag — append
- Close with try-with-resources

Buffered I/O

EFFICIENT LINE-BASED FILES

`BufferedReader` and `BufferedWriter` add an in-memory buffer around another reader or writer, so the program makes far fewer slow disk calls. `BufferedReader.readLine()` reads whole lines, and `BufferedWriter.newLine()` writes the platform line separator. `PrintWriter` adds `println` and `printf` on top.

NOTES

- Default buffer size is 8 KB
- `readLine()` strips the line terminator and returns `null` at EOF
- `flush()` pushes buffered data out; `close()` flushes automatically
- `Files.newBufferedReader(path)` defaults to UTF-8

READ & WRITE LINES

```
Path in = Path.of("input.txt"), out = Path.of("output.txt");

try (BufferedReader br = Files.newBufferedReader(in);
     BufferedWriter bw = Files.newBufferedWriter(out)) {
    String line;
    int n = 0;
    while ((line = br.readLine()) != null) {
        bw.write(++n + ": " + line);
        bw.newLine();
    }
}                                              // both closed + flushed

try (var br = new BufferedReader(new FileReader("data.csv"))) {
    long count = br.lines()                    // Stream<String>
                   .filter(l -> !l.isBlank())
                   .count();
}

try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(out))) {
    pw.println("Report");
    pw.printf("%-10s %5d%n", "items", 42);
}
```

NOTES

- → Opening two resources in one `try` header closes both
- → `br.lines()` turns the file into a lazy stream
- → `PrintWriter` swallows `IOException`s; call `checkError()` to detect them

BUFFEREDREADER

METHOD	RETURNS
`readLine()`	Next line or null
`lines()`	Stream<String>
`read()`	Next char or -1
`ready()`	true if data buffered
`skip(n)`	Skips n chars

BUFFEREDWRITER / PRINTWRITER

METHOD	EFFECT
`write(String)`	Write text
`newLine()`	Platform line break
`flush()`	Force write to disk
`println(x)`	PrintWriter: text + newline
`printf(fmt, …)`	PrintWriter: formatted

⚠ COMMON MISTAKES

- Forgetting `newLine()` and writing everything on one line
- Not closing or flushing the writer — the last buffer is lost
- Reading a huge file into a `List` when a streaming `while` loop would use constant memory

✓ TIPS & BEST PRACTICES

- Prefer `Files.newBufferedReader/Writer` — UTF-8 by default
- Use `br.lines()` with try-with-resources so the file closes
- Use `PrintWriter` when you want `printf`

QUICK SUMMARY

- Buffer = fewer disk calls
- `readLine()` until `null`
- `newLine()` — line break
- `PrintWriter` — `println` / `printf`

Files & NIO

MODERN FILE API

The **NIO.2** API (`java.nio.file`) is the modern way to work with files. A `Path` represents a location; the `Files` class provides static methods to read, write, copy, move, and inspect files, often in one line. It reports errors with clear exceptions and uses UTF-8 by default for text methods.

NOTES

- `Path.of` (Java 11) replaces `Paths.get`
- `readString` / `readAllLines` load the whole file into memory
- `Files.lines` streams lazily; close it with try-with-resources
- `StandardOpenOption` controls append, create, and truncate

ONE-LINERS

```
import java.nio.file.*;

Path p = Path.of("data", "notes.txt");          // data/notes.txt

Files.writeString(p, "Hello\nWorld\n");          // create / overwrite
Files.writeString(p, "More\n", StandardOpenOption.APPEND);
String all = Files.readString(p);                // whole file
List<String> lines = Files.readAllLines(p);      // [Hello, World, More]
Files.write(p, List.of("a", "b"));               // write lines

try (Stream<String> s = Files.lines(p)) {         // lazy, big files
    s.filter(l -> l.startsWith("E")).forEach(System.out::println);
}

byte[] bytes = Files.readAllBytes(Path.of("logo.png"));
Files.exists(p);                                 // true
Files.size(p);                                   // bytes
Files.copy(p, Path.of("backup.txt"), StandardCopyOption.REPLACE_EXISTING);
Files.move(p, Path.of("archive/notes.txt"));
Files.deleteIfExists(Path.of("temp.txt"));
```

NOTES

- → `Path.of("a", "b")` joins segments with the right separator
- → `APPEND` adds instead of overwriting
- → `copy` fails if the target exists unless you pass `REPLACE_EXISTING`
- → Directory operations are on Sheet 79

READ METHODS

METHOD	RETURNS	SIZE
`readString(p)`	String	Small
`readAllLines(p)`	List<String>	Small
`lines(p)`	Stream<String>	Any
`readAllBytes(p)`	byte[]	Small binary
`newBufferedReader(p)`	BufferedReader	Any

STANDARDOPENOPTION

OPTION	EFFECT
`CREATE`	Create if missing (default)
`APPEND`	Write at end
`TRUNCATE_EXISTING`	Empty first (default)
`CREATE_NEW`	Fail if exists

⚠ COMMON MISTAKES

- `readAllLines` on a multi-gigabyte log → `OutOfMemoryError`; use `lines`
- Leaving a `Files.lines` stream unclosed, which leaks a file handle
- Writing to a path whose parent folder doesn't exist → `NoSuchFileException`

✓ TIPS & BEST PRACTICES

- Call `Files.createDirectories(p.getParent())` before writing
- Use `Files.writeString(p, text, CREATE, APPEND)` for logs
- Convert legacy `File` objects with `file.toPath()`

QUICK SUMMARY

- `Path.of(...)` — a location
- `Files.readString` / `writeString`
- `Files.lines` — stream big files
- `copy` `move` `deleteIfExists`

File Paths & Directories

NAVIGATING THE FILE SYSTEM

A `Path` can be **absolute** (from the root) or **relative** (from the current working directory). Path methods combine, compare, and clean up paths without touching the disk. `Files` methods create, list, walk, and delete directories. Use these instead of building paths with string concatenation.

NOTES

- Relative paths resolve against `System.getProperty("user.dir")`
- `Files.list` is one level; `Files.walk` is recursive
- `Files.delete` fails on non-empty directories
- Close `list` / `walk` streams with try-with-resources

PATH MANIPULATION & DIRECTORIES

```
Path base = Path.of("/home/ada/projects");
Path file = base.resolve("app/src/Main.java");  // …/projects/app/src/Main.java
file.getFileName();                              // Main.java
file.getParent();                               // …/projects/app/src
Path.of("a/./b/../c").normalize();               // a/c
base.relativize(file);                           // app/src/Main.java
Path.of("notes.txt").toAbsolutePath();           // /current/dir/notes.txt

Files.createDirectories(Path.of("out/reports/2026"));  // all parents
Files.isDirectory(base);                                // true

try (Stream<Path> s = Files.list(base)) {              // one level
    s.forEach(System.out::println);
}
try (Stream<Path> s = Files.walk(base)) {              // recursive
    var java = s.filter(f -> f.toString().endsWith(".java")).toList();
}
try (Stream<Path> s = Files.walk(Path.of("tmp"))) {    // delete a tree
    s.sorted(Comparator.reverseOrder())
     .forEach(f -> f.toFile().delete());
}
```

NOTES

- → `resolve` appends a child path
- → `relativize` gives the path from one to another
- → Reverse-sorted walk deletes children before parents

PATH METHODS

METHOD	EXAMPLE RESULT
`getFileName()`	Main.java
`getParent()`	…/app/src
`resolve("x")`	base/x
`resolveSibling("y")`	Replace last part
`normalize()`	Removes . and ..
`toAbsolutePath()`	Full path

DIRECTORY OPERATIONS

METHOD	PURPOSE
`createDirectory(p)`	One level; fails if exists
`createDirectories(p)`	All levels; OK if exists
`list(p)`	Direct children
`walk(p, depth)`	Recursive, optional limit
`newDirectoryStream(p, "*.txt")`	Glob filter

⚠ COMMON MISTAKES

- Concatenating paths with `"/"` — breaks on Windows; use `resolve`
- Calling `createDirectory` when parents don't exist
- Assuming relative paths are relative to the source file; they're relative to the working directory

✓ TIPS & BEST PRACTICES

- Get the user's home with `Path.of(System.getProperty("user.home"))`
- Use glob filters: `Files.newDirectoryStream(dir, "*.{png,jpg}")`
- Load bundled files with `getResourceAsStream`, not file paths

QUICK SUMMARY

- `resolve` / `relativize` / `normalize`
- Absolute vs relative paths
- `createDirectories` — make tree
- `list` (flat) / `walk` (recursive)

JSON in Java

WORKING WITH JSON

JSON (JavaScript Object Notation) is the standard text format for web APIs and config files. The JDK has **no built-in JSON parser**, so Java programs use a library. The two most common are **Gson** (simple, Google) and **Jackson** (fast, feature-rich, used by Spring). Both map JSON to Java objects and back. Each has its own sheet.

NOTES

- Full Gson coverage: **Sheet 108**
- Full Jackson coverage: **Sheet 109**
- JSON keys are always double-quoted strings
- Records map cleanly to JSON objects in both libraries

JSON ↔ JAVA TYPES

JSON	JAVA
`{ "k": v }`	Object / record / Map<String, Object>
`[ … ]`	List<T> / array
`"text"`	String
`42 / 3.14`	int, long, double, BigDecimal
`true / false`	boolean
`null`	null

LIBRARY COMPARISON

FEATURE	GSON	JACKSON
`Main class`	Gson	ObjectMapper
`To JSON`	toJson(obj)	writeValueAsString(obj)
`From JSON`	fromJson(s, T.class)	readValue(s, T.class)
`Generic lists`	TypeToken	TypeReference
`Annotations`	@SerializedName	@JsonProperty
`Used by`	Android, small apps	Spring Boot, enterprise

SAME TASK, BOTH LIBRARIES

```
record User(String name, int age, List<String> roles) { }
String json = """
    {"name": "Ada", "age": 36, "roles": ["admin", "dev"]}
    """;

// Gson (Sheet 108)
Gson gson = new Gson();
User u1 = gson.fromJson(json, User.class);
String out1 = gson.toJson(u1);

// Jackson (Sheet 109)
ObjectMapper mapper = new ObjectMapper();
User u2 = mapper.readValue(json, User.class);
String out2 = mapper.writeValueAsString(u2);   // {"name":"Ada","age":36,…}
```

NOTES

- → **Serialize** = object → JSON; **deserialize** = JSON → object
- → Gson supports records from 2.10
- → Jackson supports records from 2.12
- → Add the library via Maven/Gradle (Sheets 86–87)

⚠ COMMON MISTAKES

- Parsing JSON with string methods or regex — use a library
- Mixing Gson and Jackson annotations on the same class and expecting both to apply
- Treating numbers as `double` when IDs exceed 2^{53} — use `long` or `String`

✓ TIPS & BEST PRACTICES

- Pick one library per project; Jackson if you use Spring
- Model JSON with records for concise, immutable DTOs
- Validate API JSON shapes with tests

QUICK SUMMARY

- No JSON in the JDK
- Gson — Sheet 108
- Jackson — Sheet 109
- Map JSON ↔ records/classes

CSV in Java

WORKING WITH CSV

CSV (comma-separated values) stores tables as plain text, one row per line. Simple files can be handled with `split(",")`, but real CSV allows **quoted fields** containing commas, quotes, and even line breaks, which naive splitting gets wrong. For anything user-supplied, use a library such as **OpenCSV (Sheet 111)**.

NOTES

- RFC 4180 defines the common CSV rules
- A quote inside a quoted field is doubled: `""`
- Excel may use `;` as the delimiter in some locales
- Watch for a UTF-8 BOM at the start of Excel exports

MANUAL PARSING (SIMPLE FILES ONLY)

```
record Product(String name, double price, int qty) { }

List<Product> products;
try (Stream<String> lines = Files.lines(Path.of("products.csv"))) {
    products = lines.skip(1)                          // header row
        .filter(l -> !l.isBlank())
        .map(l -> l.split(",", -1))                   // keep empty fields
        .map(f -> new Product(f[0].strip(),
                              Double.parseDouble(f[1]),
                              Integer.parseInt(f[2])))
        .toList();
}

List<String> out = new ArrayList<>();
out.add("name,price,qty");
for (Product p : products)
    out.add(String.join(",", p.name(), String.valueOf(p.price()),
                        String.valueOf(p.qty())));
Files.write(Path.of("copy.csv"), out);

"\"Smith, Ada\",42".split(",");    // ✖ ["\"Smith", " Ada\"", "42"]
```

NOTES

→ `skip(1)` drops the header

→ `split(",", -1)` keeps trailing empty columns

→ The last line shows why quoted commas break `split`

CSV RULES

CASE	ENCODED AS
Plain field	Ada
Field with comma	"Smith, Ada"
Field with quote	"She said ""hi"""
Empty field	(nothing between commas)
Field with newline	Quoted, newline inside quotes

APPROACH BY SITUATION

SITUATION	USE
Fixed, trusted, no quotes	split(",", -1)
User or Excel files	OpenCSV (Sheet 111)
Bean mapping by header	OpenCSV CsvToBean
Huge files	Stream line by line

⚠ COMMON MISTAKES

- `split(",")` on data that contains quoted commas
- Forgetting to skip or validate the header row
- Writing fields without escaping commas and quotes, which produces broken files

✓ TIPS & BEST PRACTICES

- Use a library the moment quoting is possible
- Write numbers with `Locale.ROOT` so decimals use `.`
- Strip a leading BOM: `line.replace("\uFEFF", "")`

QUICK SUMMARY

- Text tables, one row per line
- Quotes allow commas in fields
- `split` only for simple data
- OpenCSV — Sheet 111

Serialization

SAVING OBJECTS AS BYTES

Java **serialization** converts an object graph into a byte stream with `ObjectOutputStream` and rebuilds it with `ObjectInputStream`. A class opts in by implementing the marker interface `Serializable`. Fields marked `transient` are skipped. It's convenient for quick persistence, but it's fragile across versions and **unsafe with untrusted data**.

NOTES

- Every object in the graph must be `Serializable`
- `static` fields are never serialized
- Declare `serialVersionUID` to control version compatibility
- Never deserialize data from untrusted sources (remote code execution risk)

WRITE AND READ AN OBJECT

```
public class Session implements Serializable {
    private static final long serialVersionUID = 1L;
    private final String user;
    private final List<String> history = new ArrayList<>();
    private transient String password;          // not saved

    public Session(String u, String pw) { user = u; password = pw; }
}

Session s = new Session("ada", "secret");
try (var out = new ObjectOutputStream(
        new BufferedOutputStream(new FileOutputStream("session.bin")))) {
    out.writeObject(s);
}

try (var in = new ObjectInputStream(
        new BufferedInputStream(new FileInputStream("session.bin")))) {
    Session loaded = (Session) in.readObject();   // cast needed
    // loaded.password == null (transient)
}
```

NOTES

- → `readObject` returns `Object`, so you cast
- → `readObject` throws `ClassNotFoundException` as well as `IOException`
- → Constructors are **not** run during deserialization
- → `transient` fields get default values

KEY PIECES

ITEM	ROLE
`Serializable`	Marker interface (no methods)
`serialVersionUID`	Version stamp
`transient`	Exclude a field
`ObjectOutputStream`	writeObject(obj)
`ObjectInputStream`	readObject()

ALTERNATIVES

FORMAT	LIBRARY
`JSON`	Jackson / Gson
`Binary, schema-based`	Protocol Buffers, Avro
`Database`	JDBC / JPA
`Simple config`	Properties, YAML

⚠ COMMON MISTAKES

- Changing a class without a fixed `serialVersionUID` → `InvalidClassException` on old files
- Serializing objects with non-serializable fields → `NotSerializableException`
- Deserializing network or user-supplied bytes

✓ TIPS & BEST PRACTICES

- Prefer JSON or Protobuf for anything long-lived or shared
- Use `ObjectInputFilter` (Java 9+) to restrict allowed classes
- Mark caches, passwords, and connections `transient`

QUICK SUMMARY

- `implements Serializable`
- `writeObject` / `readObject`
- `transient` — skip a field
- Prefer JSON for real systems

Byte Streams & I/O Streams

READING AND WRITING RAW BYTES

`InputStream` and `OutputStream` are the base classes for **byte** I/O: images, audio, zip files, network data. Readers and writers (Sheets 76–77) handle text; streams handle binary. Decorator classes stack on top: `BufferedInputStream` adds buffering, and `DataInputStream` reads primitives like `int` and `double` in a fixed binary format.

NOTES

- `read()` returns a byte as `0-255`, or `-1` at end
- Use `InputStreamReader` to turn bytes into characters
- Java 9+: `readAllBytes()` and `transferTo(out)`
- Data streams use big-endian order

BINARY COPY & PRIMITIVE DATA

```
try (var in = new BufferedInputStream(new FileInputStream("photo.jpg"));
     var out = new BufferedOutputStream(new FileOutputStream("copy.jpg"))) {
    in.transferTo(out);                          // copy everything (Java 9+)
}

try (var in = new FileInputStream("photo.jpg")) {
    byte[] buf = new byte[8192];
    int n;
    while ((n = in.read(buf)) != -1) {           // manual chunked read
        process(buf, n);
    }
}

try (var dos = new DataOutputStream(new FileOutputStream("scores.dat"))) {
    dos.writeInt(3);
    dos.writeDouble(98.5);
    dos.writeUTF("Ada");
}
try (var dis = new DataInputStream(new FileInputStream("scores.dat"))) {
    int count = dis.readInt();                   // 3
    double score = dis.readDouble();             // 98.5
    String name = dis.readUTF();                 // "Ada"
}
```

NOTES

- → `transferTo` is the simplest stream copy
- → Only the first `n` bytes of `buf` are valid each pass
- → Read data back in the **same order** it was written

STREAM CLASS FAMILIES

PURPOSE	INPUT	OUTPUT
File bytes	FileInputStream	FileOutputStream
Buffering	BufferedInputStream	BufferedOutputStream
Primitives	DataInputStream	DataOutputStream
Objects	ObjectInputStream	ObjectOutputStream
In-memory	ByteArrayInputStream	ByteArrayOutputStream

BYTES ↔ TEXT BRIDGES

NEED	USE
Bytes → chars	new InputStreamReader(in, UTF_8)
Chars → bytes	new OutputStreamWriter(out, UTF_8)
String → bytes	s.getBytes(UTF_8)
Bytes → String	new String(bytes, UTF_8)

⚠ COMMON MISTAKES

- Using `FileReader` for binary files — bytes get corrupted by decoding
- Writing the whole buffer instead of only the `n` bytes read
- Reading primitives back in a different order than they were written

✓ TIPS & BEST PRACTICES

- For whole files, `Files.readAllBytes` / `Files.write` are simplest
- Always buffer file streams
- Use `ByteArrayOutputStream` to capture output in tests

QUICK SUMMARY

- Streams = bytes; readers = chars
- `transferTo` — copy
- `DataInput/OutputStream` — primitives
- Buffer everything

Packages & Imports

ORGANIZING AND REFERENCING CLASSES

A **package** is a namespace that groups related classes, like `java.util` or `com.acme.billing`. Every class's full name includes its package: `java.util.ArrayList`. An `import` statement lets you use the short name. Imports are purely a compile-time convenience; they don't load or include code.

NOTES

- `java.lang` (String, Math, System) is imported automatically
- Classes in the same package need no import
- Wildcard `*` imports one package, not its subpackages
- Java 25: `import module java.base;` imports a whole module

IMPORT FORMS

```
package com.acme.billing;                    // this file's package

import java.util.List;                       // single-type import
import java.util.*;                          // wildcard: all of java.util
import java.time.LocalDate;
import static java.lang.Math.PI;             // static member import
import static java.lang.Math.*;              // all static members

public class Invoice {
    List<String> items = new ArrayList<>();      // from java.util.*
    LocalDate due = LocalDate.now().plusDays(30);
    double area = PI * pow(2, 2);                // static imports

    java.sql.Date sqlDate;                   // fully qualified: avoids
    java.util.Date utilDate;                 // the Date name clash
}
```

NOTES

- → `package` must be the first statement
- → Static imports bring in fields and methods
- → When two packages share a class name, qualify one fully
- → Unused imports are harmless but clutter files

IMPORT KINDS

FORM	BRINGS IN
`import a.b.C;`	Class C
`import a.b.*;`	All classes in a.b
`import static a.b.C.m;`	Static member m
`import static a.b.C.*;`	All static members
`import module m;`	Every exported package (Java 25)

COMMON JDK PACKAGES

PACKAGE	CONTENTS
`java.lang`	String, Math, System, Integer
`java.util`	Collections, Scanner, Optional
`java.time`	LocalDate, Duration
`java.io / java.nio.file`	Files and streams
`java.util.function`	Function, Predicate

⚠ COMMON MISTAKES

- Thinking `import java.util.*` also imports `java.util.function` — it doesn't
- Importing two classes with the same simple name — compile error
- Overusing static wildcard imports so readers can't tell where `max` comes from

✓ TIPS & BEST PRACTICES

- Prefer single-type imports; let the IDE manage them
- Use static imports for test assertions and constants
- Organize imports automatically (Ctrl+Alt+O in IntelliJ)

QUICK SUMMARY

- `package` — namespace
- `import a.b.C` — short name
- `import static` — static members
- `java.lang` is automatic

Creating Packages & Modules

PACKAGES ON DISK AND JAVA MODULES

Package names map directly to **folders**: `com.acme.app` lives in `com/acme/app/`. Name packages with a reversed domain to keep them globally unique. Java 9 added the **module system** (JPMS): a `module-info.java` file declares which packages a module **exports** and which modules it **requires**, giving strong encapsulation between JARs.

NOTES

- Package names are all lowercase
- `src/main/java` is the Maven/Gradle source root
- Unexported packages are invisible to other modules
- `opens` allows reflection (needed by Jackson, JavaFX FXML)

DIRECTORY LAYOUT

PATH	PACKAGE
src/main/java/com/acme/app/Main.java	com.acme.app
src/main/java/com/acme/app/model/User.java	com.acme.app.model
src/main/java/com/acme/app/util/Strings.java	com.acme.app.util
src/main/java/module-info.java	(module descriptor)
src/test/java/com/acme/app/MainTest.java	com.acme.app (tests)

MODULE-INFO.JAVA

```
module com.acme.app {
    requires java.net.http;                 // JDK module
    requires transitive com.acme.core;      // consumers get it too
    requires static lombok;                 // compile-time only

    exports com.acme.app.api;               // public API
    exports com.acme.app.spi to com.acme.plugins;   // qualified export

    opens com.acme.app.model to com.fasterxml.jackson.databind;
    opens com.acme.app.ui to javafx.fxml;   // reflection for FXML

    uses com.acme.app.spi.Exporter;         // service loader
}

// Compile & run with modules
// javac -d out --module-path libs $(find src -name "*.java")
// java --module-path out:libs -m com.acme.app/com.acme.app.Main
```

NOTES

- → `requires` — dependency on another module
- → `exports` — packages others may compile against
- → `opens` — packages others may reflect into
- → No `module-info.java` → your code runs on the classpath (unnamed module)

⚠ COMMON MISTAKES

- Folder path not matching the `package` line — compile error or class not found
- Forgetting `opens` for FXML controllers or Jackson DTOs → `InaccessibleObjectException`
- Splitting one package across two modules — not allowed

✓ TIPS & BEST PRACTICES

- Start with the classpath; add `module-info.java` when you need it (JavaFX apps usually do)
- Group by feature (`orders` , `users`) rather than by layer
- Keep `exports` minimal

QUICK SUMMARY

- Package = folder path
- Reverse-domain names
- `module-info.java` — JPMS
- `requires` `exports` `opens`

Maven

BUILD AND DEPENDENCY MANAGEMENT

Maven builds Java projects from a declarative `pom.xml` file. You list dependencies by coordinates (`groupId:artifactId:version`) and Maven downloads them from **Maven Central**. Builds follow a fixed **lifecycle** (compile → test → package → install), and plugins do the work at each phase.

NOTES

- Convention: `src/main/java`, `src/test/java`, `target/` output
- Local repository cache: `~/.m2/repository`
- Use the Maven Wrapper (`./mvnw`) to pin the Maven version
- Search packages at central.sonatype.com

POM.XML

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.acme</groupId>
  <artifactId>billing</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.google.code.gson</groupId>
      <artifactId>gson</artifactId>
      <version>2.11.0</version>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.11.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

NOTES

- → `groupId` — organization; `artifactId` — project name
- → `maven.compiler.release` — target Java version
- → `test` scope keeps JUnit out of the final JAR
- → `-SNAPSHOT` versions mean "in development"

MVN COMMANDS

COMMAND	DOES
`mvn compile`	Compile main code
`mvn test`	Compile + run tests
`mvn package`	Build JAR in target/
`mvn install`	Put JAR in ~/.m2
`mvn dependency:tree`	Show all dependencies

DEPENDENCY SCOPES

SCOPE	AVAILABLE AT
`compile (default)`	Compile + runtime
`test`	Tests only
`provided`	Compile; supplied at runtime
`runtime`	Runtime only (e.g., JDBC driver)

⚠ COMMON MISTAKES

- Forgetting `<scope>test</scope>` and shipping test libraries
- Two versions of one library via transitive dependencies; check `dependency:tree`
- Running `mvn package` and expecting a runnable fat JAR — that needs the Shade or Assembly plugin

✓ TIPS & BEST PRACTICES

- Put versions in `<properties>` and reuse them
- Use a BOM (`<dependencyManagement>`) for aligned versions
- `mvn -q -DskipTests package` for quick local builds

QUICK SUMMARY

- `pom.xml` — project definition
- `groupId:artifactId:version`
- compile → test → package → install
- Scopes control the classpath

Gradle

SCRIPTABLE BUILDS

Gradle is a build tool configured with code instead of XML: either the **Kotlin DSL** (`build.gradle.kts`, the default for new projects) or the **Groovy DSL** (`build.gradle`). It uses the same Maven Central dependencies, supports incremental builds and caching, and is the standard for Android projects.

NOTES

- Always run through the wrapper: `./gradlew` (Windows: `gradlew.bat`)
- `implementation` hides a dependency from consumers; `api` exposes it
- `settings.gradle(.kts)` names the project and lists subprojects
- Output goes to `build/`

BUILD.GRADLE.KTS (KOTLIN DSL)

```
plugins { application }

java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
repositories { mavenCentral() }

dependencies {
    implementation("com.google.code.gson:gson:2.11.0")
    testImplementation("org.junit.jupiter:junit-jupiter:5.11.0")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
}

application { mainClass = "com.acme.billing.Main" }

tasks.test { useJUnitPlatform() }
```

NOTES

- → `toolchain` downloads/selects the right JDK
- → `./gradlew build` — compile, test, assemble
- → `./gradlew run` — run `mainClass` (application plugin)
- → `./gradlew test` / `clean` / `dependencies`
- → `useJUnitPlatform()` is required for JUnit 5

SAME IN GROOVY DSL

```
plugins { id 'application' }
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
repositories { mavenCentral() }
dependencies {
    implementation 'com.google.code.gson:gson:2.11.0'
    testImplementation 'org.junit.jupiter:junit-jupiter:5.11.0'
}
application { mainClass = 'com.acme.billing.Main' }
test { useJUnitPlatform() }
```

CONFIGURATIONS

CONFIG	MAVEN EQUIVALENT
`implementation`	compile
`api`	compile (exposed)
`compileOnly`	provided
`runtimeOnly`	runtime
`testImplementation`	test

⚠ COMMON MISTAKES

- Using `compile` (removed in Gradle 7) instead of `implementation`
- Running a system-installed `gradle` instead of `./gradlew`, causing version mismatches
- Forgetting `useJUnitPlatform()` so no tests run

✓ TIPS & BEST PRACTICES

- Prefer Kotlin DSL for IDE autocompletion
- Use a version catalog (`libs.versions.toml`) for shared versions
- `./gradlew build --scan` explains slow or failing builds

QUICK SUMMARY

- `build.gradle.kts` / `build.gradle`
- `implementation` `testImplementation`
- `./gradlew build` / `run`
- Wrapper pins the version

Math Class

BUILT-IN MATH FUNCTIONS

`java.lang.Math` is a utility class of `static` methods and constants for common numeric work: powers, roots, rounding, trigonometry, logarithms, and overflow-safe arithmetic. Most methods take and return `double`. Because it's in `java.lang`, no import is needed.

NOTES

- Trig functions use **radians**; convert with `Math.toRadians`
- `round` returns `long` for `double` input, `int` for `float`
- `*Exact` methods throw `ArithmeticException` on overflow
- `StrictMath` gives bit-identical results across platforms

COMMON CALLS

```
Math.abs(-7.5);          // 7.5
Math.pow(2, 10);         // 1024.0
Math.sqrt(144);          // 12.0
Math.cbrt(27);           // 3.0
Math.max(3, 9);          // 9
Math.min(-1, 4);         // -1

Math.round(2.5);         // 3    (half up → long)
Math.round(-2.5);        // -2   (toward +∞)
Math.floor(2.9);         // 2.0
Math.ceil(2.1);          // 3.0
Math.rint(2.5);          // 2.0  (half to even)

Math.floorDiv(-7, 2);    // -4
Math.floorMod(-7, 2);    // 1
Math.hypot(3, 4);        // 5.0
Math.clamp(15, 0, 10);   // 10   (Java 21)
Math.addExact(Integer.MAX_VALUE, 1);   // ✖ ArithmeticException
Math.round(3.14159 * 100) / 100.0;     // 3.14  (2 decimals)
```

NOTES

- → `floor` / `ceil` return `double`; cast if you need `int`
- → `round` ties go toward positive infinity
- → `clamp` limits a value to a range (Java 21+)
- → For exact money rounding use `BigDecimal.setScale`

CONSTANTS & TRIG

MEMBER	VALUE / PURPOSE
`Math.PI`	3.141592653589793
`Math.E`	2.718281828459045
`sin / cos / tan(rad)`	Trig in radians
`toRadians(deg)`	180 → π
`atan2(y, x)`	Angle of a point

LOGS & POWERS

METHOD	EXAMPLE → RESULT
`log(x)`	log(Math.E) → 1.0
`log10(x)`	log10(1000) → 3.0
`exp(x)`	exp(1) → 2.718…
`pow(b, e)`	pow(9, 0.5) → 3.0
`signum(x)`	signum(-4.2) → -1.0

⚠ COMMON MISTAKES

- `Math.sin(90)` expecting 1 — 90 is radians; use `Math.sin(Math.toRadians(90))`
- `int x = Math.round(2.7);` doesn't compile — it returns `long`
- `Math.abs(Integer.MIN_VALUE)` is still negative (overflow)

✓ TIPS & BEST PRACTICES

- Use `Math.floorMod` for wrap-around indexes (clock math, circular buffers)
- Use `*Exact` methods where silent overflow would be a bug
- Use `Random` / `ThreadLocalRandom` instead of `Math.random()` (Sheet 89)

QUICK SUMMARY

- Static methods, no import
- `pow` `sqrt` `abs` `max` `min`
- `round` `floor` `ceil`
- Radians for trig

Random Numbers

GENERATING RANDOM VALUES

`java.util.Random` produces **pseudo-random** numbers from a seed. With a fixed seed, the sequence is reproducible, which is useful for tests and simulations. `ThreadLocalRandom` is faster in multithreaded code, and `SecureRandom` produces **cryptographically strong** values for tokens and passwords. `Math.random()` is a quick shortcut for a `double` in `[0, 1)`.

NOTES

- `nextInt(bound)` returns `0` to `bound - 1`
- Java 17+: `nextInt(origin, bound)` for ranges
- Never use `Random` for security-sensitive values
- `Collections.shuffle(list, rnd)` shuffles reproducibly

RANDOM IN PRACTICE

```
Random rnd = new Random();
rnd.nextInt();                  // any int
rnd.nextInt(6);                 // 0–5
rnd.nextInt(6) + 1;             // 1–6 (die roll)
rnd.nextInt(10, 21);            // 10–20 (Java 17+)
rnd.nextDouble();               // 0.0 ≤ x < 1.0
rnd.nextBoolean();              // true / false
rnd.nextGaussian();             // normal: mean 0, sd 1

Random seeded = new Random(42);  // same sequence every run
seeded.nextInt(100);             // always the same first value

int r = ThreadLocalRandom.current().nextInt(1, 101);   // 1–100, thread-safe

SecureRandom sec = new SecureRandom();
byte[] token = new byte[16];
sec.nextBytes(token);
String hex = HexFormat.of().formatHex(token);          // 32 hex chars

List<String> deck = new ArrayList<>(List.of("A", "K", "Q"));
Collections.shuffle(deck, new Random(7));
String pick = deck.get(rnd.nextInt(deck.size()));      // random element
```

NOTES

- → Upper bounds are **exclusive**
- → `ThreadLocalRandom.current()` — never share or store it
- → `HexFormat` (Java 17) formats bytes as hex
- → Random element: index `nextInt(size)`

WHICH GENERATOR?

CLASS	USE FOR
`Random`	General purpose, seeds, tests
`ThreadLocalRandom`	Concurrent code
`SecureRandom`	Tokens, keys, passwords
`Math.random()`	Quick one-off double
`RandomGenerator (17+)`	Interface for swappable algorithms

RANGE FORMULAS

RANGE	CODE
`0 … n-1`	nextInt(n)
`1 … n`	nextInt(n) + 1
`min … max`	nextInt(min, max + 1)
`min … max (double)`	min + nextDouble() * (max - min)

⚠ COMMON MISTAKES

- `rnd.nextInt(6)` for a die — that's 0–5, add 1
- Creating `new Random()` inside a loop; reuse one instance
- Generating passwords or reset tokens with `Random`

✓ TIPS & BEST PRACTICES

- Seed `Random` in tests to make failures reproducible
- Use `rnd.ints(5, 1, 50)` for a stream of random ints
- Use `UUID.randomUUID()` for unique IDs

QUICK SUMMARY

- `Random` — seedable PRNG
- `nextInt(bound)` — exclusive upper
- `ThreadLocalRandom` — concurrent
- `SecureRandom` — security

StringBuilder

MUTABLE STRINGS

Because `String` is immutable, building text with + in a loop creates a new string every pass. `StringBuilder` is a **mutable** character buffer: `append`, `insert`, `delete`, and `reverse` modify it in place, and `toString()` produces the final `String`. Methods return the builder itself, so calls can be chained.

NOTES

- Not thread-safe; `StringBuffer` is the synchronized version (Sheet 19)
- Default capacity is 16 characters; it grows automatically
- The compiler already optimizes simple `a + b + c` expressions
- Use it when concatenating inside loops

BUILDING TEXT

```
StringBuilder sb = new StringBuilder();
sb.append("Hello").append(", ").append("World")
  .append('!').append(2026);          // Hello, World!2026

sb.insert(0, ">> ");                  // >> Hello, World!2026
sb.delete(sb.length() - 4, sb.length());   // >> Hello, World!
sb.deleteCharAt(0);                   // > Hello, World!
sb.replace(2, 7, "Howdy");            // > Howdy, World!
sb.setCharAt(0, '<');                 // < Howdy, World!
String result = sb.toString();

new StringBuilder("stressed").reverse().toString();   // "desserts"

StringBuilder csv = new StringBuilder(256);           // pre-sized
for (int i = 0; i < 5; i++) {
    if (!csv.isEmpty()) csv.append(',');
    csv.append(i);
}                                                     // "0,1,2,3,4"
sb.setLength(0);                                      // clear & reuse
```

NOTES

- → `delete(start, end)` — end is exclusive
- → `setLength(0)` empties the builder without reallocating
- → `String.join` / `Collectors.joining` are simpler for separators

METHODS

METHOD	EFFECT
`append(x)`	Add to end (any type)
`insert(i, x)`	Insert at index
`delete(s, e)`	Remove range
`replace(s, e, str)`	Replace range
`reverse()`	Reverse in place

STRING VS STRINGBUILDER

TASK	USE
`Fixed text`	String
`Few concatenations`	+ operator
`Loop concatenation`	StringBuilder
`Joining with separator`	String.join / joining()

⚠ COMMON MISTAKES

- `str += piece` inside a large loop — O(n²) copying
- Comparing two builders with `equals` — it compares references; use `compareTo` or `toString()`
- Forgetting `toString()` when a `String` is required

✓ TIPS & BEST PRACTICES

- Pre-size with `new StringBuilder(expectedLength)` for big outputs
- Chain calls for readability
- Reuse one builder in loops with `setLength(0)`

QUICK SUMMARY

- Mutable, fast concatenation
- `append` `insert` `delete` `reverse`
- Chainable methods
- `toString()` for the result

LocalDate

DATES WITHOUT TIMES

`LocalDate` (in `java.time`, Java 8+) represents a calendar date such as 2026-10-02, with **no time and no time zone**. It's immutable and thread-safe, so every "change" returns a new object. Use it for birthdays, due dates, and holidays. It replaces the old `java.util.Date` and `Calendar` APIs.

NOTES

- Months are 1–12 (unlike the old `Calendar`'s 0–11)
- `parse` expects ISO format `yyyy-MM-dd` by default
- `Period` measures date-based amounts (years, months, days)
- Formatting is on Sheet 94

WORKING WITH DATES

```
LocalDate today = LocalDate.now();                  // 2026-10-02
LocalDate d = LocalDate.of(2026, 12, 25);
LocalDate p = LocalDate.parse("2026-03-15");

d.getYear();                 // 2026
d.getMonth();                // DECEMBER
d.getMonthValue();           // 12
d.getDayOfWeek();            // FRIDAY
d.getDayOfYear();            // 359
d.isLeapYear();              // false
d.lengthOfMonth();           // 31

LocalDate due = today.plusDays(30).plusMonths(1);   // new object
today.minusWeeks(2);
today.withDayOfMonth(1);                            // first of month
today.with(TemporalAdjusters.lastDayOfMonth());

today.isBefore(d);                                  // true
long days = ChronoUnit.DAYS.between(today, d);      // 84
Period age = Period.between(LocalDate.of(1990, 5, 1), today);
age.getYears();                                     // 36
```

NOTES

- → `plusX` returns a new date; assign the result
- → `ChronoUnit.DAYS.between` — total days
- → `Period.between` — years/months/days breakdown
- → `TemporalAdjusters` handles "next Monday", "last day"

CREATION

METHOD	EXAMPLE
`now()`	Today (system zone)
`of(y, m, d)`	of(2026, 1, 31)
`parse(text)`	parse("2026-01-31")
`ofYearDay(y, n)`	ofYearDay(2026, 100)
`ofEpochDay(n)`	Days since 1970-01-01

COMPARISON & MATH

METHOD	PURPOSE
`isBefore / isAfter / isEqual`	Compare
`plusDays / Weeks / Months / Years`	Add
`minusX`	Subtract
`datesUntil(end)`	Stream of dates
`Period.between(a, b)`	Y-M-D difference

⚠ COMMON MISTAKES

- Calling `date.plusDays(1);` without assigning — the original never changes
- `of(2026, 2, 30)` → `DateTimeException`; invalid dates are rejected
- Using `LocalDate` for timestamps that need a time zone; use `ZonedDateTime` or `Instant`

✓ TIPS & BEST PRACTICES

- Store dates as ISO strings or SQL `DATE`
- Inject a `Clock` and call `LocalDate.now(clock)` for testable code
- Use `datesUntil` to loop over a range of days

QUICK SUMMARY

- Date only, no time or zone
- Immutable: `plusX` returns new
- `of` `now` `parse`
- `Period` / `ChronoUnit` for gaps

LocalTime

TIMES WITHOUT DATES

`LocalTime` represents a time of day like 14:30:15, with **no date and no time zone**, down to nanosecond precision. It's immutable and wraps around midnight: adding 2 hours to 23:00 gives 01:00. Use it for opening hours, alarms, and schedules that repeat daily.

NOTES

- Range: 00:00 to 23:59:59.999999999
- `Duration` measures time-based amounts
- Arithmetic wraps at midnight with no day overflow
- `parse` expects `HH:mm` or `HH:mm:ss`

WORKING WITH TIMES

```
LocalTime now = LocalTime.now();              // 14:32:07.123456
LocalTime open = LocalTime.of(9, 0);          // 09:00
LocalTime close = LocalTime.parse("17:30");   // 17:30

open.getHour();                  // 9
close.getMinute();               // 30
LocalTime.NOON;                  // 12:00
LocalTime.MIDNIGHT;              // 00:00

open.plusHours(2).plusMinutes(15);            // 11:15
LocalTime.of(23, 0).plusHours(2);             // 01:00 (wraps)
now.truncatedTo(ChronoUnit.MINUTES);          // 14:32
now.withSecond(0).withNano(0);                // 14:32

boolean isOpen = !now.isBefore(open) && now.isBefore(close);
Duration shift = Duration.between(open, close);
shift.toMinutes();                            // 510
shift.toHours();                              // 8
shift.toMinutesPart();                        // 30
long mins = ChronoUnit.MINUTES.between(open, close);   // 510
```

NOTES

- → `!isBefore(open)` means "at or after open"
- → `Duration.toMinutesPart` gives the leftover minutes
- → `truncatedTo` drops smaller units
- → Wrap-around means overnight spans need extra care

CREATION

METHOD	RESULT
`now()`	Current time
`of(h, m) / of(h, m, s)`	Specific time
`parse("08:45")`	From text
`ofSecondOfDay(3600)`	01:00
`MIN / MAX / NOON / MIDNIGHT`	Constants

DURATION METHODS

METHOD	EXAMPLE → RESULT
`Duration.ofMinutes(90)`	PT1H30M
`toHours()`	1
`toMinutesPart()`	30
`plus(d) / minus(d)`	Combine
`getSeconds()`	Total seconds

⚠ COMMON MISTAKES

- `Duration.between(23:00, 01:00)` is **negative** (-22h) — overnight spans need dates
- Forgetting results are new objects: `t.plusHours(1);` alone does nothing
- Comparing times with `==` instead of `equals` / `isBefore`

✓ TIPS & BEST PRACTICES

- Use `LocalDateTime` (Sheet 93) when spans cross midnight
- Truncate before display or comparison to avoid nanosecond noise
- Format with `DateTimeFormatter.ofPattern("h:mm a")` (Sheet 94)

QUICK SUMMARY

- Time only, no date or zone
- Wraps at midnight
- `plusX` `withX` `truncatedTo`
- `Duration` for gaps

LocalDateTime & Zones

DATE + TIME, WITH AND WITHOUT ZONES

`LocalDateTime` combines a date and a time but has **no zone**, so it doesn't identify an exact moment. `ZonedDateTime` adds a `ZoneId` (like `America/New_York`) and handles daylight saving. `Instant` is an exact point on the UTC timeline, ideal for timestamps and logs.

NOTES

- Store moments as `Instant` (UTC); convert to a zone for display
- `ZoneId.of("Europe/Paris")` — use region IDs, not "CET"
- DST gaps and overlaps are resolved automatically
- `Instant.toEpochMilli()` ↔ legacy millisecond timestamps

CONVERTING BETWEEN TYPES

```
LocalDateTime meeting = LocalDateTime.of(2026, 10, 2, 14, 30);
LocalDateTime ldt = LocalDate.of(2026, 1, 1).atTime(9, 0);
LocalDateTime now = LocalDateTime.now();

ZonedDateTime ny = meeting.atZone(ZoneId.of("America/New_York"));
ZonedDateTime tokyo = ny.withZoneSameInstant(ZoneId.of("Asia/Tokyo"));
// ny:    2026-10-02T14:30-04:00[America/New_York]
// tokyo: 2026-10-03T03:30+09:00[Asia/Tokyo]

Instant stamp = Instant.now();                    // 2026-10-02T18:30:00Z
long ms = stamp.toEpochMilli();
Instant back = Instant.ofEpochMilli(ms);
ZonedDateTime local = stamp.atZone(ZoneId.systemDefault());

meeting.toLocalDate();                            // 2026-10-02
meeting.plusHours(36);                            // 2026-10-04T02:30
Duration d = Duration.between(stamp, stamp.plusSeconds(90));   // PT1M30S
```

NOTES

- → `withZoneSameInstant` — same moment, different wall clock
- → `withZoneSameLocal` — same wall clock, different moment
- → `Instant` prints in UTC with a trailing `Z`

WHICH TYPE?

TYPE	HAS DATE	HAS TIME	HAS ZONE	USE FOR
`LocalDate`	✔	✖	✖	Birthdays
`LocalTime`	✖	✔	✖	Opening hours
`LocalDateTime`	✔	✔	✖	Local events, forms
`ZonedDateTime`	✔	✔	✔	Scheduling across zones
`Instant`	(UTC)	✔	UTC	Timestamps, logs

LEGACY CONVERSION

FROM	TO
`Date`	date.toInstant()
`Instant`	Date.from(instant)
`java.sql.Timestamp`	ts.toLocalDateTime()
`Calendar`	((GregorianCalendar) c).toZonedDateTime()

⚠ COMMON MISTAKES

- Storing `LocalDateTime` for events users see in different zones
- Using three-letter zone IDs like "EST" — ambiguous, and some don't observe DST
- Assuming a day is always 24 hours; DST days are 23 or 25

✓ TIPS & BEST PRACTICES

- Persist `Instant` (or `TIMESTAMP WITH TIME ZONE`)
- Convert to the user's zone only at display time
- Use `ZoneId.getAvailableZoneIds()` for zone pickers

QUICK SUMMARY

- `LocalDateTime` — no zone
- `ZonedDateTime` — zone + DST
- `Instant` — UTC moment
- Store UTC, display local

DateTimeFormatter

FORMATTING AND PARSING DATES

`DateTimeFormatter` converts `java.time` objects to text (`format`) and text back to objects (`parse`). Use built-in ISO formatters, localized styles, or a custom pattern built from **pattern letters** like `yyyy-MM-dd`. Formatters are immutable and thread-safe, so store them in `static final` constants.

NOTES

- Pattern letters are **case-sensitive**: `MM` month, `mm` minute
- Use `uuuu` or `yyyy` for year; `YYYY` is week-based year
- Pass a `Locale` for month and day names
- Parsing fails with `DateTimeParseException` on mismatch

FORMAT & PARSE

```
LocalDateTime t = LocalDateTime.of(2026, 10, 2, 14, 5);

t.format(DateTimeFormatter.ISO_LOCAL_DATE);              // 2026-10-02
DateTimeFormatter f = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
t.format(f);                                             // 02/10/2026 14:05
DateTimeFormatter.ofPattern("EEEE, MMMM d, yyyy").format(t);
                                          // Friday, October 2, 2026
DateTimeFormatter.ofPattern("h:mm a", Locale.US).format(t);   // 2:05 PM
DateTimeFormatter.ofPattern("d MMM", Locale.FRANCE).format(t);  // 2 oct.

DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM)
    .withLocale(Locale.UK).format(t);                    // 2 Oct 2026

LocalDate d = LocalDate.parse("25/12/2026",
    DateTimeFormatter.ofPattern("dd/MM/yyyy"));          // 2026-12-25

static final DateTimeFormatter LOG =
    DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm:ss.SSS");
```

NOTES

- → `'T'` — literal text in single quotes
- → Locale controls names: Friday vs vendredi
- → Reusing a `static final` formatter avoids rebuilding it

PATTERN LETTERS

LETTER	MEANING	EXAMPLE
`yyyy / uuuu`	Year	2026
`MM / MMM / MMMM`	Month	10 / Oct / October
`dd / d`	Day of month	02 / 2
`EEE / EEEE`	Weekday	Fri / Friday
`HH / hh`	Hour 0–23 / 1–12	14 / 02
`mm / ss / SSS`	Minute / second / millis	05 / 09 / 123
`a`	AM / PM	PM

PREDEFINED FORMATTERS

CONSTANT	OUTPUT
`ISO_LOCAL_DATE`	2026-10-02
`ISO_LOCAL_TIME`	14:05:00
`ISO_LOCAL_DATE_TIME`	2026-10-02T14:05:00
`ISO_INSTANT`	2026-10-02T18:05:00Z

⚠ COMMON MISTAKES

- `YYYY` instead of `yyyy` — wrong year around New Year
- `mm` for month (it's minutes) or `hh` without `a`
- Parsing a `LocalDate` with a pattern that includes time fields

✓ TIPS & BEST PRACTICES

- Keep formatters as `static final` constants
- Exchange data in ISO-8601; format for humans only at the UI
- Use `ofLocalizedDate` to respect the user's locale automatically

QUICK SUMMARY

- `format` — object → text
- `parse` — text → object
- `ofPattern("dd/MM/yyyy")`
- Case matters in patterns

Collections Utility Class

STATIC HELPERS FOR COLLECTIONS

`java.util.Collections` (with an **s**) is a utility class of static methods that operate on or return collections: sorting, searching, shuffling, counting, building read-only or thread-safe **wrappers**, and creating empty or singleton collections. Don't confuse it with the `Collection` interface.

NOTES

- Most methods modify the list **in place**
- `unmodifiableX` returns a read-only **view**, not a copy
- `List.copyOf` (Java 10) gives a true immutable copy
- `synchronizedX` wrappers lock every call

COMMON OPERATIONS

```
List<Integer> nums = new ArrayList<>(List.of(4, 1, 3, 1, 5));

Collections.sort(nums);                    // [1, 1, 3, 4, 5]
Collections.reverse(nums);                 // [5, 4, 3, 1, 1]
Collections.shuffle(nums);                 // random order
Collections.swap(nums, 0, 1);
Collections.max(nums);                     // 5
Collections.min(nums, Comparator.reverseOrder());   // 5
Collections.frequency(nums, 1);            // 2
Collections.rotate(nums, 2);               // shift right by 2
Collections.fill(nums, 0);                 // all zeros

List<String> blanks = Collections.nCopies(3, "-");    // [-, -, -]
List<String> empty = Collections.emptyList();
Set<String> one = Collections.singleton("only");

List<String> view = Collections.unmodifiableList(names);
view.add("x");                             // ✖ UnsupportedOperationException
names.add("x");                            // view now shows "x" too!

List<String> safe = Collections.synchronizedList(new ArrayList<>());
Collections.addAll(safe, "a", "b", "c");
```

NOTES

- → `max` / `min` accept an optional comparator
- → `rotate` moves elements by a distance, wrapping around
- → Unmodifiable views reflect changes in the backing list

METHOD GROUPS

GROUP	METHODS
`Ordering`	sort, reverse, shuffle, swap, rotate
`Searching`	binarySearch, max, min, frequency
`Filling`	fill, nCopies, addAll, copy
`Wrappers`	unmodifiableX, synchronizedX, checkedX
`Factories`	emptyList, singletonList, emptyMap

READ-ONLY OPTIONS

APPROACH	COPY?	REFLECTS CHANGES?
`Collections.unmodifiableList(l)`	No (view)	Yes
`List.copyOf(l)`	Yes	No
`List.of(...)`	New	n/a
`stream().toList()`	New	n/a

⚠ COMMON MISTAKES

- Assuming `unmodifiableList` is immutable — the original can still change it
- Iterating a `synchronizedList` without holding its lock → `ConcurrentModificationException`
- `Collections.copy(dest, src)` requires `dest` to already be at least as long as `src`

✓ TIPS & BEST PRACTICES

- Prefer `List.of` / `List.copyOf` for immutable data
- Prefer `ConcurrentHashMap` / `CopyOnWriteArrayList` over synchronized wrappers
- Return `Collections.emptyList()` or `List.of()` instead of `null`

QUICK SUMMARY

- Static helpers for collections
- `sort` `reverse` `shuffle` `max` `frequency`
- `unmodifiable` — read-only view
- `synchronized` — locked wrapper

Arrays Utility Class

STATIC HELPERS FOR ARRAYS

`java.util.Arrays` provides static methods for working with arrays: printing, sorting, searching, filling, copying, comparing, hashing, and converting to lists and streams. Arrays have no useful methods of their own, so this class is where nearly all array work happens.

NOTES

- `Arrays.asList` returns a **fixed-size** list backed by the array
- Primitive arrays don't work with `asList` as you'd expect
- `equals` compares contents; `deepEquals` handles nested arrays

ARRAY OPERATIONS

```
int[] a = {5, 3, 8, 1};

Arrays.toString(a);                    // "[5, 3, 8, 1]"
Arrays.sort(a);                        // [1, 3, 5, 8]
Arrays.binarySearch(a, 5);             // 2
int[] big = Arrays.copyOf(a, 6);       // [1, 3, 5, 8, 0, 0]
int[] mid = Arrays.copyOfRange(a, 1, 3);// [3, 5]
Arrays.fill(big, 4, 6, -1);            // [1, 3, 5, 8, -1, -1]
Arrays.equals(a, new int[]{1, 3, 5, 8});// true

int sum = Arrays.stream(a).sum();      // 17

List<String> fixed = Arrays.asList("x", "y", "z");
fixed.set(0, "X");                     // ✔ allowed
fixed.add("w");                        // ✖ UnsupportedOperationException
List<String> growable = new ArrayList<>(Arrays.asList("x", "y"));

Integer[] boxed = {3, 1, 2};
Arrays.sort(boxed, Comparator.reverseOrder());   // [3, 2, 1]
Arrays.setAll(a, i -> i * i);          // [0, 1, 4, 9]
```

NOTES

- `copyOf` pads with defaults when growing
- `fill(arr, from, to, val)` — `to` is exclusive
- `asList` allows `set` but not `add` / `remove`
- `setAll` computes each element from its index

METHOD REFERENCE

METHOD	PURPOSE
`toString / deepToString`	Readable text
`sort / parallelSort`	Sort (in place)
`binarySearch`	Find in sorted array
`fill / setAll`	Assign values
`copyOf / copyOfRange`	New arrays
`equals / deepEquals`	Compare contents
`stream`	IntStream / Stream

ARRAY ↔ LIST

DIRECTION	CODE
`Array → fixed List`	Arrays.asList(arr)
`Array → mutable List`	new ArrayList<>(Arrays.asList(arr))
`Array → immutable List`	List.of(arr)
`List → array`	list.toArray(String[]::new)
`int[] → List<Integer>`	Arrays.stream(a).boxed().toList()

⚠ COMMON MISTAKES

- `Arrays.asList(intArray)` gives a `List<int[]>` with one element
- Adding to an `asList` result → `UnsupportedOperationException`
- Printing an array with `println(arr)` instead of `Arrays.toString`

✓ TIPS & BEST PRACTICES

- Use `deepToString` for 2D arrays
- Use `Arrays.stream` for sums, averages, and filtering
- Use `List.of(arr)` for an immutable list from an object array

QUICK SUMMARY

- `toString` `sort` `fill` `copyOf`
- `equals` — compare contents
- `asList` — fixed-size view
- `stream` — aggregate

Optional

A CONTAINER FOR MAYBE-ABSENT VALUES

`Optional<T>` wraps a value that may or may not be present, making "no result" explicit in a method's return type instead of returning `null`. It provides methods to transform the value, supply defaults, or throw when it's missing, so callers handle absence on purpose.

NOTES

- Designed mainly for **return types**
- Don't use it for fields, parameters, or collections
- `Optional.of(null)` throws; use `ofNullable`
- Primitive versions: `OptionalInt`, `OptionalDouble`

CREATING & CONSUMING

```
Optional<String> a = Optional.of("Ada");
Optional<String> b = Optional.ofNullable(map.get("missing"));   // empty
Optional<String> c = Optional.empty();

a.isPresent();                          // true
b.isEmpty();                            // true (Java 11)
a.get();                                // "Ada"  (avoid: throws if empty)

String name = b.orElse("Guest");                    // "Guest"
String lazy = b.orElseGet(() -> loadDefault());     // called only if empty
String must = a.orElseThrow();                      // throws if empty
String err = b.orElseThrow(() -> new NotFound("user"));

a.ifPresent(n -> System.out.println("Hi " + n));
b.ifPresentOrElse(System.out::println, () -> System.out.println("none"));

int len = a.map(String::length).orElse(0);          // 3
var upper = a.filter(n -> n.startsWith("A")).map(String::toUpperCase);
Optional<Address> addr = findUser(id).flatMap(User::address);
var nicks = users.stream().map(this::nick)
                 .flatMap(Optional::stream).toList();
```

NOTES

- → `orElse` always evaluates its argument; `orElseGet` only when needed
- → `map` transforms the value if present
- → `flatMap` avoids `Optional<Optional<T>>`
- → `Optional::stream` drops empties in a stream

METHODS

METHOD	EMPTY CASE
`orElse(v)`	Returns v
`orElseGet(supplier)`	Calls supplier
`orElseThrow()`	NoSuchElementException
`ifPresent(fn)`	Does nothing
`map(fn)`	Stays empty
`or(() -> other)`	Uses other Optional

DO / DON'T

DO	DON'T
`Return Optional<T> from finders`	Use Optional fields
`Chain map / filter / orElse`	Call get() without checking
`Return empty lists, not Optional<List>`	Pass Optional as a parameter
`Use OptionalInt for primitives`	Return null from an Optional method

⚠ COMMON MISTAKES

- `if (opt.isPresent()) opt.get()` — use `ifPresent`, `map`, or `orElse`
- `orElse(expensiveCall())` runs the call even when a value exists
- Returning `null` from a method declared to return `Optional`

✓ TIPS & BEST PRACTICES

- Make "not found" methods return `Optional<T>`
- End chains with `orElseThrow` when absence is a bug
- Stream APIs like `findFirst` and `max` already return `Optional`

QUICK SUMMARY

- Explicit "maybe a value"
- `ofNullable` — wrap nullable
- `orElse` `orElseGet` `orElseThrow`
- `map` / `flatMap` / `filter`

System Class

ACCESS TO THE RUNTIME ENVIRONMENT

`java.lang.System` exposes the standard streams, the clock, environment variables, JVM system properties, and a few low-level utilities. All members are `static`. It's how a program reads configuration from its environment, measures elapsed time, and exits with a status code.

NOTES

- `nanoTime` is for measuring durations, not the time of day
- Environment variables are read-only from Java
- `exit(0)` = success; non-zero = failure
- `System.gc()` is only a hint to the garbage collector

COMMON CALLS

```
long start = System.nanoTime();
doWork();
long ms = (System.nanoTime() - start) / 1_000_000;      // elapsed ms

long epochMs = System.currentTimeMillis();              // wall clock
String home = System.getProperty("user.home");          // /home/ada
String javaVer = System.getProperty("java.version");    // 21.0.4
String mode = System.getProperty("app.mode", "dev");    // -Dapp.mode=prod
String dbUrl = System.getenv("DATABASE_URL");           // null if unset
Map<String, String> env = System.getenv();

int[] src = {1, 2, 3, 4, 5};
int[] dst = new int[3];
System.arraycopy(src, 1, dst, 0, 3);                    // dst = [2, 3, 4]

String nl = System.lineSeparator();                     // "\n" or "\r\n"
System.identityHashCode(obj);

System.setOut(new PrintStream(new FileOutputStream("log.txt")));  // redirect
System.err.println("warning");
if (badArgs) System.exit(1);                            // stop with error code
```

NOTES

- `-Dkey=value` on the `java` command sets properties
- `getenv` returns `null` for unset variables
- `arraycopy(src, srcPos, dst, dstPos, length)`
- Avoid `exit` in libraries and servers

USEFUL SYSTEM PROPERTIES

KEY	EXAMPLE VALUE
`user.home`	/home/ada
`user.dir`	Current working directory
`java.version`	21.0.4
`os.name`	Linux / Windows 11
`file.separator`	/ or \
`line.separator`	\n or \r\n

TIMING

METHOD	MEASURES	USE
`nanoTime()`	Arbitrary origin, ns	Elapsed time
`currentTimeMillis()`	Epoch ms	Timestamps
`Instant.now()`	Epoch, ns precision	Modern timestamps

⚠ COMMON MISTAKES

- Measuring elapsed time with `currentTimeMillis` — clock changes skew it; use `nanoTime`
- Calling `getenv("X").trim()` without a null check
- Calling `System.exit` from deep inside library code

✓ TIPS & BEST PRACTICES

- Use `Objects.requireNonNullElse(System.getenv("PORT"), "8080")`
- Use JMH for real benchmarks, not ad-hoc `nanoTime`
- Prefer `Files` and `Path.of(System.getProperty("user.home"))` for user paths

QUICK SUMMARY

- `out` `err` `in` — streams
- `nanoTime` — durations
- `getenv` / `getProperty` — config
- `exit(code)` — terminate

Regex — Pattern & Matcher

REGULAR EXPRESSIONS

`java.util.regex.Pattern` compiles a regular expression once; a `Matcher` applies it to text to find matches, extract **capture groups**, and replace text. `String` has shortcuts (`matches`, `replaceAll`, `split`) that compile a new pattern every call. In Java string literals, every regex backslash must be **doubled**.

NOTES

- `matches()` must match the **entire** input; `find()` searches for the next occurrence
- Compile patterns once and reuse them (`static final`)
- Groups are numbered from 1; group 0 is the whole match
- `Pattern.quote(text)` escapes user input

USING PATTERN & MATCHER

```
static final Pattern DATE =
    Pattern.compile("(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})");

Matcher m = DATE.matcher("Due 2026-10-02, paid 2026-09-15");
while (m.find()) {
    System.out.println(m.group("month"));          // 10, then 09
}

DATE.matcher("2026-10-02").matches();               // true (whole string)
"a1b22c333".replaceAll("\\d+", "#");                 // a#b#c#
"Ada Lovelace".replaceAll("(\\w+) (\\w+)", "$2, $1"); // Lovelace, Ada
"a, b;c".split("[,;]\\s*");                          // [a, b, c]

Pattern email = Pattern.compile("^[\\w.+-]+@[\\w-]+\\.[\\w.]+$",
                                Pattern.CASE_INSENSITIVE);
email.matcher("ada@example.com").matches();          // true

List<String> words = Pattern.compile("\\b\\w{5,}\\b")
    .matcher("Regular expressions are handy")
    .results().map(MatchResult::group).toList();  // [Regular, expressions…]
```

NOTES

- → `(?<name>…)` — named group, read with `group("name")`
- → `$1`, `$2` refer to groups in replacements
- → `results()` (Java 9) streams all matches

SYNTAX CHEAT SHEET

TOKEN	MATCHES
`.`	Any char except newline
`\\d \\w \\s`	Digit, word char, whitespace
`[abc] / [^abc]`	One of / none of
`* + ?`	0+, 1+, 0 or 1
`{n,m}`	Between n and m
`^ $`	Start / end
`( ) |`	Group, alternation
`\\b`	Word boundary

FLAGS

FLAG	INLINE	EFFECT
`CASE_INSENSITIVE`	(?i)	Ignore case
`MULTILINE`	(?m)	^ $ per line
`DOTALL`	(?s)	. matches newline
`COMMENTS`	(?x)	Allow spaces/comments

⚠ COMMON MISTAKES

- Single backslashes: `"\d"` doesn't compile; write `"\\d"`
- Using `matches()` when you meant `find()`
- Calling `String.replaceAll` in a hot loop — compile a `Pattern` once

✓ TIPS & BEST PRACTICES

- Use named groups for readability
- Escape user input with `Pattern.quote` before inserting it in a regex
- Test patterns at regex101.com (Java flavor)

QUICK SUMMARY

- `Pattern.compile` once
- `matcher.find()` / `matches()`
- `group(n)` / `group("name")`
- Double every backslash

Scanner — Advanced

BEYOND KEYBOARD INPUT

`Scanner` can read from **files, strings, and streams**, not just `System.in`. You can change how it splits input with `useDelimiter`, read typed values with a specific `Locale`, test the next token against a pattern, and stream all tokens with `tokens()`. Basics are on Sheet 17.

NOTES

- Default delimiter is whitespace (`\p{javaWhitespace}+`)
- `nextDouble` respects the Scanner's locale: `3,5` vs `3.5`
- `Scanner(File)` throws `FileNotFoundException`
- Scanners on files should be closed (try-with-resources)

ADVANCED SCANNER USAGE

```
try (Scanner sc = new Scanner(Path.of("scores.txt"))) {      // file
    while (sc.hasNextLine()) {
        String[] parts = sc.nextLine().split(":");
    }
}

Scanner csv = new Scanner("Ada,36,98.5").useDelimiter(",");   // string source
String name = csv.next();         // "Ada"
int age = csv.nextInt();          // 36
double score = csv.nextDouble(); // 98.5

Scanner de = new Scanner("3,5").useLocale(Locale.GERMANY);
de.nextDouble();                  // 3.5

Scanner s = new Scanner("id=42 name=Ada");
s.findInLine("id=(\\d+)");
s.match().group(1);               // "42"

new Scanner("x 1 y 2").tokens().toList();   // [x, 1, y, 2]
new Scanner("a1 b2").hasNext("[a-z]\\d");   // true

Scanner in = new Scanner(System.in);
int n = Integer.parseInt(in.nextLine().trim());   // avoids newline bug
```

NOTES

- → `Scanner(Path)` reads a file (UTF-8 by default in Java 18+)
- → `useDelimiter` takes a regex
- → `findInLine` + `match()` extracts groups
- → Reading every line then parsing avoids `nextInt` / `nextLine` mixups

SOURCES

CONSTRUCTOR	READS FROM
`Scanner(System.in)`	Keyboard
`Scanner(Path / File)`	File
`Scanner(String)`	Text in memory
`Scanner(InputStream, charset)`	Any byte stream
`Scanner(Readable)`	Any Reader

CONFIGURATION

METHOD	EFFECT
`useDelimiter(regex)`	Change token separator
`useLocale(locale)`	Number format
`useRadix(16)`	Parse hex ints
`reset()`	Restore defaults

⚠ COMMON MISTAKES

- Forgetting the locale: `nextDouble()` fails on "3.5" with a German default locale
- Not closing file Scanners
- Using `Scanner` for big files where `BufferedReader` would be far faster

✓ TIPS & BEST PRACTICES

- Set `Locale.ROOT` for machine-generated numeric data
- Use `tokens()` with streams for quick parsing
- Prefer `Files.lines` + `split` for large structured files

QUICK SUMMARY

- Reads files, strings, streams
- `useDelimiter` — custom splits
- `useLocale` — number formats
- `findInLine` / `tokens()`

Comparable & Comparator

DEFINING ORDER

Comparable gives a class a single **natural order** by implementing `compareTo` inside the class. **Comparator** defines an order *outside* the class, so one type can be sorted many ways. Both return a negative number, zero, or a positive number for less than, equal, or greater than.

NOTES

- `compareTo` should be consistent with `equals`
- `Comparator.comparing` builds comparators from key extractors
- Avoid subtraction (`a - b`); it overflows — use `Integer.compare`
- `TreeSet` / `TreeMap` treat compare-equal items as duplicates

BOTH APPROACHES

```
public record Employee(String name, String dept, int age, Double rating)
        implements Comparable<Employee> {
    @Override
    public int compareTo(Employee o) {            // natural order: by name
        return name.compareTo(o.name);
    }
}

Collections.sort(staff);                          // uses compareTo

Comparator<Employee> byAge = Comparator.comparingInt(Employee::age);
staff.sort(byAge);                                // youngest first
staff.sort(byAge.reversed());                     // oldest first

staff.sort(Comparator.comparing(Employee::dept)
        .thenComparing(Employee::age, Comparator.reverseOrder())
        .thenComparing(Employee::name));

staff.sort(Comparator.comparing(Employee::rating,
        Comparator.nullsLast(Comparator.naturalOrder())));   // null-safe

Optional<Employee> oldest = staff.stream().max(byAge);
```

NOTES

- → `comparingInt` avoids boxing
- → `thenComparing` breaks ties in order
- → `nullsLast` puts `null` keys at the end instead of throwing
- → Comparators also work with `max`, `min`, `sorted`, `TreeMap`

RETURN VALUE MEANING

RESULT	MEANING
`negative`	this / a comes first
`0`	Equal for ordering
`positive`	this / a comes after

COMPARATOR FACTORY METHODS

METHOD	PURPOSE
`comparing(key)`	Order by extracted key
`comparingInt / Double`	Primitive keys
`thenComparing(key)`	Tie-breaker
`reversed()`	Flip order
`naturalOrder / reverseOrder`	Built-in
`nullsFirst / nullsLast`	Handle nulls

⚠ COMMON MISTAKES

- `return a.age - b.age;` overflows for large values; use `Integer.compare`
- Comparators that never tie-break, so `TreeSet` drops "equal" elements
- Calling `.reversed()` in the middle of a chain and reversing everything before it

✓ TIPS & BEST PRACTICES

- Implement `Comparable` only when there's one obvious order
- Build comparators declaratively with `comparing` + `thenComparing`
- Store common comparators as `static final` constants

QUICK SUMMARY

- `Comparable` — natural order, in class
- `Comparator` — external, many orders
- -, 0, + return values
- `comparing().thenComparing()`

JUnit 5 — Basics

UNIT TESTING WITH JUNIT

JUnit is Java's standard unit-testing framework. JUnit 5 ("Jupiter") marks test methods with `@Test`, checks results with static **assertion** methods, and sets up shared state with lifecycle annotations. Build tools find tests in `src/test/java` and run them with `mvn test` or `./gradlew test`. JUnit 6 (2025) keeps the same Jupiter API.

NOTES

- Imports come from `org.junit.jupiter.api`
- Test classes and methods can be package-private
- A failed assertion stops that test only

A TEST CLASS

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    private Calculator calc;

    @BeforeEach
    void setUp() { calc = new Calculator(); }      // runs before every test

    @Test
    @DisplayName("adds two positive numbers")
    void add() {
        assertEquals(5, calc.add(2, 3));           // expected, actual
    }

    @Test
    void divideByZeroThrows() {
        var ex = assertThrows(ArithmeticException.class,
                              () -> calc.divide(1, 0));
        assertEquals("/ by zero", ex.getMessage());
    }

    @Test void doubles() { assertEquals(0.3, 0.1 + 0.2, 1e-9); }   // delta
}
```

NOTES

- → `assertEquals(expected, actual)` — order matters for messages
- → `assertThrows` returns the exception for further checks
- → `assertAll(...)` reports every failure, not just the first
- → Doubles need a tolerance (delta)

ASSERTIONS

METHOD	PASSES WHEN
`assertEquals(e, a)`	e equals a
`assertNotNull(x)`	x != null
`assertTrue / assertFalse`	Condition holds
`assertThrows(T.class, code)`	code throws T
`assertIterableEquals(e, a)`	Same elements, same order

LIFECYCLE ANNOTATIONS

ANNOTATION	RUNS
`@BeforeAll`	Once before all tests (static)
`@BeforeEach`	Before each test
`@AfterEach`	After each test
`@AfterAll`	Once after all tests (static)

⚠ COMMON MISTAKES

- Swapping expected and actual, which makes failure messages misleading
- Comparing doubles with `assertEquals(a, b)` without a delta
- Tests that depend on each other's order or shared static state

✓ TIPS & BEST PRACTICES

- One behavior per test; name tests after the behavior
- Arrange–Act–Assert structure keeps tests readable
- Use AssertJ for fluent assertions if your team prefers it

QUICK SUMMARY

- `@Test` — mark a test
- `assertEquals` `assertThrows` `assertAll`
- `@BeforeEach` — fresh setup
- `mvn test` / `gradlew test`

JUnit 5 — Advanced

DATA-DRIVEN AND ORGANIZED TESTS

JUnit 5 can run one test method with many inputs using **parameterized tests**, group related tests with `@Nested` classes, tag tests for selective runs, skip tests conditionally with **assumptions**, and inject helpers like temporary directories. Parameterized tests need the `junit-jupiter-params` artifact (included in `junit-jupiter`).

NOTES

- Each parameter set shows as a separate test in reports
- `@CsvSource` converts strings to parameter types automatically
- `@MethodSource` references a static method returning a stream

ADVANCED FEATURES

```
@ParameterizedTest @ValueSource(strings = {"racecar", "level"})
void palindromes(String w) { assertTrue(Strings.isPalindrome(w)); }

@ParameterizedTest(name = "{0} + {1} = {2}")
@CsvSource({"1, 2, 3", "10, -5, 5", "0, 0, 0"})
void add(int a, int b, int expected) {
    assertEquals(expected, calc.add(a, b));
}

@ParameterizedTest
@MethodSource("orders")
void totals(Order o, double exp) { assertEquals(exp, o.total(), 0.01); }
static Stream<Arguments> orders() {
    return Stream.of(Arguments.of(new Order(2, 5.0), 10.0),
                     Arguments.of(new Order(0, 9.0), 0.0));
}

@Nested class WhenEmpty { @Test void sizeIsZero() { /* … */ } }

@Test @Tag("slow")
void bigImport(@TempDir Path dir) {
    assumeTrue(System.getenv("CI") != null);   // skip unless on CI
    Files.writeString(dir.resolve("in.csv"), "a,b");
}
```

NOTES

- → `name` customizes how each case is displayed
- → `Arguments.of` packs several parameters
- → `@Nested` classes group tests by scenario
- → A failed assumption **skips** rather than fails

PARAMETER SOURCES

ANNOTATION	SUPPLIES
`@ValueSource`	One literal per run
`@CsvSource`	Rows of values
`@MethodSource`	Stream of Arguments
`@EnumSource`	Each enum constant

RUNNING SUBSETS

TOOL	COMMAND
`Maven`	mvn test -Dgroups=slow
`Gradle`	test { useJUnitPlatform { includeTags 'slow' } }
`One class`	mvn test -Dtest=CalculatorTest

⚠ COMMON MISTAKES

- Non-static `@MethodSource` methods (only allowed with `@TestInstance(PER_CLASS)`)
- Commas inside `@CsvSource` values without quoting them: `"'a, b', 2"`
- Using assumptions to hide genuinely failing tests

✓ TIPS & BEST PRACTICES

- Replace copy-pasted tests with one parameterized test
- Use `@Nested` to mirror "given/when" scenarios
- Tag slow or integration tests and run them separately

QUICK SUMMARY

- `@ParameterizedTest` + sources
- `@Nested` — grouping
- `@Tag` — selective runs
- `@TempDir` / assumptions

Mockito

FAKING DEPENDENCIES IN TESTS

Mockito creates **mock objects**: stand-ins for real dependencies (databases, HTTP clients, services) whose behavior you script with `when(...).thenReturn(...)`. After running the code under test, `verify(...)` checks how the mock was called. This isolates the unit you're testing from slow or unpredictable collaborators.

NOTES

- Artifact: `org.mockito:mockito-junit-jupiter` (includes core)
- Unstubbed methods return defaults: `null`, `0`, `false`, empty collections
- Mockito 5 can mock final classes by default
- Mock **interfaces you own**, not value objects

MOCKING A REPOSITORY

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {
    @Mock UserRepository repo;               // fake dependency
    @Mock EmailSender email;
    @InjectMocks UserService service;        // real object, mocks injected

    @Test
    void registersNewUser() {
        when(repo.existsByEmail("ada@x.com")).thenReturn(false);
        when(repo.save(any())).thenAnswer(inv -> inv.getArgument(0));

        User u = service.register("ada@x.com");

        assertEquals("ada@x.com", u.email());
        verify(email).sendWelcome("ada@x.com");          // called once
        verify(repo, never()).delete(any());
    }

    @Test
    void rejectsDuplicates() {
        when(repo.existsByEmail(anyString())).thenReturn(true);
        assertThrows(DuplicateException.class,
                     () -> service.register("a@x.com"));
        verifyNoInteractions(email);
    }
}
```

NOTES

- → `@InjectMocks` builds `UserService` with the mocks
- → `thenAnswer` computes the return value from the call
- → `verify(mock).method(args)` asserts a call happened
- → `never()`, `times(2)`, `atLeastOnce()` control counts

STUBBING

SYNTAX	EFFECT
`when(m.f()).thenReturn(v)`	Return v
`thenThrow(new X())`	Throw
`doThrow(e).when(m).voidMethod()`	Stub void methods
`thenAnswer(inv -> …)`	Compute result

MATCHERS & CAPTORS

TOOL	USE
`any()` / `any(T.class)`	Any argument
`eq(value)`	Exact, when mixing matchers
`ArgumentCaptor<T>`	Capture and inspect args

⚠ COMMON MISTAKES

- Mixing raw values and matchers: `f(1, any())` — use `f(eq(1), any())`
- Mocking the class under test instead of its dependencies
- Over-verifying every call, making tests break on harmless refactors

✓ TIPS & BEST PRACTICES

- Verify outcomes and important interactions only
- Use `ArgumentCaptor` to inspect complex arguments
- Use `spy` sparingly — it means partially real objects

QUICK SUMMARY

- `@Mock` — fake dependency
- `when().thenReturn()` — script
- `verify()` — assert calls
- `@InjectMocks` — wire it up

Logging — SLF4J & Log4j

STRUCTURED APPLICATION LOGS

SLF4J is a logging **facade**: your code calls its API, and a backend such as **Logback** or **Log4j 2** decides formatting and where logs go (console, files, JSON). Log levels let you control verbosity without changing code. Use logging instead of `System.out.println` for anything that runs in production.

NOTES

- Add `slf4j-api` plus exactly **one** backend binding
- `{}` placeholders avoid building strings for disabled levels
- Pass the exception **last** to log its stack trace
- The JDK also has `java.util.logging` and `System.Logger`

USING SLF4J

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class OrderService {
    private static final Logger log =
        LoggerFactory.getLogger(OrderService.class);

    public void place(Order o) {
        log.info("Placing order {} for {}", o.id(), o.customer());
        log.debug("Items: {}", o.items());          // cheap if DEBUG off
        try {
            charge(o);
        } catch (PaymentException e) {
            log.error("Payment failed for {}", o.id(), e);   // + stack trace
        }
        if (o.total() > 10_000) log.warn("Large order {}", o.id());
    }
}

// MDC: add context to every log line in this thread
MDC.put("requestId", requestId);
try { handle(); } finally { MDC.clear(); }
```

NOTES

- → One `static final` logger per class
- → Exception as the final argument → full stack trace
- → MDC values appear when the pattern includes `%X{requestId}`

LOGBACK.XML

```
<configuration>
  <appender name="OUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss} %-5level %logger{20} - %msg%n</pattern>
    </encoder>
  </appender>
  <logger name="com.acme" level="DEBUG"/>
  <root level="INFO"><appender-ref ref="OUT"/></root>
</configuration>
```

LEVELS (LOW → HIGH)

LEVEL	USE FOR
TRACE	Very detailed flow
DEBUG	Diagnostics
INFO	Normal milestones
WARN	Unexpected, recoverable
ERROR	Failures

⚠ COMMON MISTAKES

- String concatenation in log calls: `log.debug("x=" + x)` — use `{}`
- Logging an exception with `e.getMessage()` only, losing the stack trace
- Adding two SLF4J backends — SLF4J picks one and warns

✓ TIPS & BEST PRACTICES

- Log at boundaries: requests in, responses out, errors
- Never log passwords, tokens, or personal data
- Use JSON output in production for log search tools

QUICK SUMMARY

- SLF4J = API, Logback/Log4j = engine
- `LoggerFactory.getLogger(X.class)`
- `{}` placeholders
- Exception last for stack traces

HttpClient (Built-in)

HTTP WITHOUT DEPENDENCIES

`java.net.http.HttpClient` (Java 11+) is the JDK's modern HTTP client. It supports HTTP/1.1 and HTTP/2, synchronous and asynchronous requests, timeouts, redirects, and pluggable body handlers. Build an immutable `HttpRequest`, send it, and read the `HttpResponse`. No library needed.

NOTES

- Create one `HttpClient` and reuse it; it's thread-safe
- Non-2xx responses don't throw; check `statusCode()`
- `send` throws `IOException` and `InterruptedException`
- Use a JSON library (Sheets 108–109) for bodies

GET, POST, ASYNC

```
HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(5))
    .followRedirects(HttpClient.Redirect.NORMAL)
    .build();

String base = "https://api.example.com";
HttpRequest get = HttpRequest.newBuilder(URI.create(base + "/users/1"))
    .header("Accept", "application/json")
    .timeout(Duration.ofSeconds(10))
    .GET().build();

var res = client.send(get, HttpResponse.BodyHandlers.ofString());
if (res.statusCode() == 200) System.out.println(res.body());

HttpRequest post = HttpRequest.newBuilder(URI.create(base + "/users"))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString("{\"name\":\"Ada\"}"))
    .build();
client.send(post, HttpResponse.BodyHandlers.discarding());

client.sendAsync(get, HttpResponse.BodyHandlers.ofString())
      .thenApply(HttpResponse::body)
      .thenAccept(System.out::println)
      .join();                                    // wait (demo only)
```

NOTES

- → Builders produce immutable request objects
- → `BodyHandlers` decide how the response body is read
- → `sendAsync` returns a `CompletableFuture` (Sheet 114)

BODY HANDLERS

HANDLER	BODY TYPE
`ofString()`	String
`ofByteArray()`	byte[]
`ofFile(path)`	Path (saved to disk)
`ofLines()`	Stream<String>
`discarding()`	Ignored

REQUEST METHODS

BUILDER CALL	HTTP METHOD
`GET()`	GET
`POST(publisher)`	POST
`PUT(publisher)`	PUT
`DELETE()`	DELETE
`method("PATCH", p)`	Any other

⚠ COMMON MISTAKES

- Creating a new `HttpClient` per request — wasteful
- Assuming 404/500 throw an exception
- Forgetting the `Content-Type` header on POST bodies

✓ TIPS & BEST PRACTICES

- Set both connect and request timeouts
- Wrap the client in a small service class for testability
- Use `ofFile` to download large files without holding them in memory

QUICK SUMMARY

- `HttpClient` — reusable, thread-safe
- `HttpRequest.newBuilder(uri)`
- `send` / `sendAsync`
- Check `statusCode()` yourself

OkHttp

A POPULAR HTTP CLIENT LIBRARY

OkHttp (by Square) is a widely used HTTP client, especially on Android, where it underpins Retrofit. It offers connection pooling, transparent GZIP, caching, automatic retries, and **interceptors** for logging or adding headers. Requests are built with `Request.Builder`, executed synchronously with `execute()` or asynchronously with `enqueue()`.

NOTES

- Artifact: `com.squareup.okhttp3:okhttp`
- Reuse one `OkHttpClient` across the app
- Always close the `Response` (try-with-resources)
- `response.body().string()` can be read only **once**

REQUESTS WITH OKHTTP

```
OkHttpClient client = new OkHttpClient.Builder()
    .connectTimeout(Duration.ofSeconds(5))
    .addInterceptor(chain -> chain.proceed(chain.request().newBuilder()
        .header("Authorization", "Bearer " + token).build()))
    .build();

Request req = new Request.Builder().url(base + "/users/1").build();
try (Response res = client.newCall(req).execute()) {
    if (!res.isSuccessful()) throw new IOException("HTTP " + res.code());
    String json = res.body().string();
}

MediaType JSON = MediaType.get("application/json");
RequestBody body = RequestBody.create("{\"name\":\"Ada\"}", JSON);
Request post = new Request.Builder().url(base + "/users")
    .post(body).build();

client.newCall(post).enqueue(new Callback() {           // async
    public void onResponse(Call call, Response res) throws IOException {
        try (res) { System.out.println(res.code()); }
    }
    public void onFailure(Call call, IOException e) { e.printStackTrace(); }
});
```

NOTES

→ The interceptor adds an auth header to every request

→ `isSuccessful()` is true for 200–299

→ `enqueue` runs on OkHttp's thread pool

→ In OkHttp 4, `RequestBody.create(String, MediaType)` argument order matters

BUILDER METHODS

METHOD	PURPOSE
`url(String)`	Target URL
`header(k, v)`	Set header
`get() / post(body)`	HTTP method
`put(body) / delete()`	Other methods
`tag(obj)`	Attach metadata

OKHTTP VS JDK HTTPCLIENT

ASPECT	OKHTTP	JAVA.NET.HTTP
Dependency	Yes	None
Android	Standard	Not available
Interceptors	Built-in	Manual wrapping
Caching	Built-in	No

⚠ COMMON MISTAKES

- Not closing `Response` → leaked connections and "A connection was leaked" warnings
- Calling `body().string()` twice — the stream is consumed after the first call
- Creating a client per request, which defeats connection pooling

✓ TIPS & BEST PRACTICES

- Add `HttpLoggingInterceptor` (okhttp-logging) for debugging
- Use Retrofit on top of OkHttp for typed REST APIs
- Use `client.newBuilder()` to derive clients with different timeouts

QUICK SUMMARY

- `OkHttpClient` — reuse it
- `Request.Builder` → `newCall`
- `execute()` sync / `enqueue()` async
- Close every `Response`

Gson

JSON WITH GOOGLE'S GSON

Gson converts Java objects to JSON (`toJson`) and back (`fromJson`) with almost no setup. It reads fields directly via reflection, so classes don't need getters or annotations. `GsonBuilder` customizes naming, nulls, dates, and pretty printing. For generic types like `List<User>`, use `TypeToken`.

NOTES

- Artifact: `com.google.code.gson:gson`
- Records are supported since Gson 2.10
- `Gson` instances are thread-safe; create one and reuse it
- Null fields are **omitted** by default

SERIALIZE & DESERIALIZE

```
record User(String name, int age, List<String> roles) { }

Gson gson = new Gson();
String json = gson.toJson(new User("Ada", 36, List.of("admin")));
// {"name":"Ada","age":36,"roles":["admin"]}

User u = gson.fromJson(json, User.class);

Type listType = new TypeToken<List<User>>(){}.getType();
List<User> users = gson.fromJson("[{\"name\":\"Alan\",\"age\":41}]", listType);

Gson pretty = new GsonBuilder().setPrettyPrinting().serializeNulls()
    .setFieldNamingPolicy(FieldNamingPolicy.LOWER_CASE_WITH_UNDERSCORES)
    .create();

class Product {
    @SerializedName("product_id") String id;     // custom JSON key
    transient String cache;                        // skipped
}

JsonObject obj = JsonParser.parseString(json).getAsJsonObject();   // tree model
obj.get("name").getAsString();                    // "Ada"
obj.getAsJsonArray("roles").size();               // 1
```

NOTES

- → `TypeToken` captures the generic type erased at runtime
- → `transient` fields are excluded
- → The tree model (`JsonObject`) handles JSON without classes

CORE API

CALL	PURPOSE
`toJson(obj)`	Object → JSON string
`fromJson(str, Class)`	JSON → object
`fromJson(str, Type)`	JSON → generic type
`JsonParser.parseString(s)`	Tree model

GSONBUILDER OPTIONS

OPTION	EFFECT
`setPrettyPrinting()`	Indented output
`serializeNulls()`	Include null fields
`setFieldNamingPolicy(…)`	snake_case, etc.
`registerTypeAdapter(T, adapter)`	Custom (e.g., LocalDate)
`excludeFieldsWithoutExposeAnnotation()`	Only @Expose fields

⚠ COMMON MISTAKES

- `gson.fromJson(json, List.class)` returns a list of `LinkedTreeMap`, not your type
- `LocalDate` fields without a type adapter — Gson can't access java.time internals on modern JDKs
- Expecting numbers in `Map<String, Object>` to be ints — Gson parses them as `Double`

✓ TIPS & BEST PRACTICES

- Keep one configured `Gson` as a constant
- Register `TypeAdapter`s for `java.time` types
- Use Jackson (Sheet 109) if you're on Spring or need streaming/performance features

QUICK SUMMARY

- `toJson` / `fromJson`
- `TypeToken` for generics
- `@SerializedName` — rename keys
- `GsonBuilder` — configure

Jackson

HIGH-PERFORMANCE JSON

Jackson is the most widely used Java JSON library and the default in Spring Boot. Its `ObjectMapper` converts between JSON and Java objects (data binding), exposes a tree model (`JsonNode`), and offers streaming APIs. Annotations control naming, inclusion, and formats. Jackson 3 (2025) moved to `tools.jackson.*` packages; the API below is Jackson 2.

NOTES

- Artifact: `com.fasterxml.jackson.core:jackson-databind`
- `ObjectMapper` is thread-safe after configuration
- Unknown JSON properties **fail** by default
- Add `jackson-datatype-jsr310` for `java.time` types

DATA BINDING & TREE MODEL

```
ObjectMapper mapper = new ObjectMapper()
    .registerModule(new JavaTimeModule())
    .configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);

record User(String name, int age, LocalDate joined) { }

String json = mapper.writeValueAsString(new User("Ada", 36, today));
User u = mapper.readValue(json, User.class);
var list = mapper.readValue(arrayJson, new TypeReference<List<User>>() {});

class Product {
    @JsonProperty("product_id") private String id;
    @JsonIgnore private String internalNote;
    @JsonInclude(JsonInclude.Include.NON_NULL) private String color;
}

JsonNode root = mapper.readTree(json);
root.get("name").asText();                  // "Ada"
root.path("missing").asInt(0);              // 0 (path() never null)
```

NOTES

- → `TypeReference` captures generic types
- → Records work out of the box (2.12+)
- → `get()` returns null for missing fields; `path()` returns a missing node

OBJECTMAPPER METHODS

METHOD	PURPOSE
`writeValueAsString(o)`	Object → JSON
`readValue(src, T.class)`	JSON → object
`readValue(src, TypeReference)`	JSON → generic
`readTree(src)`	JSON → JsonNode

ANNOTATIONS

ANNOTATION	EFFECT
`@JsonProperty("x")`	Rename / include
`@JsonIgnore`	Skip field
`@JsonInclude(NON_NULL)`	Omit nulls
`@JsonFormat(pattern)`	Date/number format

⚠ COMMON MISTAKES

- Creating a new `ObjectMapper` per call — it's expensive to build
- Forgetting `JavaTimeModule` → errors on `LocalDate` fields
- Private fields with no getters or `@JsonProperty`, which Jackson can't see

✓ TIPS & BEST PRACTICES

- Configure one shared mapper (Spring Boot does this for you)
- Ignore unknown properties when consuming external APIs
- Use `JsonNode` for exploratory or partial parsing

QUICK SUMMARY

- `ObjectMapper` — central class
- `readValue` / `writeValueAsString`
- `TypeReference` for generics
- Annotations shape JSON

Lombok

GENERATED BOILERPLATE

Lombok is an annotation processor that generates boilerplate at compile time: getters, setters, constructors, `equals`/`hashCode`, `toString`, builders, and loggers. Your source stays short while the compiled class contains the full methods. IDEs need the Lombok plugin (built into IntelliJ) to see generated members.

NOTES

- Add as `compileOnly` + `annotationProcessor` (Gradle) or `provided` (Maven)
- Generated code doesn't appear in your source files
- Records (Sheet 72) cover many `@Value` use cases natively

BEFORE → AFTER

```
@Data                                   // getters, setters, equals,
public class Customer {                 // hashCode, toString, required-args ctor
    private Long id;
    private String name;
    private String email;
}

@Value                                  // immutable: private final fields,
@Builder                                // getters, all-args ctor, no setters
public class Address {
    String street;
    String city;
    @Builder.Default String country = "US";
}

Address a = Address.builder().street("1 Main").city("Austin").build();
a.getCity();                            // "Austin"

@Slf4j @RequiredArgsConstructor         // logger + ctor for final fields
public class OrderService { private final OrderRepository repo; }
```

NOTES

- → `@Data` = `@Getter @Setter @ToString @EqualsAndHashCode @RequiredArgsConstructor`
- → `@Value` makes the class final and its fields private final
- → `@RequiredArgsConstructor` enables constructor injection
- → `@Builder.Default` keeps a field's default in builders

COMMON ANNOTATIONS

ANNOTATION	GENERATES
`@Getter / @Setter`	Accessors
`@NoArgsConstructor`	No-arg constructor
`@AllArgsConstructor`	Constructor for all fields
`@Builder`	Fluent builder
`@Slf4j`	SLF4J logger field

⚠ COMMON MISTAKES

- `@Data` on JPA entities — generated `equals` / `hashCode` / `toString` can trigger lazy loading and loops
- Forgetting annotation-processor configuration, so builds fail with "cannot find symbol"
- `@EqualsAndHashCode` on a subclass without `callSuper = true`

✓ TIPS & BEST PRACTICES

- Prefer records for simple immutable data
- On entities, use `@Getter @Setter` and write `equals` by ID yourself
- Use `@Builder` for classes with many optional fields

QUICK SUMMARY

- Compile-time code generation
- `@Data` — mutable bean
- `@Value` — immutable class
- `@Builder` `@Slf4j`

OpenCSV

ROBUST CSV READING AND WRITING

OpenCSV correctly handles the parts of CSV that `split` gets wrong: quoted fields, embedded commas and quotes, and multi-line values. Read rows as `String[]` with `CSVReader`, write with `CSVWriter`, or map rows directly to annotated Java beans with `CsvToBean` and `StatefulBeanToCsv`.

NOTES

- Artifact: `com.opencsv:opencsv`
- Read methods throw `CsvValidationException` and `IOException`
- Bean binding requires a no-arg constructor (records aren't supported)
- Separators other than `,` are set with a `CSVParserBuilder`

READ, WRITE, BIND

```
try (CSVReader r = new CSVReaderBuilder(new FileReader("people.csv"))
        .withSkipLines(1).build()) {                    // skip header
    String[] row;
    while ((row = r.readNext()) != null) {
        System.out.println(row[0] + " is " + row[1]);  // quotes handled
    }
}

try (CSVWriter w = new CSVWriter(new FileWriter("out.csv"))) {
    w.writeNext(new String[]{"name", "note"});
    w.writeNext(new String[]{"Ada", "Said \"hi\", left"});   // auto-escaped
}

public class Person {                                   // bean mapping
    @CsvBindByName(column = "name") private String name;
    @CsvBindByName(column = "age")  private int age;
    // no-arg constructor, getters/setters
}
List<Person> people = new CsvToBeanBuilder<Person>(new FileReader("people.csv"))
    .withType(Person.class).build().parse();

try (Writer out = new FileWriter("people-out.csv")) {
    new StatefulBeanToCsvBuilder<Person>(out).build().write(people);
}
```

NOTES

- → `withSkipLines(1)` drops the header for array-based reading
- → `CSVWriter` quotes every field by default
- → Bean binding matches header names to annotated fields

CLASSES

CLASS	PURPOSE
`CSVReader`	Rows as String[]
`CSVWriter`	Write String[] rows
`CSVReaderBuilder`	Skip lines, custom parser
`CsvToBeanBuilder`	Rows → objects

CUSTOMIZING

NEED	CODE
`Semicolon separator`	new CSVParserBuilder().withSeparator(';').build()
`No quotes on output`	CSVWriter.NO_QUOTE_CHARACTER
`Bind by position`	@CsvBindByPosition(position = 0)

⚠ COMMON MISTAKES

- Using records or classes without no-arg constructors for bean binding
- Reading Excel files saved with `;` separators using the default parser
- Forgetting to skip the header when reading `String[]` rows

✓ TIPS & BEST PRACTICES

- Use `readAll()` only for small files; stream with `readNext()` otherwise
- Specify UTF-8 explicitly with `Files.newBufferedReader(path)`
- Basic CSV concepts are on Sheet 81

QUICK SUMMARY

- `CSVReader` / `CSVWriter`
- Handles quotes and commas
- `CsvToBean` — map to objects
- Builders customize parsing

Threads

RUNNING CODE CONCURRENTLY

A **thread** is an independent path of execution inside a program. You create one by passing a `Runnable` (or lambda) to `Thread` and calling `start()`, which runs the task alongside the calling thread. `join()` waits for a thread to finish. Threads share memory, which makes them powerful and also the source of **race conditions**.

NOTES

- Call `start()`, not `run()`; `run()` executes on the current thread
- A thread can be started only once
- The JVM exits when only **daemon** threads remain
- Prefer executors (Sheet 113) over creating threads by hand

CREATING & COORDINATING THREADS

```
Runnable task = () ->
    System.out.println(Thread.currentThread().getName() + " running");
Thread t1 = new Thread(task, "worker-1");
Thread t2 = Thread.ofPlatform().name("worker-2").start(task);   // Java 21
t1.start();
t1.join();                          // wait for t1
t2.join();

Thread sleeper = new Thread(() -> {
    try {
        Thread.sleep(5_000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();     // restore flag
        System.out.println("woken early");
    }
});
sleeper.start();
sleeper.interrupt();                // politely ask it to stop

int[] count = {0};                  // ✖ race condition demo
Runnable inc = () -> { for (int i = 0; i < 10_000; i++) count[0]++; };
// two threads running inc → total is usually < 20_000
```

NOTES

- → Output order between threads is unpredictable
- → `interrupt()` sets a flag; blocking calls throw `InterruptedException`
- → `count[0]++` is read-modify-write: not atomic
- → Fixes: `synchronized`, `AtomicInteger` (Sheet 115)

THREAD STATES

STATE	MEANING
`NEW`	Created, not started
`RUNNABLE`	Running or ready
`BLOCKED`	Waiting for a lock
`WAITING / TIMED_WAITING`	join, sleep, wait
`TERMINATED`	Finished

THREAD METHODS

METHOD	PURPOSE
`start()`	Begin execution
`join() / join(ms)`	Wait for completion
`sleep(ms)`	Pause current thread
`interrupt()`	Request stop
`setDaemon(true)`	Don't block JVM exit

⚠ COMMON MISTAKES

- Calling `run()` instead of `start()` — nothing runs in parallel
- Swallowing `InterruptedException` without restoring the interrupt flag
- Sharing mutable data between threads without synchronization

✓ TIPS & BEST PRACTICES

- Use `ExecutorService` or virtual threads instead of raw `Thread`
- Give threads meaningful names for debugging
- Prefer immutable data shared between threads

QUICK SUMMARY

- `new Thread(runnable).start()`
- `join()` — wait
- `interrupt()` — cooperative stop
- Shared state needs synchronization

ExecutorService

THREAD POOLS

An `ExecutorService` manages a **pool of threads** and a queue of tasks, so you submit work instead of creating threads yourself. `submit` returns a `Future` for retrieving the result later. Pools must be **shut down** when you're done. Since Java 19 `ExecutorService` is `AutoCloseable`, so try-with-resources waits and shuts it down for you.

NOTES

- `Callable<T>` returns a value and may throw; `Runnable` doesn't
- `future.get()` blocks until the result is ready
- Exceptions inside tasks surface as `ExecutionException` from `get()`
- Size CPU-bound pools near `Runtime.getRuntime().availableProcessors`

SUBMITTING TASKS

```
try (ExecutorService pool = Executors.newFixedThreadPool(4)) {
    Future<Integer> f = pool.submit(() -> expensiveCalc());   // Callable
    pool.submit(() -> System.out.println("fire and forget")); // Runnable

    int result = f.get();                          // blocks
    Integer maybe = f.get(2, TimeUnit.SECONDS);    // TimeoutException if slow

    List<Callable<String>> jobs = List.of(() -> fetch("a"), () -> fetch("b"));
    for (Future<String> r : pool.invokeAll(jobs)) // run all, wait for all
        System.out.println(r.get());
    String fastest = pool.invokeAny(jobs);         // first successful result
}                                                  // close(): shutdown + wait

ScheduledExecutorService sched = Executors.newScheduledThreadPool(1);
sched.scheduleAtFixedRate(() -> ping(), 0, 30, TimeUnit.SECONDS);
// later:
sched.shutdown();
if (!sched.awaitTermination(5, TimeUnit.SECONDS)) sched.shutdownNow();
```

NOTES

- → try-with-resources (Java 19+) calls `close()` = shutdown + await
- → `invokeAll` returns futures in the same order as tasks
- → `shutdownNow` interrupts running tasks
- → Scheduled pools run periodic jobs

EXECUTORS FACTORIES

FACTORY	POOL
`newFixedThreadPool(n)`	n threads, unbounded queue
`newCachedThreadPool()`	Grows as needed
`newSingleThreadExecutor()`	One thread, ordered
`newScheduledThreadPool(n)`	Delayed / periodic
`newVirtualThreadPerTaskExecutor()`	One virtual thread per task (21+)

FUTURE METHODS

METHOD	PURPOSE
`get()`	Wait for result
`get(t, unit)`	Wait with timeout
`isDone()`	Finished?
`cancel(true)`	Cancel / interrupt

⚠ COMMON MISTAKES

- Never calling `shutdown()` — the JVM may not exit
- Ignoring the `Future` of a task that throws, so the exception vanishes
- Using `newCachedThreadPool` for unbounded bursts of blocking work → thousands of threads

✓ TIPS & BEST PRACTICES

- Use try-with-resources with executors on Java 19+
- Prefer `CompletableFuture` (Sheet 114) for chaining async steps
- Name threads with a custom `ThreadFactory` for clearer logs

QUICK SUMMARY

- Pool + task queue
- `submit` → `Future`
- `invokeAll` / `invokeAny`
- Always shut down

CompletableFuture

COMPOSABLE ASYNC PIPELINES

`CompletableFuture<T>` represents a result that will be available later and lets you **chain** what happens next without blocking: transform it, run another async step, combine several, or recover from errors. Tasks run on the common `ForkJoinPool` unless you pass your own executor.

NOTES

- `thenApply` — sync transform; `thenCompose` — async next step
- `join()` throws unchecked `CompletionException`; `get()` throws checked
- Pass an executor for blocking I/O work
- Non-`Async` callbacks run on whichever thread completed the stage

BUILDING A PIPELINE

```
ExecutorService io = Executors.newFixedThreadPool(8);

var userF = CompletableFuture.supplyAsync(() -> loadUser(id), io);
var ordersF = CompletableFuture.supplyAsync(() -> loadOrders(id), io);

CompletableFuture<String> summary = userF
    .thenCombine(ordersF, (u, os) -> u.name() + ": " + os.size() + " orders")
    .thenApply(String::toUpperCase)
    .exceptionally(ex -> "unavailable (" + ex.getMessage() + ")");

summary.thenAccept(System.out::println);          // non-blocking callback
String s = summary.join();                        // block only at the edge

CompletableFuture<Profile> p = userF.thenCompose(u -> fetchProfileAsync(u));

List<CompletableFuture<Price>> quotes = shops.stream()
    .map(sh -> CompletableFuture.supplyAsync(() -> sh.quote(item), io))
    .toList();
CompletableFuture.allOf(quotes.toArray(CompletableFuture[]::new)).join();
Price best = quotes.stream().map(CompletableFuture::join)
                   .min(Comparator.comparing(Price::amount)).orElseThrow();
```

NOTES

- → `thenCombine` merges two independent futures
- → `exceptionally` provides a fallback value
- → `thenCompose` flattens a future-returning function
- → `allOf` waits for many; then `join` each result

CHAINING METHODS

METHOD	INPUT → OUTPUT
`thenApply(fn)`	T → U
`thenAccept(fn)`	T → void
`thenCompose(fn)`	T → CompletableFuture<U>
`thenCombine(other, fn)`	(T, U) → V
`exceptionally(fn)`	Throwable → T
`handle(fn)`	(T, Throwable) → U

STARTING & COMBINING

METHOD	PURPOSE
`supplyAsync(supplier, ex)`	Async with result
`runAsync(runnable)`	Async, no result
`allOf(fs…)`	All complete
`anyOf(fs…)`	First completes
`orTimeout(n, unit)`	Fail after timeout

⚠ COMMON MISTAKES

- Calling `join()` right after `supplyAsync`, which makes it synchronous
- Running blocking I/O on the common pool, starving other tasks
- Using `thenApply` with a function that returns a future, producing `CF<CF<T>>`

✓ TIPS & BEST PRACTICES

- Block (`join`) only at the outermost layer
- Add `orTimeout` to remote calls
- With virtual threads (Sheet 116), plain blocking code is often simpler

QUICK SUMMARY

- `supplyAsync` — start
- `thenApply` / `thenCompose` — chain
- `thenCombine` / `allOf` — combine
- `exceptionally` — recover

Synchronization & Locks

SAFE ACCESS TO SHARED STATE

When threads share mutable data, you must coordinate access. `synchronized` lets only one thread at a time run a block guarded by the same lock, and guarantees **visibility** of changes. `ReentrantLock` adds timeouts and fairness. **Atomic** classes and **concurrent collections** often avoid explicit locking entirely.

NOTES

- `volatile` guarantees visibility, **not** atomicity
- Always unlock a `Lock` in `finally`
- Deadlock: two threads each waiting for the other's lock
- Prefer higher-level tools over `wait` / `notify`

FOUR WAYS TO PROTECT A COUNTER

```
class SyncCounter {                                  // 1. synchronized
    private int n;
    public synchronized void inc() { n++; }
    public synchronized int get() { return n; }
}

class LockCounter {                                  // 2. explicit lock
    private final ReentrantLock lock = new ReentrantLock();
    private int n;
    public void inc() {
        lock.lock();
        try { n++; } finally { lock.unlock(); }
    }
}

AtomicInteger hits = new AtomicInteger();            // 3. atomic
hits.incrementAndGet();
hits.updateAndGet(x -> Math.min(x + 5, 100));

ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();   // 4.
counts.merge("page", 1, Integer::sum);               // atomic per key

private volatile boolean running = true;             // visibility flag
public void stop() { running = false; }
```

NOTES

- `synchronized` methods lock on `this`
- `lock` / `unlock` must be balanced with `try/finally`
- `merge` on `ConcurrentHashMap` is atomic
- `volatile` is right for a simple stop flag

TOOL SELECTION

NEED	USE
Simple critical section	synchronized
tryLock / timeout / fairness	ReentrantLock
Many readers, few writers	ReentrantReadWriteLock
Single numeric counter	AtomicInteger / LongAdder
Shared map	ConcurrentHashMap
Producer-consumer	BlockingQueue

DEADLOCK PREVENTION

RULE	WHY
Lock in a consistent order	No circular wait
Keep critical sections short	Less contention
Use tryLock with timeout	Back off instead of hanging
Don't call unknown code while locked	Avoid hidden locks

⚠ COMMON MISTAKES

- Assuming `volatile int x; x++` is thread-safe — it isn't
- Synchronizing reads but not writes (or vice versa)
- Synchronizing on a mutable or public object like a `String` literal

✓ TIPS & BEST PRACTICES

- Prefer immutability and confinement over locks
- Use `java.util.concurrent` collections instead of `Collections.synchronizedX`
- Virtual threads (Java 21–23) pinned on `synchronized`; use `ReentrantLock` there or Java 24+

QUICK SUMMARY

- `synchronized` — mutual exclusion
- `ReentrantLock` — flexible lock
- `Atomic*` — lock-free counters
- `volatile` — visibility only

Virtual Threads (Java 21)

MILLIONS OF CHEAP THREADS

Virtual threads (final in Java 21) are lightweight threads managed by the JVM rather than the OS. When one blocks on I/O, the JVM parks it and frees the underlying **carrier** thread for other work. This lets plain, blocking, one-thread-per-request code scale to hundreds of thousands of concurrent tasks without async callbacks.

NOTES

- Best for **I/O-bound** work: HTTP calls, database queries, files
- No benefit for CPU-bound work; use a fixed platform pool there
- Don't pool virtual threads; create one per task
- Java 24+ removes most `synchronized` pinning

USING VIRTUAL THREADS

```
Thread vt = Thread.ofVirtual().name("vt-1").start(() -> handle(request));
vt.join();

Thread.startVirtualThread(() -> System.out.println("hello"));

try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<String>> pages = urls.stream()
        .map(u -> exec.submit(() -> fetch(u)))   // 10,000 tasks is fine
        .toList();
    for (Future<String> p : pages) process(p.get());
}                                                 // waits for all

Thread.currentThread().isVirtual();              // true inside a virtual thread

Semaphore dbLimit = new Semaphore(20);           // limit scarce resources
Callable<Row> job = () -> {
    dbLimit.acquire();
    try { return queryDb(); } finally { dbLimit.release(); }
};
```

NOTES

- → Each task gets its own virtual thread
- → Blocking `fetch` doesn't tie up an OS thread
- → Use a `Semaphore` to cap access to limited resources like DB connections
- → Thread-locals work but get costly with millions of threads

PLATFORM VS VIRTUAL

ASPECT	PLATFORM THREAD	VIRTUAL THREAD
Backed by	OS thread	JVM-scheduled
Memory	~1 MB stack	~KB, grows as needed
Creation cost	High	Very low
Pooling	Yes	No
Good for	CPU-bound	Blocking I/O

PINNING (BLOCKS THE CARRIER)

CAUSE	STATUS
synchronized + blocking (≤ Java 23)	Pins; use ReentrantLock
synchronized (Java 24+)	Fixed (JEP 491)
Native method / JNI	Pins
Object.wait()	Fixed in 24+

⚠ COMMON MISTAKES

- Putting virtual threads in a fixed-size pool, which removes the benefit
- Expecting CPU-heavy code to run faster
- Unbounded fan-out hammering a database — limit with a `Semaphore`

✓ TIPS & BEST PRACTICES

- Switch servers (Tomcat, Jetty, Spring Boot 3.2+) to virtual threads with a config flag
- Keep code blocking and simple; skip reactive complexity for I/O workloads
- Watch for pinning with `-Djdk.tracePinnedThreads` (older JDKs) or JFR

QUICK SUMMARY

- Lightweight JVM threads
- One per task, never pooled
- Ideal for blocking I/O
- `newVirtualThreadPerTaskExecutor()`

JDBC Basics

TALKING TO RELATIONAL DATABASES

JDBC (Java Database Connectivity) is the standard API in `java.sql` for running SQL against relational databases. A vendor **driver** (PostgreSQL, MySQL, SQLite, H2…) on the classpath implements it. You open a `Connection` with a JDBC URL, create statements, execute SQL, and read rows from a `ResultSet`.

NOTES

- Drivers register automatically (JDBC 4+); no `Class.forName` needed
- Connections, statements, and result sets must be **closed**
- `ResultSet` column indexes start at **1**
- Use `PreparedStatement` for any user input (Sheet 119)

JDBC URLS

DATABASE	URL FORMAT
PostgreSQL	jdbc:postgresql://localhost:5432/shop
MySQL	jdbc:mysql://localhost:3306/shop
SQLite	jdbc:sqlite:shop.db
H2 (in-memory)	jdbc:h2:mem:test

CONNECT, QUERY, READ

```
String url = "jdbc:postgresql://localhost:5432/shop";

try (Connection conn = DriverManager.getConnection(url, "app", "secret");
     Statement st = conn.createStatement();
     ResultSet rs = st.executeQuery("SELECT id, name, price FROM product")) {

    while (rs.next()) {                       // move to next row
        long id = rs.getLong("id");
        String name = rs.getString("name");
        BigDecimal price = rs.getBigDecimal(3);   // by index (1-based)
    }
} catch (SQLException e) {
    System.err.println(e.getSQLState() + ": " + e.getMessage());
}
```

NOTES

- → Resources close in reverse order: `rs`, `st`, `conn`
- → `rs.next()` returns `false` after the last row
- → Read by column name or 1-based index
- → `getSQLState` gives a portable error code

CORE INTERFACES

INTERFACE	ROLE
DriverManager / DataSource	Open connections
Connection	Session with the database
Statement	Run static SQL
PreparedStatement	Run parameterized SQL
ResultSet	Rows returned by a query

EXECUTE METHODS

METHOD	RETURNS	FOR
executeQuery(sql)	ResultSet	SELECT
executeUpdate(sql)	int (rows)	INSERT / UPDATE / DELETE / DDL
executeBatch()	int[]	Batched statements

⚠ COMMON MISTAKES

- Building SQL with string concatenation of user input → SQL injection
- Forgetting to close connections → pool exhaustion or "too many connections"
- Calling `rs.getString(0)` — indexes start at 1

✓ TIPS & BEST PRACTICES

- Use a connection pool in real apps (Sheet 120)
- Keep credentials in environment variables, not code
- Use H2 or Testcontainers for database tests

QUICK SUMMARY

- Driver + JDBC URL
- `Connection` → `Statement` → `ResultSet`
- `while (rs.next())` — iterate
- Close everything (try-with-resources)

JDBC CRUD Operations

CREATE, READ, UPDATE, DELETE

CRUD maps directly to SQL `INSERT`, `SELECT`, `UPDATE`, and `DELETE`. In JDBC, writes use `executeUpdate`, which returns the number of affected rows. Group related writes in a **transaction** by turning off auto-commit, then `commit()` on success or `rollback()` on failure. Batching sends many statements in one round trip.

NOTES

- Auto-commit is **on** by default: each statement commits immediately
- Request generated IDs with `RETURN_GENERATED_KEYS`

CRUD + TRANSACTION

```
String ins = "INSERT INTO product(name, price) VALUES (?, ?)";
try (var ps = conn.prepareStatement(ins, Statement.RETURN_GENERATED_KEYS)) {
    ps.setString(1, "Lamp");
    ps.setBigDecimal(2, new BigDecimal("29.99"));
    ps.executeUpdate();                                  // 1 row
    try (ResultSet keys = ps.getGeneratedKeys()) {
        long id = keys.next() ? keys.getLong(1) : -1;
    }
}

String upd = "UPDATE product SET price = ? WHERE id = ?";
try (var ps = conn.prepareStatement(upd)) {
    ps.setBigDecimal(1, new BigDecimal("24.99"));
    ps.setLong(2, 7);
    int rows = ps.executeUpdate();                       // 0 → not found
}

conn.setAutoCommit(false);                               // transaction
String sub = "UPDATE acct SET bal = bal - ? WHERE id = ?";
String add = "UPDATE acct SET bal = bal + ? WHERE id = ?";
try (var debit = conn.prepareStatement(sub);
     var credit = conn.prepareStatement(add)) {
    debit.setInt(1, 100);  debit.setLong(2, 1);  debit.executeUpdate();
    credit.setInt(1, 100); credit.setLong(2, 2); credit.executeUpdate();
    conn.commit();                                       // both or neither
} catch (SQLException e) { conn.rollback(); throw e; }
finally { conn.setAutoCommit(true); }
```

NOTES

- → Generated keys come back as a small `ResultSet`
- → `commit` makes both updates permanent together
- → `rollback` undoes everything since the last commit

SCHEMA

```
CREATE TABLE product (
  id    BIGSERIAL PRIMARY KEY,
  name  VARCHAR(100) NOT NULL,
  price NUMERIC(10,2) NOT NULL,
  stock INT DEFAULT 0
);
```

CRUD → JDBC

OPERATION	SQL	JDBC CALL
Create	INSERT	executeUpdate + getGeneratedKeys
Read	SELECT	executeQuery
Update	UPDATE	executeUpdate
Delete	DELETE	executeUpdate

⚠ COMMON MISTAKES

- Forgetting `commit()` with auto-commit off — changes vanish when the connection closes
- Not rolling back after a failed statement in a transaction
- Inserting thousands of rows one round trip at a time instead of batching

✓ TIPS & BEST PRACTICES

- Wrap transactions in a helper that commits or rolls back automatically
- Batch large inserts: `ps.addBatch()` in a loop, then `executeBatch()`
- Use `UPDATE … RETURNING` (PostgreSQL) to get updated values in one call

QUICK SUMMARY

- `executeUpdate` — writes, returns row count
- `executeQuery` — reads
- `setAutoCommit(false)` + `commit` / `rollback`
- Batch for bulk writes

PreparedStatement

SAFE, PARAMETERIZED SQL

A `PreparedStatement` is SQL with ? **placeholders** whose values are sent separately from the query text. The database never interprets those values as SQL, which **prevents SQL injection**. Statements are also precompiled, so re-executing with new values is faster. Use one for every query that includes variable data.

NOTES

- Parameter indexes start at **1**
- Placeholders replace **values only**, not table or column names
- `setNull(i, Types.X)` for SQL NULL

INJECTION VS PARAMETERS

```
String user = request.getParameter("user");          // e.g. "x' OR '1'='1"

// ✖ Vulnerable: input becomes part of the SQL
String bad = "SELECT * FROM users WHERE name = '" + user + "'";

// ✔ Safe: input is only ever a value
String sql = "SELECT id, email FROM users WHERE name = ? AND active = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, user);
    ps.setBoolean(2, true);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) System.out.println(rs.getString("email"));
    }
}

try (var ps = conn.prepareStatement("INSERT INTO log(msg, at) VALUES (?,?)")) {
    for (String m : messages) {
        ps.setString(1, m);
        ps.setTimestamp(2, Timestamp.from(Instant.now()));
        ps.addBatch();                                // queue
    }
    int[] counts = ps.executeBatch();                 // one round trip
}
```

NOTES

- → The malicious string is treated as a plain name
- → Each `?` is filled by position
- → `addBatch` + `executeBatch` inserts many rows efficiently

SETTER METHODS

JAVA TYPE	SETTER	SQL TYPE
`String`	setString	VARCHAR / TEXT
`int / long`	setInt / setLong	INTEGER / BIGINT
`BigDecimal`	setBigDecimal	NUMERIC
`LocalDate`	setObject(i, date)	DATE
`null`	setNull(i, Types.VARCHAR)	NULL

IN CLAUSE WITH N VALUES

```
List<Long> ids = List.of(3L, 7L, 9L);
String qs = String.join(",",
    Collections.nCopies(ids.size(), "?"));
var ps = conn.prepareStatement(
    "SELECT * FROM product WHERE id IN (" + qs + ")");
for (int i = 0; i < ids.size(); i++)
    ps.setLong(i + 1, ids.get(i));
```

⚠ COMMON MISTAKES

- Concatenating user input into SQL, even "just for a quick test"
- Trying to bind a table or column name with `?` — validate against an allow-list instead
- Using `setInt(i, value)` with a null `Integer` → NPE; use `setObject` or `setNull`

✓ TIPS & BEST PRACTICES

- Use `setObject` for `java.time` types (JDBC 4.2+)
- Build `IN (?, ?, ?)` dynamically, never by concatenating values
- Libraries like JDBI or jOOQ reduce boilerplate while staying safe

QUICK SUMMARY

- `?` placeholders + `setX(i, v)`
- Stops SQL injection
- Values only, not identifiers
- `addBatch` / `executeBatch`

Connection Pooling (HikariCP)

REUSING DATABASE CONNECTIONS

Opening a database connection is slow (network handshake, authentication). A **connection pool** keeps a set of open connections and lends them out; calling `close()` returns the connection to the pool instead of closing it. **HikariCP** is the fastest, most common pool and the default in Spring Boot.

NOTES

- Artifact: `com.zaxxer:HikariCP`
- Code depends on `javax.sql.DataSource`, not on Hikari directly
- Create **one** pool per database for the whole application
- Close the pool on application shutdown

CONFIGURING HIKARICP

```
HikariConfig cfg = new HikariConfig();
cfg.setJdbcUrl("jdbc:postgresql://localhost:5432/shop");
cfg.setUsername(System.getenv("DB_USER"));
cfg.setPassword(System.getenv("DB_PASS"));
cfg.setMaximumPoolSize(10);
cfg.setMinimumIdle(2);
cfg.setConnectionTimeout(5_000);          // ms to wait for a free connection
cfg.setLeakDetectionThreshold(30_000);    // warn if held > 30 s
cfg.setPoolName("shop-pool");

HikariDataSource ds = new HikariDataSource(cfg);   // app-wide singleton

try (Connection conn = ds.getConnection();          // borrow
     var ps = conn.prepareStatement("SELECT count(*) FROM product");
     var rs = ps.executeQuery()) {
    rs.next();
    System.out.println(rs.getLong(1));
}                                                   // close() = return to pool

Runtime.getRuntime().addShutdownHook(new Thread(ds::close));
```

NOTES

- → `getConnection()` blocks until a connection is free or the timeout passes
- → Leak detection logs a stack trace for unreturned connections
- → `ds.close()` shuts down the whole pool

KEY SETTINGS

PROPERTY	DEFAULT	MEANING
`maximumPoolSize`	10	Max connections
`minimumIdle`	= max	Idle connections kept
`connectionTimeout`	30 s	Wait for a connection
`idleTimeout`	10 min	Retire idle connections
`maxLifetime`	30 min	Recycle connections

SIZING GUIDANCE

SITUATION	GUIDANCE
`Typical web app`	Start small (~10)
`Rule of thumb`	cores × 2 + effective disks
`Many app instances`	Total across instances ≤ DB limit
`Virtual threads`	Pool size still limits DB concurrency

⚠ COMMON MISTAKES

- Creating a new `HikariDataSource` per request
- Forgetting to close borrowed connections — the pool runs dry and requests time out
- Setting the pool huge "for performance"; databases slow down with too many connections

✓ TIPS & BEST PRACTICES

- Load settings from a properties file or environment
- Enable leak detection in development
- Expose pool metrics (Micrometer) to see waits and usage

QUICK SUMMARY

- Pool = reuse open connections
- HikariCP — fast default
- `ds.getConnection()` / `close()` returns it
- One pool per database

Hibernate & JPA Basics

OBJECT-RELATIONAL MAPPING

JPA (Jakarta Persistence) is the standard API for mapping Java classes to database tables; **Hibernate** is its most popular implementation. You annotate classes as **entities**, and an `EntityManager` saves, loads, and tracks them, generating SQL for you. Changes to managed entities are written automatically when the transaction commits.

NOTES

- Packages: `jakarta.persistence.*` (older code: `javax.persistence`)
- Entities need a no-arg constructor and an `@Id`
- Records can't be entities (they're final and immutable)
- Spring Data JPA wraps all of this in repositories

MAPPING ENTITIES

```
@Entity
@Table(name = "customer")
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;
    @Column(unique = true) private String email;

    @OneToMany(mappedBy = "customer", cascade = CascadeType.ALL)
    private List<Order> orders = new ArrayList<>();

    protected Customer() { }                          // required by JPA
    public Customer(String n, String e) { name = n; email = e; }
}

@Entity
public class Order {
    @Id @GeneratedValue private Long id;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "customer_id")
    private Customer customer;
    private BigDecimal total;
}
```

NOTES

- → `@Id` + `@GeneratedValue` — database-generated primary key
- → `mappedBy` marks the owning side (`Order.customer`)
- → `LAZY` loads the customer only when accessed
- → `cascade` saves/deletes orders with their customer

CORE ANNOTATIONS

ANNOTATION	PURPOSE
`@Entity / @Table`	Class ↔ table
`@Id / @GeneratedValue`	Primary key
`@Column`	Column details
`@Enumerated(STRING)`	Store enum by name

RELATIONSHIPS

ANNOTATION	EXAMPLE
`@OneToOne`	User ↔ Profile
`@OneToMany`	Customer → Orders
`@ManyToOne`	Order → Customer
`@ManyToMany`	Student ↔ Course

⚠ COMMON MISTAKES

- `@Enumerated` default (`ORDINAL`) — reordering the enum corrupts data; use `STRING`
- EAGER fetching everywhere, which loads half the database per query
- Using `@Data` (Lombok) on entities — see Sheet 110

✓ TIPS & BEST PRACTICES

- Make `@ManyToOne` relationships `LAZY`
- Manage schemas with Flyway or Liquibase, not `hbm2ddl=update` in production
- Log SQL in development to see what Hibernate generates

QUICK SUMMARY

- JPA = API, Hibernate = engine
- `@Entity` + `@Id`
- Relationship annotations
- `EntityManager` tracks entities

Hibernate CRUD & JPQL

QUERYING WITH JPA

An `EntityManager` performs CRUD on entities inside a **transaction**: `persist` inserts, `find` loads by ID, changing a managed entity updates it, and `remove` deletes. For queries, **JPQL** is SQL-like but uses entity and field names, and the **Criteria API** (`CriteriaBuilder`) builds type-checked queries in code.

NOTES

- Managed entities are saved automatically at commit ("dirty checking")
- `merge` copies a detached object's state into a managed one
- JPQL uses class names: `FROM Customer`, not `FROM customer`
- `JOIN FETCH` solves most N+1 query problems

CRUD, JPQL, CRITERIA

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

Customer c = new Customer("Ada", "ada@x.com");
em.persist(c);                                  // INSERT on commit
Customer found = em.find(Customer.class, c.getId());
found.setName("Ada L.");                        // UPDATE (dirty checking)
em.remove(em.find(Customer.class, 99L));        // DELETE

List<Customer> big = em.createQuery(
        "SELECT DISTINCT c FROM Customer c JOIN FETCH c.orders o " +
        "WHERE o.total > :min",
        Customer.class)
    .setParameter("min", new BigDecimal("100"))
    .getResultList();

CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Customer> q = cb.createQuery(Customer.class);
Root<Customer> root = q.from(Customer.class);
q.select(root).where(cb.like(root.get("email"), "%@x.com"));
List<Customer> matches = em.createQuery(q).getResultList();

em.getTransaction().commit(); em.close();
```

NOTES

- → `:min` is a **named parameter** (safe like `?`)
- → `JOIN FETCH` loads orders in the same query
- → Criteria queries are verbose but composable at runtime

ENTITYMANAGER METHODS

METHOD	SQL
`persist(e)`	INSERT
`find(Class, id)`	SELECT by PK
`merge(e)`	UPDATE (detached)
`remove(e)`	DELETE
`createQuery(jpql)`	SELECT / UPDATE / DELETE

JPQL QUICK REFERENCE

CLAUSE	EXAMPLE
`Select`	SELECT c FROM Customer c
`Filter`	WHERE c.email LIKE :p
`Join`	JOIN c.orders o

⚠ COMMON MISTAKES

- N+1 queries: looping over customers and touching `getOrders()` for each
- `remove` on a detached entity fails; `find` it first
- Concatenating values into JPQL instead of using parameters

✓ TIPS & BEST PRACTICES

- Use `JOIN FETCH` or entity graphs for associations you'll read
- Use DTO projections (`SELECT new com.x.Dto(...)`) for read-only lists
- Spring Data derived queries (`findByEmail`) remove most boilerplate

QUICK SUMMARY

- `persist` `find` `merge` `remove`
- Managed changes auto-save
- JPQL — entity-based SQL
- Criteria API — query in code

Swing Basics

DESKTOP GUIS WITH SWING

Swing (`javax.swing`) is the long-standing GUI toolkit bundled with the JDK. A window is a `JFrame`; you add **components** like labels, buttons, and text fields to panels inside it. All Swing work must happen on the **Event Dispatch Thread (EDT)**, so start your UI with `SwingUtilities.invokeLater`.

NOTES

- Swing ships with every JDK; no dependency needed
- Swing is not thread-safe; touch components only on the EDT
- Long tasks on the EDT freeze the UI; use `SwingWorker`

A MINIMAL WINDOW

```
import javax.swing.*;

public class HelloSwing {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(HelloSwing::createUi);   // run on EDT
    }
    private static void createUi() {
        JFrame frame = new JFrame("Greeter");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JTextField name = new JTextField(12);
        JButton greet = new JButton("Greet");
        JLabel out = new JLabel("…");
        greet.addActionListener(e -> out.setText("Hi, " + name.getText()));

        JPanel panel = new JPanel();
        panel.add(new JLabel("Name:")); panel.add(name);
        panel.add(greet);               panel.add(out);
        frame.add(panel);
        frame.pack();                       // size to fit contents
        frame.setLocationRelativeTo(null);  // center on screen
        frame.setVisible(true);
    }
}
```

NOTES

- → `invokeLater` queues UI creation on the EDT
- → `EXIT_ON_CLOSE` ends the program when the window closes
- → `pack()` sizes the frame to its components' preferred sizes
- → Call `setVisible(true)` last

COMMON COMPONENTS

COMPONENT	PURPOSE
`JLabel`	Text or icon
`JButton`	Clickable button
`JTextField / JPasswordField`	One-line input
`JComboBox<T>`	Drop-down list
`JTable / JList`	Data display

TOP-LEVEL & UTILITIES

CLASS	USE
`JFrame`	Main window
`JDialog`	Dialog window
`JOptionPane`	Quick message / input boxes
`SwingWorker`	Background tasks

⚠ COMMON MISTAKES

- Building or updating components from `main` or a background thread
- Running slow work (network, DB) inside a button listener, freezing the window
- Calling `setVisible(true)` before adding components, causing layout glitches

✓ TIPS & BEST PRACTICES

- `JOptionPane.showMessageDialog(frame, "Saved")` for quick alerts
- Set a modern look: `UIManager.setLookAndFeel(UIManager.getSy`
- FlatLaf gives Swing a contemporary appearance

QUICK SUMMARY

- `JFrame` → `JPanel` → components
- Start with `invokeLater` (EDT)
- `pack()` then `setVisible(true)`
- `SwingWorker` for slow tasks

Swing Layout Managers

ARRANGING COMPONENTS

Swing positions components with **layout managers** rather than fixed coordinates, so windows resize sensibly. Each container has one layout manager, set with `setLayout`. Complex screens are built by **nesting** panels, each with the layout that suits its part of the window.

NOTES

- `JPanel` defaults to `FlowLayout`
- `JFrame`'s content pane defaults to `BorderLayout`
- `GridBagLayout` is powerful but verbose; MigLayout is a popular alternative
- Avoid `setLayout(null)` with absolute positions

NESTING LAYOUTS

```
JFrame f = new JFrame("Layouts");
f.setLayout(new BorderLayout(8, 8));                  // hgap, vgap

JPanel toolbar = new JPanel(new FlowLayout(FlowLayout.LEFT));
toolbar.add(new JButton("New"));
toolbar.add(new JButton("Open"));
f.add(toolbar, BorderLayout.NORTH);

JPanel form = new JPanel(new GridLayout(2, 2, 6, 6));   // rows, cols, gaps
form.add(new JLabel("Name"));  form.add(new JTextField());
form.add(new JLabel("Email")); form.add(new JTextField());
f.add(form, BorderLayout.CENTER);

JPanel side = new JPanel();
side.setLayout(new BoxLayout(side, BoxLayout.Y_AXIS));  // vertical stack
side.add(new JCheckBox("Subscribe"));
side.add(new JCheckBox("Remember me"));
f.add(side, BorderLayout.EAST);

f.pack();
f.setVisible(true);
```

NOTES

- → `BorderLayout` regions: NORTH, SOUTH, EAST, WEST, CENTER
- → CENTER takes all leftover space
- → `GridLayout` cells are equal size

LAYOUT MANAGERS

LAYOUT	ARRANGES	GOOD FOR
`FlowLayout`	Left-to-right, wraps	Button rows
`BorderLayout`	5 regions	Main window frame
`GridLayout`	Equal grid	Keypads, simple forms
`BoxLayout`	Single row or column	Stacks
`GridBagLayout`	Flexible grid	Complex forms

BORDERLAYOUT REGIONS

REGION	SIZE BEHAVIOR
`NORTH / SOUTH`	Preferred height, full width
`EAST / WEST`	Preferred width, remaining height
`CENTER`	Everything left over

⚠ COMMON MISTAKES

- Adding two components to the same `BorderLayout` region — only the last one shows
- Using `setSize` on components inside layouts; set preferred sizes or let layouts decide
- Absolute positioning that breaks on different fonts and screen scaling

✓ TIPS & BEST PRACTICES

- Sketch the window as nested rectangles first
- Add `BorderFactory.createEmptyBorder(8, 8, 8, 8)` for padding
- Use `CardLayout` to switch between screens

QUICK SUMMARY

- One layout manager per container
- Nest panels for complex UIs
- `BorderLayout` — frame skeleton
- `GridLayout` / `BoxLayout` / `FlowLayout`

Swing Event Handling

RESPONDING TO USER ACTIONS

Swing components fire **events** (clicks, key presses, text changes) and call the **listeners** you register. Most listeners are interfaces with one method, so lambdas work well; multi-method listeners have **adapter** classes you can subclass. Listeners run on the EDT, so slow work must move to a `SwingWorker`.

NOTES

- `ActionEvent` fires for buttons, menu items, and Enter in text fields
- Use `DocumentListener` to react to every text change
- Adapters (`MouseAdapter`, `KeyAdapter`) let you override only what you need
- Key bindings are more reliable than `KeyListener`

LISTENERS IN ACTION

```
JButton save = new JButton("Save");
save.addActionListener(e -> saveFile());              // lambda listener

JLabel area = new JLabel("Click me");
area.addMouseListener(new MouseAdapter() {            // override only one
    public void mouseClicked(MouseEvent e) { area.setText("Clicked"); }
});

JTextField search = new JTextField(20);
search.getDocument().addDocumentListener(new DocumentListener() {
    public void insertUpdate(DocumentEvent e)  { filter(search.getText()); }
    public void removeUpdate(DocumentEvent e)  { filter(search.getText()); }
    public void changedUpdate(DocumentEvent e) { }
});

JButton load = new JButton("Load");
load.addActionListener(e -> new SwingWorker<String, Void>() {
    protected String doInBackground() { return slowDownload(); } // worker
    protected void done() {                                      // EDT
        try { area.setText(get()); }
        catch (Exception ex) { area.setText("Failed"); }
    }
}.execute());
```

NOTES

- → `MouseAdapter` avoids writing five empty methods
- → `DocumentListener` catches typing, pasting, and deleting
- → `doInBackground` runs off the EDT; `done` returns to it
- → `get()` retrieves the background result

LISTENER TYPES

LISTENER	FIRES ON
`ActionListener`	Button click, Enter, menu item
`MouseListener / MouseAdapter`	Click, press, enter, exit
`KeyListener / KeyAdapter`	Key press / release
`DocumentListener`	Text content changes

KEY BINDING (PREFERRED)

STEP	CODE
`Map key`	im.put(KeyStroke.getKeyStroke("ctrl S"), "save")
`Map action`	am.put("save", saveAction)

⚠ COMMON MISTAKES

- Calling `slowDownload()` directly in `actionPerformed`, freezing the UI
- Updating labels from the `doInBackground` thread instead of `done` / `process`
- Adding the same listener repeatedly (e.g., inside a refresh method), so it fires several times

✓ TIPS & BEST PRACTICES

- Use `Action` objects to share behavior between buttons and menus
- Debounce search fields with a `javax.swing.Timer`
- Keep listeners thin; call well-named methods

QUICK SUMMARY

- Register listeners on components
- Lambdas for single-method listeners
- Adapters for multi-method ones
- `SwingWorker` for slow work

JavaFX Basics

MODERN JAVA DESKTOP UIS

JavaFX is Java's modern GUI toolkit, maintained as the open-source **OpenJFX** project and added as a dependency (it's no longer bundled with the JDK). An app extends `Application` and builds its UI in `start(Stage)`. The **Stage** is the window, the **Scene** holds the content, and the content is a tree of **nodes** called the scene graph.

NOTES

- Artifacts: `org.openjfx:javafx-controls` (+ `javafx-fxml`)
- Use the `javafx-maven-plugin` or `org.openjfx.javafxplugin` for Gradle
- UI updates must happen on the **JavaFX Application Thread**
- From other threads, use `Platform.runLater(...)`

HELLO JAVAFX

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class HelloFx extends Application {
    @Override
    public void start(Stage stage) {
        Label label = new Label("Hello, JavaFX!");
        Button btn = new Button("Click");
        btn.setOnAction(e -> label.setText("Clicked"));

        VBox root = new VBox(10, label, btn);     // spacing 10
        Scene scene = new Scene(root, 320, 200);  // width, height

        stage.setTitle("Hello"); stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) { launch(args); }
}
```

NOTES

- → `launch` starts the toolkit and calls `start` on the FX thread
- → `VBox(10, …)` stacks children with 10 px spacing
- → One `Stage` can switch between several `Scene`s

CORE CLASSES

CLASS	ROLE
`Application`	Entry point; init / start / stop
`Stage`	Window
`Scene`	Content of a window
`Parent / Node`	Scene-graph elements
`Platform`	runLater, exit

APPLICATION LIFECYCLE

METHOD	THREAD	WHEN
`init()`	Launcher	Before UI exists
`start(Stage)`	FX thread	Build and show UI
`stop()`	FX thread	App closing

⚠ COMMON MISTAKES

- Running a JavaFX app on a plain JDK without the OpenJFX dependency → "JavaFX runtime components are missing"
- Updating controls from a background thread → `IllegalStateException`
- Doing slow work inside `start`, delaying the first window

✓ TIPS & BEST PRACTICES

- Use `Task` / `Service` for background work (Sheet 129)
- Package with `jlink` / `jpackage` to ship a native installer
- Prefer FXML + controllers for larger screens (Sheets 131–132)

QUICK SUMMARY

- `extends Application` + `start(Stage)`
- Stage → Scene → nodes
- `launch(args)` in `main`
- FX thread for all UI changes

JavaFX Layouts

LAYOUT PANES

JavaFX arranges nodes with **layout panes**, each a `Parent` with its own positioning rules. Panes nest to build complex screens, and they resize their children automatically. Spacing, padding (`Insets`), alignment, and **grow priorities** control how extra space is shared.

NOTES

- Panes are nodes, so they nest freely
- `setPadding(new Insets(10))` adds inner space
- `HBox.setHgrow(node, Priority.ALWAYS)` lets a child stretch
- Constraints are set with static methods on the pane class

BUILDING A FORM SCREEN

```
BorderPane root = new BorderPane();
root.setPadding(new Insets(10));

HBox toolbar = new HBox(8, new Button("New"), new Button("Save"));
root.setTop(toolbar);

GridPane form = new GridPane();
form.setHgap(8); form.setVgap(8);
form.add(new Label("Name"), 0, 0);  form.add(new TextField(), 1, 0); // col,row
form.add(new Label("Email"), 0, 1);  form.add(new TextField(), 1, 1);
TextArea notes = new TextArea();
form.add(notes, 0, 2, 2, 1);                          // colspan 2
GridPane.setVgrow(notes, Priority.ALWAYS);
root.setCenter(form);

Label status = new Label("Ready");
HBox bar = new HBox(status);
bar.setAlignment(Pos.CENTER_RIGHT);
root.setBottom(bar);

StackPane badge = new StackPane(new Circle(20), new Label("3"));   // layered
```

NOTES

- → `GridPane.add(node, col, row, colspan, rowspan)`
- → `setVgrow` lets the notes area take extra height
- → `StackPane` layers children on top of each other
- → `Pos` constants set alignment

LAYOUT PANES

PANE	ARRANGES
`VBox` / `HBox`	Single column / row
`BorderPane`	Top, bottom, left, right, center
`GridPane`	Rows and columns
`StackPane`	Layers, centered
`FlowPane`	Wrapping row or column
`AnchorPane`	Pinned to edges

SPACING & SIZING

PROPERTY	EXAMPLE
`spacing`	new VBox(10)
`padding`	setPadding(new Insets(8, 12, 8, 12))
`alignment`	setAlignment(Pos.CENTER)
`grow`	HBox.setHgrow(n, Priority.ALWAYS)
`size limits`	setPrefWidth / setMaxWidth

⚠ COMMON MISTAKES

- Using absolute coordinates (`setLayoutX`) instead of panes
- Forgetting grow priorities, so fields don't stretch when the window resizes
- Deep nesting of `VBox` / `HBox` where one `GridPane` would be simpler

✓ TIPS & BEST PRACTICES

- Start with `BorderPane` as the window skeleton
- Use `GridPane` for forms
- Use Scene Builder to design layouts visually (generates FXML)

QUICK SUMMARY

- Panes position children
- `BorderPane` · `GridPane` · `VBox` / `HBox`
- `Insets` padding, `spacing` gaps
- Grow priorities share space

JavaFX Controls

BUILT-IN UI CONTROLS

JavaFX includes a rich set of **controls** in `javafx.scene.control`: buttons, text inputs, choices, lists, tables, sliders, date pickers, menus, and dialogs. Each control exposes **properties** (text, value, selection) that you can read, set, listen to, or bind (Sheet 130). Data-heavy controls use `ObservableLists`.

NOTES

- `ListView` / `ComboBox` / `TableView` display `ObservableList`s
- Selection lives in `getSelectionModel()`
- `TableColumn`s use cell value factories to read data
- `Alert` provides ready-made dialogs

COMMON CONTROLS

```
TextField name = new TextField();
name.setPromptText("Your name");
CheckBox agree = new CheckBox("I agree");
var size = new ComboBox<>(FXCollections.observableArrayList("S", "M", "L"));
size.setValue("M");
Slider volume = new Slider(0, 100, 50);
DatePicker date = new DatePicker(LocalDate.now());

var list = new ListView<>(FXCollections.observableArrayList("A", "B"));
list.getSelectionModel().selectedItemProperty()
    .addListener((obs, old, sel) -> System.out.println(sel));

record Person(String name, int age) { }
TableView<Person> table = new TableView<>();
var nameCol = new TableColumn<Person, String>("Name");
nameCol.setCellValueFactory(
    c -> new SimpleStringProperty(c.getValue().name()));
table.getColumns().add(nameCol);
table.setItems(FXCollections.observableArrayList(new Person("Ada", 36)));

Alert a = new Alert(Alert.AlertType.CONFIRMATION, "Delete file?");
a.showAndWait().filter(r -> r == ButtonType.OK).ifPresent(r -> delete());
```

NOTES

- `setPromptText` shows placeholder text
- Selection listeners get old and new values
- Cell value factories map row objects to column values
- `showAndWait` returns an `Optional<ButtonType>`

CONTROLS

CONTROL	KEY PROPERTY
`TextField / TextArea`	textProperty()
`CheckBox`	selectedProperty()
`ComboBox / ChoiceBox`	valueProperty()
`Slider`	valueProperty()
`DatePicker`	valueProperty()
`ListView / TableView`	items + selectionModel

DIALOGS

CLASS	USE
`Alert`	Info / warning / error / confirm
`TextInputDialog`	Ask for text
`ChoiceDialog`	Pick from options
`FileChooser`	Open / save files
`DirectoryChooser`	Pick a folder

⚠ COMMON MISTAKES

- Replacing a list's items instead of modifying its `ObservableList`, losing listeners
- Using `PropertyValueFactory("name")` with records — it looks for `getName()`; use a lambda
- Reading `getText()` once instead of binding or listening for changes

✓ TIPS & BEST PRACTICES

- Use `FXCollections.observableArrayList` for all list data
- Use `TableView` with records and lambda cell factories
- Use `Tooltip` and prompt text for discoverability

QUICK SUMMARY

- Controls expose properties
- `ObservableList` drives lists/tables
- `getSelectionModel()` — selection
- `Alert` and choosers for dialogs

JavaFX Events & Threading

HANDLING INPUT AND BACKGROUND WORK

JavaFX nodes fire **events** for actions, mouse, keyboard, and drag-and-drop. Register handlers with convenience setters like `setOnAction`, or with `addEventHandler` for multiple handlers. Events travel down (**filters**) then up (**handlers**) the scene graph, and `consume()` stops them. Long work runs in a `Task` so the UI stays responsive.

NOTES

- Handlers run on the JavaFX Application Thread
- `addEventFilter` sees events before children do
- `Task<T>` updates `progress` / `message` properties safely
- `Platform.runLater` posts code to the FX thread

EVENTS & TASKS

```
Button save = new Button("Save");
save.setOnAction(e -> saveFile());

Rectangle box = new Rectangle(100, 60);
box.setOnMouseClicked(e -> {
    if (e.getButton() == MouseButton.SECONDARY) showMenu(e.getScreenX());
});

scene.setOnKeyPressed(e -> {
    if (e.isControlDown() && e.getCode() == KeyCode.S) saveFile();
});
textField.addEventFilter(KeyEvent.KEY_TYPED, e -> {
    if (!e.getCharacter().matches("\\d")) e.consume();   // digits only
});

Task<List<String>> load = new Task<>() {
    @Override protected List<String> call() throws Exception {
        updateMessage("Downloading…");
        return fetchLines();                    // background thread
    }
};
load.setOnSucceeded(e -> list.getItems().setAll(load.getValue())); // FX
load.setOnFailed(e -> status.setText("Error: " + load.getException()));
status.textProperty().bind(load.messageProperty());
new Thread(load).start();
```

NOTES

- → `consume()` blocks the keystroke from reaching the field
- → `call()` runs off the FX thread
- → `setOnSucceeded` / `setOnFailed` run on the FX thread
- → Binding to `messageProperty` shows progress text

EVENT SETTERS

SETTER	EVENT
`setOnAction`	Button, menu, Enter in field
`setOnMouseClicked`	Mouse click
`setOnKeyPressed`	Key down
`setOnCloseRequest`	Window closing (Stage)

THREADING TOOLS

TOOL	USE
`Platform.runLater(r)`	Run on FX thread
`Task<T>`	One background job
`Service<T>`	Restartable job

⚠ COMMON MISTAKES

- Calling `list.getItems().add(...)` inside `call()` — UI change off the FX thread
- Blocking the FX thread with `Thread.sleep` or network calls
- Using a filter where a handler would do, accidentally consuming events for children

✓ TIPS & BEST PRACTICES

- Bind progress bars: `bar.progressProperty().bind(task.progres`
- Run tasks on an `ExecutorService` rather than raw threads
- Use `setOnCloseRequest` to confirm unsaved changes

QUICK SUMMARY

- `setOnX` / `addEventHandler`
- Filters ↓, handlers ↑, `consume()`
- `Task` for background work
- `Platform.runLater` for UI updates

Properties & Binding

OBSERVABLE STATE

JavaFX values are held in **properties**: observable wrappers that notify listeners when they change. **Binding** links one property to another so it updates automatically: `label.textProperty().bind(field.textProperty())` mirrors typing in real time. Bindings can be one-way, two-way, or computed from several values.

NOTES

- A bound property can't be set directly; call `unbind()` first
- Bidirectional bindings keep two properties in sync
- `Bindings` has helpers: `concat`, `when`, `createStringBinding`
- Listeners receive `(observable, oldValue, newValue)`

PROPERTIES, BINDINGS, LISTENERS

```
public class Person {                                   // property pattern
    private final StringProperty name = new SimpleStringProperty("");
    public StringProperty nameProperty() { return name; }
    public String getName() { return name.get(); }
    public void setName(String v) { name.set(v); }
}

TextField input = new TextField();
Label echo = new Label();
echo.textProperty().bind(input.textProperty());         // one-way

a.textProperty().bindBidirectional(b.textProperty());   // two-way

IntegerProperty qty = new SimpleIntegerProperty(2);
DoubleProperty price = new SimpleDoubleProperty(9.5);
NumberBinding total = qty.multiply(price);              // 19.0, auto-updates
totalLbl.textProperty().bind(Bindings.format("$%.2f", total));

Button save = new Button("Save");
save.disableProperty().bind(input.textProperty().isEmpty());   // computed

qty.addListener((obs, oldV, newV) -> System.out.println(oldV + " → " + newV));
```

NOTES

- → The property pattern: field + `xProperty()` + get/set
- → Fluent operations (`multiply`, `isEmpty`) build bindings
- → `Bindings.format` produces a bound string
- → The Save button enables itself when text is entered

PROPERTY TYPES

TYPE	HOLDS
`StringProperty`	String
`IntegerProperty / DoubleProperty`	Numbers
`BooleanProperty`	boolean
`ObjectProperty<T>`	Any object

BINDING METHODS

METHOD	EFFECT
`bind(obs)`	Follow another value
`bindBidirectional(p)`	Sync both ways
`unbind()`	Stop following
`Bindings.when(c).then(a).otherwise(b)`	Conditional
`Bindings.createStringBinding(fn, deps…)`	Custom computed

⚠ COMMON MISTAKES

- Calling `set()` on a bound property → `RuntimeException: A bound value cannot be set`
- Listeners or bindings that keep large objects alive; unbind when screens close
- Forgetting dependencies in `createXxxBinding`, so it never recomputes

✓ TIPS & BEST PRACTICES

- Bind `disableProperty` for form validation
- Expose properties from view models; bind controls to them
- Use `Bindings.isEmpty`, `greaterThan`, `and`, `or` for readable rules

QUICK SUMMARY

- Properties = observable values
- `bind` — one-way
- `bindBidirectional` — two-way
- Fluent / `Bindings` computed values

FXML

DECLARATIVE LAYOUTS IN XML

FXML describes a JavaFX scene graph in XML, separating layout from logic. `FXMLLoader` reads the file and builds the nodes. `fx:controller` names the Java class that handles events (Sheet 132), `fx:id` links elements to controller fields, and `#method` attributes link events to controller methods. **Scene Builder** edits FXML visually.

NOTES

- Requires the `javafx-fxml` artifact
- Store `.fxml` files under `src/main/resources`
- Modules must `opens` the controller package to `javafx.fxml`
- Element names are class names; attributes are properties

LOGIN.FXML

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<VBox xmlns:fx="http://javafx.com/fxml"
      fx:controller="com.acme.ui.LoginController"
      spacing="10" alignment="CENTER" prefWidth="300">
  <padding><Insets top="20" right="20" bottom="20" left="20"/></padding>
  <Label text="Sign in" styleClass="title"/>
  <TextField fx:id="userField" promptText="Username"/>
  <PasswordField fx:id="passField" promptText="Password"/>
  <Button text="Log in" defaultButton="true" onAction="#handleLogin"/>
  <Label fx:id="errorLabel" styleClass="error"/>
</VBox>
```

NOTES

- → `<?import?>` lines work like Java imports
- → `fx:id="userField"` → controller field `userField`
- → `onAction="#handleLogin"` → controller method
- → `defaultButton` fires on Enter

LOADING FXML

```
public void start(Stage stage) throws IOException {
    FXMLLoader loader = new FXMLLoader(
        getClass().getResource("/com/acme/ui/login.fxml"));
    Parent root = loader.load();
    LoginController ctrl = loader.getController();
    stage.setScene(new Scene(root));
    stage.show();
}
```

FXML ATTRIBUTES

ATTRIBUTE	PURPOSE
`fx:controller`	Controller class
`fx:id`	Inject node into a field
`onAction="#m"`	Event → method
`styleClass`	CSS class (Sheet 133)
`fx:include`	Embed another FXML file

⚠ COMMON MISTAKES

- Wrong resource path → `NullPointerException: Location is required`
- `fx:id` without a matching `@FXML` field (or a typo) → field stays `null`
- Forgetting `opens … to javafx.fxml` in `module-info.java` → `IllegalAccessException`

✓ TIPS & BEST PRACTICES

- Paths starting with `/` are relative to the classpath root
- Design in Scene Builder, then polish by hand
- Split big screens with `fx:include`

QUICK SUMMARY

- XML layout → `FXMLLoader.load()`
- `fx:controller` names the logic class
- `fx:id` → fields, `#method` → events
- Keep layout separate from code

FXML Controllers

THE LOGIC BEHIND FXML

An FXML **controller** is a plain class whose `@FXML`-annotated fields receive the nodes marked with `fx:id`, and whose `@FXML` methods handle events wired with `#method`. After injection, the loader calls `initialize()`, the place to set up bindings and listeners. Controllers keep event logic out of the layout file.

NOTES

- Fields and methods may be `private` when annotated `@FXML`
- The controller needs a public no-arg constructor (unless you set a factory)
- `initialize()` runs after all `fx:id` fields are injected

LOGINCONTROLLER.JAVA

```
public class LoginController {
    @FXML private TextField userField;
    @FXML private PasswordField passField;
    @FXML private Label errorLabel;   private AuthService auth;

    @FXML
    private void initialize() {                      // after injection
        errorLabel.setVisible(false);
        userField.textProperty()
                 .addListener((o, a, b) -> errorLabel.setVisible(false));
    }

    @FXML
    private void handleLogin(ActionEvent e) {         // onAction="#handleLogin"
        if (auth.login(userField.getText(), passField.getText())) {
            ((Stage) userField.getScene().getWindow()).close();
        } else {
            errorLabel.setText("Invalid login"); errorLabel.setVisible(true);
        }
    }

    public void setAuthService(AuthService a) { auth = a; }   // pass data
}

var loader = new FXMLLoader(getClass().getResource("login.fxml"));
Parent root = loader.load();
loader.<LoginController>getController().setAuthService(authService);
```

NOTES

- Field names must match `fx:id` values exactly
- Pass dependencies after `load()` with a setter, or use a controller factory

CONTROLLER WIRING

FXML	CONTROLLER
`fx:id="userField"`	@FXML TextField userField;
`onAction="#handleLogin"`	@FXML void handleLogin(ActionEvent e)
`(automatic)`	@FXML void initialize()

PASSING DATA BETWEEN SCREENS

APPROACH	HOW
`Setter after load`	loader.getController().setX(…)
`Controller factory`	loader.setControllerFactory(…)

⚠ COMMON MISTAKES

- Using controller fields in the constructor — they're still `null`; use `initialize()`
- Mismatched names between `fx:id` and the field
- Putting business logic in controllers instead of services or models

✓ TIPS & BEST PRACTICES

- Keep controllers thin: UI wiring only
- Use a shared view-model with properties for multi-screen state

QUICK SUMMARY

- `@FXML` fields ← `fx:id`
- `@FXML` methods ← `#handler`
- `initialize()` — post-injection setup
- Pass data via setters or factories

JavaFX CSS Styling

STYLING WITH CSS

JavaFX controls are styled with **CSS** using JavaFX-specific property names prefixed with `-fx-`. Add stylesheets to a `Scene` or `Parent`, select nodes by **style class**, **id**, or type, and react to states with **pseudo-classes** like `:hover` and `:focused`. The default theme is **Modena**, which you override rather than replace.

NOTES

- Properties use `-fx-` names: `-fx-background-color`, not `background-color`
- Control type selectors are lowercase-hyphenated: `.button`, `.text-field`
- Inline `setStyle` overrides stylesheet rules
- Looked-up colors (variables) like `-fx-accent` retheme everything

APP.CSS

```
.root {
    -fx-font-family: "Inter", "Segoe UI", sans-serif;
    -fx-font-size: 13px;
    -fx-accent: #5c6bc0;                /* looked-up color: theme accent */
    -fx-base: #20232f;                  /* dark base derives other colors */
}
.button {
    -fx-background-color: -fx-accent;
    -fx-text-fill: white;
    -fx-background-radius: 6;
    -fx-padding: 6 14;
}
.button:hover   { -fx-background-color: derive(-fx-accent, 20%); }
.button:pressed { -fx-background-color: derive(-fx-accent, -15%); }
.text-field:focused { -fx-border-color: -fx-accent; }
.error  { -fx-text-fill: #e57373; }
#title  { -fx-font-size: 20px; -fx-font-weight: bold; }
```

NOTES

- → `.root` styles the scene's root node
- → `derive(color, %)` lightens or darkens
- → `.error` is a custom style class
- → `#title` targets the node with id `title`

APPLYING STYLES

```
scene.getStylesheets().add(getClass()
    .getResource("/app.css").toExternalForm());
errorLabel.getStyleClass().add("error");
titleLabel.setId("title");
saveBtn.setStyle("-fx-font-weight: bold;");
var bad = PseudoClass.getPseudoClass("invalid");
field.pseudoClassStateChanged(bad, true);
```

COMMON -FX- PROPERTIES

PROPERTY	EXAMPLE
`-fx-background-color`	#fff, linear-gradient(…)
`-fx-text-fill`	white
`-fx-font-size / -fx-font-weight`	14px / bold
`-fx-padding`	4 8
`-fx-border-color / -fx-border-radius`	#ccc / 4
`-fx-effect`	dropshadow(gaussian, #0004, 8, 0, 0, 2)

⚠ COMMON MISTAKES

- Using web CSS names without `-fx-` — they're silently ignored
- Wrong stylesheet path → no styles and only a console warning
- Hard-coding colors everywhere instead of overriding `-fx-accent` / `-fx-base`

✓ TIPS & BEST PRACTICES

- Add `stylesheets="@app.css"` to FXML roots
- Use custom pseudo-classes for validation states
- AtlantaFX offers modern ready-made themes

QUICK SUMMARY

- `-fx-` property names
- Selectors: `.class`, `#id`, `:hover`
- `scene.getStylesheets().add(…)`
- Override Modena's looked-up colors

JavaFX Charts & Media

VISUALIZING DATA AND PLAYING MEDIA

JavaFX includes ready-made **charts** in `javafx.scene.chart`: line, bar, area, scatter, bubble, and pie. XY charts take one or more `XYChart.Series` of data points plotted against two axes. The `javafx-media` module adds `Media`, `MediaPlayer`, and `MediaView` for audio and video playback.

NOTES

- Charts are nodes; add them to any layout
- Updating a series' data list updates the chart live
- Media requires the `javafx-media` artifact
- Supported media: MP3, WAV, AAC, MP4 (H.264), HLS streams

CHARTS & MEDIA

```
NumberAxis x = new NumberAxis(1, 12, 1);
NumberAxis y = new NumberAxis();
x.setLabel("Month");  y.setLabel("Sales ($k)");
LineChart<Number, Number> line = new LineChart<>(x, y);

XYChart.Series<Number, Number> s2026 = new XYChart.Series<>();
s2026.setName("2026");
s2026.getData().addAll(new XYChart.Data<>(1, 23),
                       new XYChart.Data<>(2, 31));
line.getData().add(s2026);

var bar = new BarChart<String, Number>(new CategoryAxis(), new NumberAxis());
XYChart.Series<String, Number> q = new XYChart.Series<>();
q.getData().add(new XYChart.Data<>("Q1", 120));
bar.getData().add(q);

PieChart pie = new PieChart(FXCollections.observableArrayList(
    new PieChart.Data("Java", 45), new PieChart.Data("Kotlin", 30)));

Media media = new Media(Path.of("intro.mp4").toUri().toString());
MediaPlayer player = new MediaPlayer(media);
MediaView view = new MediaView(player);
player.setOnReady(player::play);
```

NOTES

- → Numeric axes take lower bound, upper bound, tick unit
- → One series = one line or bar group; add several to compare
- → `Media` needs a URI string, not a file path
- → `setOnReady` waits until media metadata loads

CHART TYPES

CHART	AXES
`LineChart / AreaChart`	X, Y
`BarChart / StackedBarChart`	Category, Number
`ScatterChart / BubbleChart`	Number, Number
`PieChart`	None (slices)

MEDIAPLAYER CONTROLS

METHOD	EFFECT
`play() / pause() / stop()`	Playback
`seek(Duration)`	Jump to time
`setVolume(0.0–1.0)`	Volume
`setCycleCount(INDEFINITE)`	Loop
`currentTimeProperty()`	Bind a progress slider

⚠ COMMON MISTAKES

- Passing a plain file path to `new Media(...)` → `MediaException`; use `toUri().toString()`
- Re-creating a whole chart for new data instead of updating its series
- Forgetting the `javafx-media` dependency or module requirement

✓ TIPS & BEST PRACTICES

- Call `chart.setAnimated(false)` when updating data rapidly
- Style charts with CSS: `.chart-series-line { -fx-stroke: … }`
- Keep a reference to `MediaPlayer` so it isn't garbage-collected mid-playback

QUICK SUMMARY

- `XYChart.Series` + `Data` points
- Line, bar, pie, scatter charts
- `Media` → `MediaPlayer` → `MediaView`
- URIs for media sources

Sheet 135

Quick Reference — Syntax, Types & OOP

program structure · types & casting · strings · operators · control flow & switch · exceptions · methods & lambdas · classes, records, sealed

STRUCTURE & OUTPUT

SYNTAX	DOES	#
`public static void main(String[] a)`	entry point	002
`void main() { }`	compact source file (25)	002
`System.out.println(x)`	print line	003
`System.out.printf("%d%n", n)`	formatted	003
`var list = new ArrayList<String>()`	local inference	005

TYPES & CASTING

SYNTAX	DOES	#
`int / long / double / boolean`	primitives	006
`(int) 3.9`	narrowing cast → 3	007
`Integer.parseInt("42")`	parse	007
`Integer / Double`	wrapper types	008
`final int MAX = 10;`	constant	005

STRINGS

SYNTAX	DOES	#
`s.equals(t)`	compare content	014
`s.substring(1, 4)`	slice	015
`s.split(",") / String.join`	split / join	015
`"%s=%d".formatted(k, v)`	format	016
`""" text block """`	multiline string	019
`new StringBuilder().append()`	build efficiently	090

OPERATORS

OPERATOR	DOES	#
`/ and %`	int division truncates	009
`== vs equals`	reference vs value	010
`&& || !`	short-circuit logic	011
`& | ^ ~ << >> >>>`	bitwise	012
`a ? b : c`	ternary	022

CONTROL FLOW

SYNTAX	DOES	#
`if / else if / else`	branch	021
`for (int i = 0; i < n; i++)`	counter loop	025
`for (var x : list)`	enhanced for	028
`break / continue / label:`	loop control	029
`switch (x) { case 1 -> … }`	arrow switch	024
`case Circle c when c.r() > 1`	pattern + guard	024

EXCEPTIONS

SYNTAX	DOES	#
`try { } catch (IOException e)`	catch	030
`finally { }`	always runs	033
`try (var r = open()) { }`	try-with-resources	033
`throws IOException`	declare checked	031
`class MyEx extends RuntimeException`	custom	032

METHODS & LAMBDAS

SYNTAX	DOES	#
`static int add(int a, int b)`	method	034
`void log(String... msgs)`	varargs	043
`x -> x * 2`	lambda	040
`String::length`	method reference	041
`Function / Predicate / Supplier`	functional types	040

CLASSES

SYNTAX	DOES	#
`class Dog extends Animal`	inheritance	062
`implements Runnable`	interface	069
`super(name)`	parent constructor	063
`@Override`	check overriding	064
`abstract class / method`	must subclass	068
`private + getters`	encapsulation	066

MODERN TYPES

SYNTAX	DOES	#
`record Point(int x, int y) {}`	immutable data	072
`enum Color { RED, GREEN }`	enum	071
`sealed interface S permits A, B`	closed hierarchy	073
`if (o instanceof String s)`	pattern instanceof	065
`default void m() { }`	interface default	069

⚠ COMMON MISTAKES

- `==` on strings or boxed numbers — use `equals`
- Integer division `5 / 2 == 2`; cast first for decimals
- Swallowing exceptions with an empty `catch`

✓ TIPS & BEST PRACTICES

- Use records for data carriers and `var` for obvious local types
- Prefer arrow `switch` expressions — no fall-through
- Close resources with try-with-resources

QUICK SUMMARY

- Static types, classes everywhere, `main` starts the program
- `equals` for value equality
- `switch` + patterns + records + sealed = modern data modelling
- Lambdas and method refs for functional code

Sheet 136

Quick Reference — Collections, Streams & APIs

collections · maps & sets · generics · streams & collectors · sorting · Optional · dates · files · build tools · testing · concurrency · JDBC

LISTS & QUEUES

SYNTAX	DOES	#
`List.of(1, 2, 3)`	immutable list	045
`new ArrayList<>(list)`	mutable copy	045
`list.add / get / remove`	core ops	045
`ArrayDeque push / pop`	stack	051
`offer / poll / peek`	queue	050
`PriorityQueue`	min-heap	050

MAPS & SETS

SYNTAX	DOES	#
`Map.of("a", 1)`	immutable map	047
`map.getOrDefault(k, 0)`	safe lookup	047
`map.merge(k, 1, Integer::sum)`	count	047
`map.computeIfAbsent(k, …)`	group into lists	047
`new HashSet<>() / contains`	set	048
`TreeMap / TreeSet`	sorted	049

GENERICS

SYNTAX	DOES	#
`class Box<T>`	generic class	052
`<T> T first(List<T> xs)`	generic method	052
`<T extends Comparable<T>>`	bound	052
`List<? extends Number>`	read-only wildcard	052
`List<? super Integer>`	write wildcard	052

STREAMS

SYNTAX	DOES	#
`list.stream()`	start	053
`.filter(x -> x > 0)`	keep	054
`.map(String::trim)`	transform	054
`.reduce(0, Integer::sum)`	fold	055
`.toList()`	collect to list	055
`Collectors.groupingBy(…)`	group	055
`Collectors.joining(", ")`	join strings	055

SORTING & OPTIONAL

SYNTAX	DOES	#
`Collections.sort(list)`	natural order	095
`list.sort(Comparator.comparing(P::age))`	by key	101
`.thenComparing(P::name)`	tie-break	101
`.reversed()`	descending	101
`Optional.ofNullable(x)`	maybe value	097
`.orElse(d) / .map(f)`	use it	097

DATES & TIME

SYNTAX	DOES	#
`LocalDate.now()`	today	091
`LocalDate.of(2026, 10, 17)`	build a date	091
`date.plusDays(7)`	arithmetic	091
`Duration / Period`	time spans	093
`ZonedDateTime`	with time zone	093
`DateTimeFormatter.ofPattern`	format / parse	094

FILES & DATA

SYNTAX	DOES	#
`Files.readString(path)`	read file	078
`Files.readAllLines(path)`	lines	078
`Files.writeString(p, s)`	write	078
`Path.of("a", "b.txt")`	build path	079
`Files.walk(dir)`	walk tree	079
`Jackson ObjectMapper`	JSON	109

BUILD & TEST

ITEM	DOES	#
`mvn package / test`	Maven	086
`./gradlew build`	Gradle	087
`@Test / assertEquals`	JUnit 5	102
`@ParameterizedTest`	data-driven	103
`when(m.x()).thenReturn(y)`	Mockito	104
`log.info("x={}", x)`	SLF4J	105

CONCURRENCY & JDBC

API	DOES	#
`Executors.newVirtualThreadPerTaskExecutor()`	virtual threads	116
`CompletableFuture.supplyAsync`	async	114
`synchronized / ReentrantLock`	locking	115
`ConcurrentHashMap`	thread-safe map	115
`DriverManager.getConnection`	JDBC	117
`PreparedStatement`	safe SQL params	119

⚠ COMMON MISTAKES

- Modifying a list while looping over it (`ConcurrentModificationException`)
- `List.of` / `Map.of` are immutable — `add` throws
- Building SQL by string concatenation instead of `PreparedStatement`

✓ TIPS & BEST PRACTICES

- Program to interfaces: `List` , `Map` , `Set`
- Use `java.time` , never `Date` / `Calendar` , for new code
- Use virtual threads for blocking I/O on Java 21+

QUICK SUMMARY

- ArrayList / HashMap / HashSet are the defaults
- Streams: `filter` → `map` → `collect`
- `Comparator.comparing` for sorting
- `Files` + `Path` for I/O; JUnit + Mockito for tests